Advanced Machine Learning

Non scholae, sed vitae discimus

Vincenzo Salvatore

Raffaele Mercurio

Contents

1 Foundations of Deep Learning and Optimization 9

1.1 Introduction to the Core Concepts 9

1.2 Parametric Models and Optimization Foundations 10

1.2.1 Linear Regression ($f : \mathbb{R}^D \to \mathbb{R}$) 10

1.2.2 Logistic Regression ($f : \mathbb{R}^D \to [0, 1]$) 10

1.2.3 Softmax Classifier ($f : \mathbb{R}^D \to \{0, \ldots, K\}$) 11

1.3 Deep Feedforward Networks and Backpropagation 13

1.3.1 Deep Neural Networks 13

1.3.2 Backpropagation 13

1.4 The Role of Activation Functions 17

1.4.1 ReLU (Rectified Linear Unit) 17

1.4.2 Leaky ReLU 17

1.4.3 ELU (Exponential Linear Units) 18

1.4.4 GELU (Gaussian Error Linear Unit) 18

1.5 Weight Initialization and Signal Propagation 19
1.5.1 The Pitfalls of Naive Initialization . . . 19
1.5.2 Variance Scaling Methods: The Solution . . . 19
1.6 Gradient-Based Optimization Algorithms 20
1.7 Regularization in Deep Neural Networks 21
1.7.1 Dropout . . . 21
1.7.2 The Common Pattern of Modern Regularization . . . 22
1.8 Feature Normalization Techniques 22
1.8.1 Batch Normalization . . . 23
1.8.2 Alternative Normalization Methods . . . 23
1.9 Chapter Summary and Conceptual Graph 24

2 Convolutional Neural Networks and Computer Vision . . . 27

2.1 Image Representation and the Imaging Process 27
2.2 Motivations for Convolutions 28
2.2.1 The Problem with Fully Connected Layers . . . 28
2.2.2 Exploiting Image Structure: Locality and Stationarity . . . 28
2.3 The Core Building Block: The Convolutional Layer 28
2.3.1 Key Hyperparameters of the Convolution . . . 29
2.3.2 Formal Mathematical Definition . . . 30
2.4 Classic Image Processing Filters 30
2.5 Output Shape and Parameter Counting for 3D Volumes 32
2.5.1 Spatial Output Size Formula . . . 32
2.6 The Pooling Layer 32
2.6.1 Purpose and Properties of Pooling . . . 32
2.6.2 Max Pooling . . . 32
2.6.3 Average Pooling . . . 33
2.7 Visualizing Convolutional Networks 33
2.8 Vision Regularization: Data Augmentation 33
2.8.1 Classic Spatial and Color Augmentations . . . 34
2.8.2 Advanced Stochastic Augmentations . . . 34
2.9 Transfer Learning in Computer Vision 35
2.9.1 The Fundamental Concept (Why Use It?) . . . 35
2.9.2 Practical Implementation . . . 35
2.9.3 Pervasiveness: The Modern Standard . . . 35
2.10 Chapter Summary and Conceptual Graph 36

3 Sequence Models . . . 39

3.1 Introduction to Sequence Modeling 39
3.1.1 Taxonomy of Sequence Tasks . . . 40
3.1.2 Forecasting as a Core Sequence Problem . . . 41

3.2 Temporal Convolutional Networks 41
3.2.1 One-Dimensional Convolutions on Sequences . . . 41
3.2.2 Causal and Dilated Convolutions . . . 42
3.2.3 Trade-offs of Temporal Convolutional Networks . . . 44

3.3 Recurrent Neural Networks 45
3.3.1 The Recurrence Principle . . . 45
3.3.2 The Vanilla RNN Formulation . . . 45
3.3.3 Unrolling the Computational Graph . . . 46
3.3.4 Sequence-to-Sequence: Synchronized vs. Encoder-Decoder Mappings . . . 46
3.3.5 Case Study: Character-Level Language Modeling . . . 48
3.3.6 Case Study: Image Captioning (CNN + RNN) . . . 50

3.4 Backpropagation Through Time and Gradient Flow 51
3.4.1 BPTT and Gradient Flow in Vanilla RNNs . . . 51
3.4.2 Exploding and Vanishing Gradients . . . 52
3.4.3 Vanilla RNN Trade-offs . . . 52

3.5 Long Short-Term Memory 53
3.5.1 The Dual-State Architecture . . . 53
3.5.2 The LSTM Cell Architecture . . . 53
3.5.3 Detailed Breakdown of Cell Components . . . 54
3.5.4 Why LSTM Solves the Vanishing Gradient Problem . . . 55

3.6 Chapter Summary and Conceptual Graph 56

4 Attention Mechanisms and Transformer Networks . . . 59

4.1 Introduction: Transcending the Recurrent Bottleneck 59

4.2 From Recurrence to Attention 60
4.2.1 The Information Bottleneck in Vanilla Seq2Seq . . . 60
4.2.2 The Bahdanau Additive Attention Mechanism . . . 60
4.2.3 Step-by-Step Walkthrough: Dynamic Alignment at Decoding Step $t = 2$ . . . 61
4.2.4 Interpreting Attention Matrices and Word Reordering . . . 64
4.2.5 Spatial Attention in Computer Vision: Show, Attend and Tell . . . 64

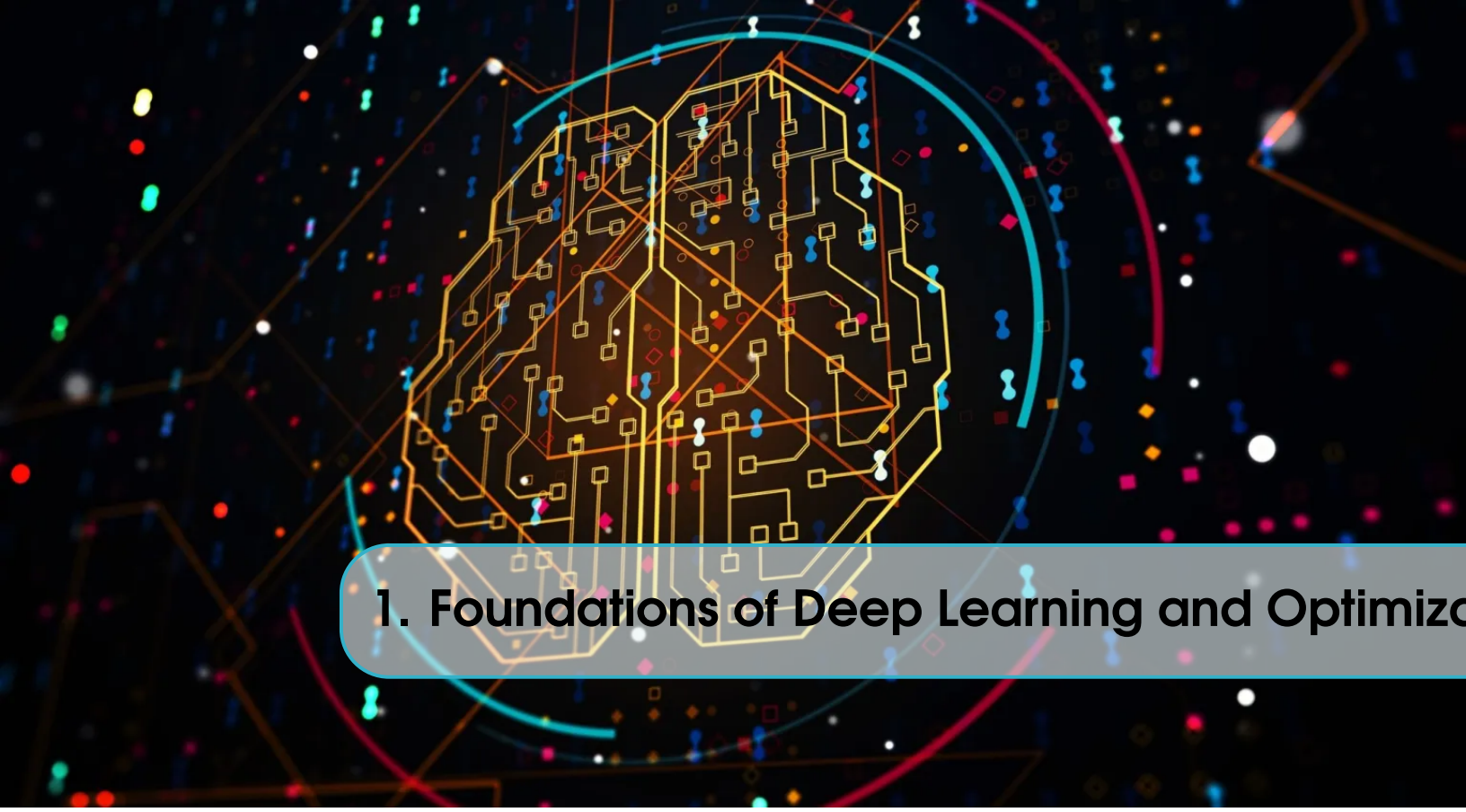
4.3 Self-Attention and Scaled Dot-Product Mechanics 65
4.3.1 From General Attention to Self-Attention . . . 65
4.3.2 The Query, Key, Value (Q, K, V) Formulation . . . 65
4.3.3 Maximum Path Length and Computational Complexity . . . 66

4.4 Positional Encodings, Masking, and Multi-Head Attention 68
4.4.1 Permutation Equivariance: Why Pure Attention Ignores Order . . . 68
4.4.2 Positional Encodings (PE) . . . 68
4.4.3 Masked Self-Attention (Causal Attention) . . . 69
4.4.4 Multi-Head Self-Attention (MHSA) . . . 70
4.5 The Transformer Architecture 71
4.5.1 The Transformer Encoder Block . . . 71
4.5.2 The Transformer Decoder Block . . . 72
4.5.3 Comparative Summary: RNNs versus Transformers . . . 72
4.6 Transformers in Computer Vision 74
4.6.1 Visual Self-Similarity and Non-Local Filtering . . . 74
4.6.2 The Evolution of Vision-to-Language Pipelines . . . 74
4.6.3 Vision Transformers (ViT) . . . 74
4.7 Pre-training, BERT, and Model Compression 76
4.7.1 The "ImageNet Moment" for Natural Language Processing . . . 76
4.7.2 BERT: Bidirectional Encoder Representations from Transformers . . . 76
4.7.3 ELECTRA: Efficient Pre-training as Discriminators . . . 77
4.7.4 Knowledge Distillation and Model Compression . . . 78
4.8 Chapter Summary and Conceptual Graph 79

5 Self-Supervised Learning . . . 81

5.1 Introduction: Transcending the Supervised Bottleneck 81
5.2 Foundations of Self-Supervised Learning 82
5.2.1 The Pretext-Downstream Paradigm . . . 82
5.2.2 Evaluation Protocols for Learned Representations . . . 82
5.3 Heuristic Pretext Tasks 83
5.3.1 Predicting Geometric Rotations . . . 83
5.3.2 Spatial Context: Relative Patch Location and Jigsaw Puzzles . . . 84
5.3.3 Context Encoders: Feature Learning by Inpainting . . . 86
5.3.4 Image and Video Colorization . . . 86
5.4 Masked Autoencoders (MAE) 89
5.4.1 From Convolutional Inpainting to Vision Transformers . . . 89
5.4.2 The Asymmetric Encoder-Decoder Architecture . . . 89
5.4.3 Reconstruction Objective and Normalization . . . 90
5.4.4 Linear Probing vs. Fine-Tuning in Generative SSL . . . 90
5.5 Contrastive Representation Learning 90
5.5.1 Instance Discrimination and Hyperspherical Geometry . . . 90
5.5.2 The InfoNCE Loss Formulation . . . 91

5.5.3 SimCLR: A Simple Framework for Contrastive Learning 91
5.5.4 Momentum Contrast (MoCo and MoCo v2) 92
5.6 Non-Contrastive Siamese Architectures 94
5.6.1 The Representation Collapse Problem 94
5.6.2 BYOL: Bootstrap Your Own Latent 94
5.6.3 SimSiam: Exploring Simple Siamese Representation Learning 95
5.6.4 DINO: Self-Distillation with No Labels 96
5.7 Chapter Summary and Conceptual Graph 97
5.7.1 Comparative Summary of SSL Paradigms 97
5.7.2 Logical Roadmap of Representation Learning 98



1. Foundations of Deep Learning and Optimiza

1.1 Introduction to the Core Concepts

The contents of this chapter establish the theoretical and mathematical foundations of modern deep learning, largely based on the introductory modules of the course [23, 27].

Key Word

- According to Arthur Samuel [48], **Machine Learning** is the field of study that gives computers the ability to learn without being explicitly programmed.
- Tom Mitchell (1998) defined the well-posed learning problem: a computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.

Machine learning is heavily applied in **Computer Vision**, which can be viewed from three main perspectives: as a science (exploring the computational model of human perception), as engineering (building systems that perceive the world), and through its applications.

Example

Some typical **Computer Vision Applications** include:

- **Medical imaging**: Supporting medical diagnosis and visualization.
- **Surveillance**: Tracking people in airports or train stations.
- **Autonomous driving**: Lane-keeping and pre-crash intervention.

—

1.2 Parametric Models and Optimization Foundations

The majority of machine learning models can be formulated as **Parametric Models**. A parametric model takes an input x and uses a set of parameters (or weights) W and biases b to produce an output prediction z:

$$z = f(x, W) = Wx + b$$

While the exact nature of the prediction depends on the task, what unifies these models is the **Optimization Process**. We define a Loss function L that measures the error of our predictions, and we iteratively adjust the weights using **Gradient Descent**:

$$W^{t+1} = W^t - \alpha \nabla_W L$$

where α is the learning rate.

By simply changing the *activation function* applied to the linear score z and the corresponding *loss function*, we obtain different classical models: Linear Regression, Logistic Regression, and the Softmax Classifier.

1.2.1 Linear Regression ($f : \mathbb{R}^D \to \mathbb{R}$)

Linear Regression is used when the target variable is continuous. We use the identity activation function (i.e., no activation), so the predicted output $\hat{y}$ is exactly the linear score:

$$\hat{y} = z = Wx + b$$

We optimize this model by minimizing the **Mean Squared Error (MSE)** loss:

$$L = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

1.2.2 Logistic Regression ($f : \mathbb{R}^D \to [0, 1]$)

Logistic Regression is used for binary classification. We map the linear score z to a probability between 0 and 1 using the **Sigmoid** activation function $\sigma(z)$:

$$P(Y = 1 | X = x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The appropriate loss function for this model is the **Binary Cross-Entropy Loss (BCE)**. For a dataset of N samples, it is defined as:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\left[\underbrace{y_i \log(\sigma(z_i))}_{\text{Active if } y=1} + \underbrace{(1-y_i)\log(1-\sigma(z_i))}_{\text{Active if } y=0}\right]$$

Note

Understanding the BCE Formula: This equation uses a clever mathematical trick to combine two mutually exclusive cases into a single line:

- When the true class is $y_i = 1$, the second term $(1-1) = 0$ disappears, and the loss evaluates only $-\log(\sigma(z_i))$, penalizing the model if it predicts a low probability.
- When the true class is $y_i = 0$, the first term 0 disappears, and the loss evaluates only $-\log(1-\sigma(z_i))$, penalizing the model if it predicts a high probability for class 1.

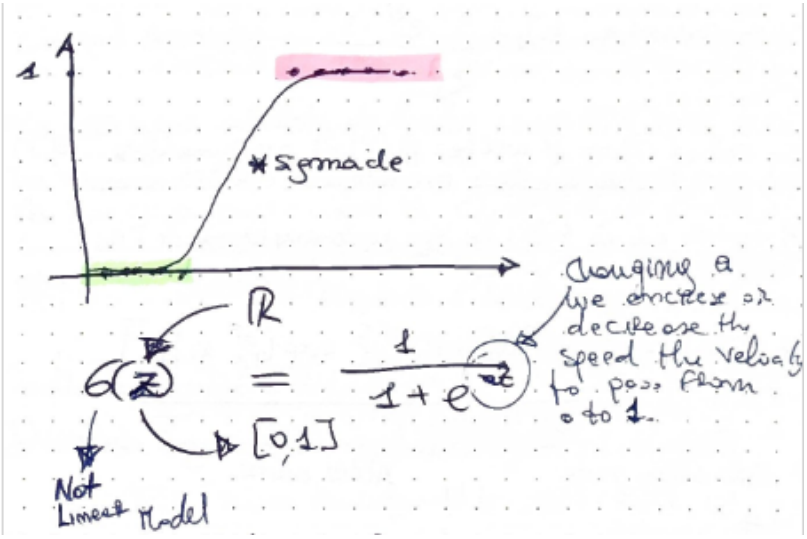


Figure 1.1: Illustration of the Sigmoid function in Logistic Regression.

1.2.3 Softmax Classifier ($f : \mathbb{R}^D \to \{0,\ldots,K\}$)

The Softmax Classifier generalizes Logistic Regression to multiple, mutually exclusive classes. It maps the raw scores z into a valid probability distribution over the classes using the **Softmax function**:

$$P(Y = k|X = x_i) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Note

Why Softmax? We want to interpret raw classifier scores as *probabilities*. The Softmax function transforms scores such that they are strictly positive and sum to 1.

Example

For an input image of size $32 \times 32 \times 3$ (e.g., CIFAR-10), the flattened array x has 3072 numbers. If we are classifying into 10 categories, the weight matrix W will have dimensions 10×3072, producing a 10×1 vector of raw class scores before the Softmax is applied.

We then optimize the model using the **Categorical Cross-Entropy Loss**, often referred to as the Negative Log-Likelihood (NLL). Over the entire dataset of N samples, expanding the Softmax probability formula, the loss becomes:

$$NLL(W,X) = -\sum_{i=1}^{N} \left[\underbrace{W_{y_i}^T x_i}_{\text{Punteggio della classe giusta}} - \underbrace{\log\left(\sum_{k=1}^{K} \exp(W_k^T x_i)\right)}_{\text{Somma dei punteggi delle altre classi}} \right]$$

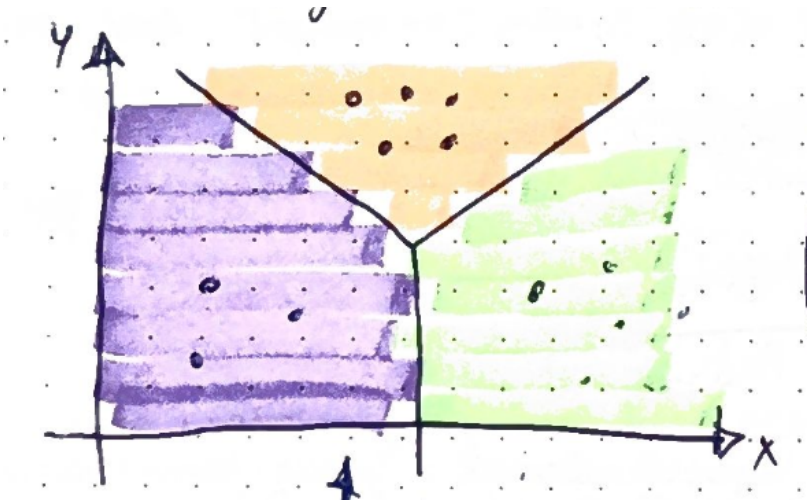


Figure 1.2: Abstract representation of logistic regression.

Note

Understanding the Softmax NLL Formula: By looking at the expanded formula, we see that the optimizer tries to maximize the linear score $W_{y_i}^T x_i$ of the *correct class* y_i, while simultaneously pushing down the log-sum of the scores of *all available classes*. This creates a competitive environment where increasing the score of the correct class implicitly decreases the relative probability of the others.

—

1.3 Deep Feedforward Networks and Backpropagation

1.3.1 Deep Neural Networks

By stacking multiple layers and applying non-linear activation functions, we construct multi-layer Neural Networks (Multi-Layer Perceptrons, MLPs):

$$f = W_2 \max(0, W_1 x + b_1) + b_2$$

This hierarchical representation enables the network to learn rich, non-linear representations from raw data.

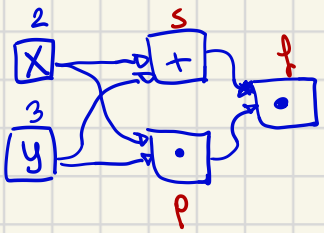
1.3.2 Backpropagation

To train a deep network, we compute the gradient of the loss with respect to all parameters using the **Backpropagation** algorithm [47], which systematically applies the multivariable chain rule from the output back to the inputs (backward pass).

Note

Backpropagation with Matrices: When inputs and weights are vectors or matrices, local gradients become *Jacobian matrices*. However, Jacobians are often sparse. Instead of explicitly forming them, practical frameworks use implicit vector-Jacobian products. A crucial dimensional consistency check to remember: the downstream gradient $\frac{\partial L}{\partial x}$ **always has the exact same shape** as the tensor x!

Here are some solved exercises demonstrating the Forward and Backward pass step-by-step:



FORWARD PASS

$s = x+y = 2+3 = 5$

$p = x \cdot y = 2 \cdot 3 = 6$

$f = s \cdot p = 30$

Reverse Pass

- $f = s \cdot p$ → $\frac{\partial f}{\partial s} = p$, $\frac{\partial f}{\partial p} = s$
- $p = x \cdot y$ → $\frac{\partial p}{\partial x} = y$, $\frac{\partial p}{\partial y} = x$
- $s = x+y$ → $\frac{\partial s}{\partial x} = 1+0 = 1$, $\frac{\partial s}{\partial y} = 0+1 = 1$

Applico la chain Rule

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial s} \cdot \frac{\partial s}{\partial x} + \frac{\partial f}{\partial p} \cdot \frac{\partial p}{\partial x} = p \cdot 1 + s \cdot y = 6 \cdot 1 + 5 \cdot 3 = 21$$

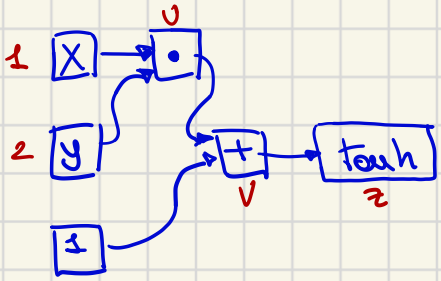
$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial s} \cdot \frac{\partial s}{\partial y} + \frac{\partial f}{\partial p} \cdot \frac{\partial p}{\partial y} = p \cdot 1 + s \cdot x = 6 \cdot 1 + 5 \cdot 2 = 16$$

Cosa capisco da questo esercizio? Capisco quanto un Peso x e y impattano rispetto alla funzione di Loss f. Vedo che X ha un impatto maggiore.

Esercizio 3

Consider a 2-Layer function: $z = \tanh(xy+1)$ $x=1$ e $y=2$

- Build a computation graph:
- forward pass: compute z.
- Reverse pass: compute gradient $\frac{\partial z}{\partial x}, \frac{\partial z}{\partial y}$.



FORWARD:

- $u = x \cdot y = 2$
- $v = u+1 = 3$
- $z = \tanh(v) = \tanh(3) \simeq 0{,}995$

Reverse PASS

- $z = \tanh(v) \rightsquigarrow \frac{\partial z}{\partial v} = \operatorname{sech}^2(v) = 1 - \tanh^2(v) \simeq 1 - (0.995)^2 \simeq 0.01$
- $v = u+1 \rightsquigarrow \frac{\partial v}{\partial u} = 1$
- $u = x \cdot y \rightsquigarrow \frac{\partial u}{\partial x} = y = 2$, $\frac{\partial u}{\partial y} = x = 1$
- Chain Rule

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial v} \cdot \frac{\partial v}{\partial u} \cdot \frac{\partial u}{\partial x} = 0{,}01 \cdot 1 \cdot 2 \simeq 0{,}02$$

$$\frac{\partial z}{\partial y} = \frac{\partial z}{\partial v} \cdot \frac{\partial v}{\partial u} \cdot \frac{\partial u}{\partial y} = 0{,}01 \cdot 1 \cdot 1 \simeq 0{,}01$$

Esercizio 4

given: $g(w) = (w-3)^2$ $w_0 = 0$ $\alpha = 0{,}1$

$\boxed{g'(w) = 2(w-3)}$

$$w^k = w^{k-1} - \alpha \nabla g'(w^{k-1})$$

① WRITE the gradient descent Rule
② Compute the first 5 iter... $w_1 \dots w_5$
③ sKetch the cost function values.

$w_{k+1} = w_k - \alpha g(w_k) =$
$w_{k+1} = w_k - 0,2(w_k - 3) =$
$w_{k+1} = w_k - 0,2 w_k + 0,6$
$w_{k+1} = (1-0,2) w_k + 0,6$
$w_{k+1} = 0,8 w_k + 0,6$

Notiamo come solo lo 0,8% del vecchio valore è mantenuto

$w_0 = 0$
$w_1 = 0,8 \cdot 0 + 0,6 = 0,6$
$w_2 = 0,8 \cdot 0,6 + 0,6 = 1,08$
$w_3 = 0,8 \cdot 1,08 + 0,6 = 1,464$
$w_4 = 0,8 \cdot 1,464 + 0,6 = 1,7712$
$w_5 = 0,8 \cdot 1,7712 + 0,6 = 2,01696$

Ad ogni passo l'iterazione ci spinge un po più vicino al minimo reale $w^* = 3$. La velocità con cui ci avviciniamo a 3 diminuisce gradualmente.

Esercizio 5 $g(w_1, w_2) = w_1^2 + 2w_2^2$

① Compute the gradient $\nabla g(w)$
② Apply GD from initial point $(w_1, w_2) = (2, 1)$ with learning rate $\alpha = 0,2$
③ Compute the first 3 update explicitly.

for computing the gradient $\nabla g(w_1, w_2)$ we have to apply the first derivate to the function $g(w_1, w_2) \rightsquigarrow g(w_1, w_2) = 2w_1 + 4w_2$

$$\nabla g(w_1, w_2) = \begin{bmatrix} 2w_1 \\ 4w_2 \end{bmatrix}$$ ← Il gradiente è la derivata della funzione.

Rule for each weight:

w_1: $w_1^{k+1} = w_1^k - \alpha \nabla g(w_1) =$
$w_1^{k+1} = w_1^k - 0,2(2w_1^k) =$
$w_1^{k+1} = w_1^k - 0,4 w_1^k =$
$\boxed{w_1^{k+1} = 0,6 w_1^k}$

w_2: $w_2^{k+1} = w_2^k - \alpha \nabla g(w_2^k) =$
$w_2^{k+1} = w_2^k - 0,2(4w_2^k)$
$w_2^{k+1} = w_2^k - 0,8 w_2^k =$
$\boxed{w_2^{k+1} = 0,2 w_2^k}$

Questo è il modo in cui cambieremo i Pesi.

Let's compute the iteration starting with $(2, 1)$
$K=1 \rightsquigarrow (w_1, w_2) = (0.6 \cdot 2, 0.2 \cdot 1)^0 = (1.2, 0.2)$
$K=2 \rightsquigarrow (1.2, 0.2) = (0,72, 0,04)$
$K=3 \rightsquigarrow (0,72, 0.04) = (0.432, 0.008)$

Obiettivo: Vogliamo capire come la funzione di LOSS guida l'aggiornamento dei Pesi per far sì che la nostra Rete produca output sempre più vicini a quelli desiderati.

EX.

Immagina di avere un modello lineare usiamo ad esempio $(x, y) = (2, 5)$

$\hat{y} = w \cdot x + b$ (peso, bias, input, output predetto)

PASSO 1: Inizializzo i PESI

Pesi iniziali: $w = 0,5$ $b = 0,0$

! noi non sappiamo nulla quindi impostiamo valori casuali.

PASSO 2: FORWARD PASS (PREDIZIONE)

$\hat{y} = w \cdot x + b = 0.5 \cdot 2 + 0 = 1$ ← PREDIZIONE

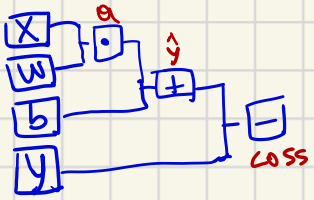
PASSO 3: calcolo della LOSS (ERRORE). usiamo MSE

$L = \frac{1}{2}(\hat{y} - y)^2 = \frac{(1-5)^2}{2} = \frac{16}{2} = 8$ ← siamo molto lontani dal TARGET.

PASSO 4: Backpropagation (calcolo del GRADIENTE): come cambiamo i Pesi ??

❗ Deriviamo la Loss Rispetto a $\hat{y}$

GRAFO COMPUT:



$\begin{cases} a = w \cdot x \\ \hat{y} = a + b \\ L = \hat{y} - y \end{cases}$

BACKWARD PASS

- $\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$
- $\frac{\partial \hat{y}}{\partial a} = 1$
- $\frac{\partial a}{\partial w} = x$
- $\frac{\partial \hat{y}}{\partial b} = 1$

w ↦ $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a} \cdot \frac{\partial a}{\partial w} = (\hat{y} - y) \cdot x = (-4) \cdot 2 = \boxed{-8}$

b ↦ $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b} = (\hat{y} - y) \cdot 1 = \boxed{-4}$

PASSO 5: GD update dei Pesi. Regola di aggiornamento: $w := w - \alpha \frac{\partial L}{\partial w}$

$b := b - \alpha \frac{\partial L}{\partial b}$

$\alpha = 0{,}1$

$w^1 = 0.5 - 0.1 \cdot (-8) = 1{,}3$ NUOVI PESI K=1

$b^1 = 0 - 0.1 \cdot (-4) = 0{,}4$

PREDICT

$\hat{y} = w' x + b' = 1{,}3 \cdot 2 + 0{,}4 = 3$ Nota come la predizione diventa più precisa e vicino al 5 del DATASET.

Ovviamente questo Processo viene eseguito su ogni Riga del Dataset. Non sulla stessa.

—

1.4 The Role of Activation Functions

An **activation function** is a mathematical operator applied to the affine output of a layer ($z = Wx + b$) before passing it to subsequent layers. Its primary role is to introduce **non-linearity** into the network.

Note

Why do we need Non-Linearity? If we only stacked linear layers without activation functions, the entire deep network would mathematically collapse into a single linear transformation. For example, a 2-layer linear network $f = W_2(W_1 x)$ can be rewritten as $f = W'x$ where $W' = W_2 W_1$. Non-linear activations allow the network to approximate arbitrarily complex, non-linear decision boundaries.

Some of the most widely used activation functions are illustrated in Figure 1.3:

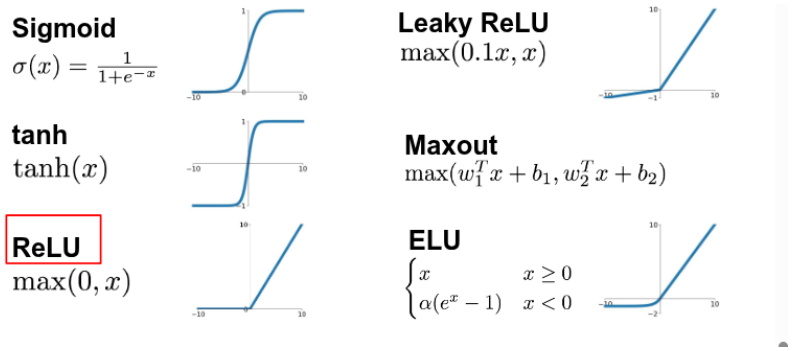


Figure 1.3: Illustration of common activation functions.

1.4.1 ReLU (Rectified Linear Unit)

Formula: $f(x) = \max(0, x)$

ReLU is the classic default activation function across deep learning architectures.

- **Pros:** It does not saturate in the positive region. It is highly computationally efficient (just a thresholding operation) and converges significantly faster than Sigmoid or Tanh in practice (often roughly $6\times$ faster).
- **Cons:** Its outputs are not zero-centered. Most importantly, it suffers from the **"Dead ReLU"** problem: if an activation falls into the negative regime ($x < 0$), its local gradient is exactly 0. If a neuron is pushed into this regime by an unfortunate large weight update, it will never update its weights again.

1.4.2 Leaky ReLU

Formula: $f(x) = \max(\alpha x, x)$ (where typically $\alpha = 0.01$)

Introduced by Maas et al. [41], Leaky ReLU is designed specifically to prevent the "dead neuron" problem of standard ReLU.

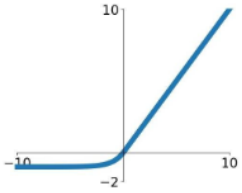
- **Pros:** Retains all the computational advantages of ReLU (no positive saturation, fast evaluation), while the small non-zero slope for negative inputs guarantees that neurons will never permanently die.

1.4.3 ELU (Exponential Linear Units)

Formula: $f(x) = x$ if $x > 0$, else $\alpha(e^x - 1)$

Proposed by Clevert et al. [14], ELU is defined to retain ReLU advantages while shifting mean activations toward zero:

- **Pros:** Inherits the benefits of ReLU, but produces outputs with a mean closer to zero. Its smooth negative saturation regime adds noise robustness compared to unbounded negative regimes.
- **Cons:** More computationally demanding because evaluating the exponential function e^x is slower than simple comparisons.



$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\,(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

Figure 1.4: Illustration of ELU.

1.4.4 GELU (Gaussian Error Linear Unit)

Formula: $f(x) = x \cdot \Phi(x)$, where $\Phi(x)$ is the cumulative distribution function (CDF) of the standard normal distribution. Introduced by Hendrycks and Gimpel [34]:

- **Pros:** Exhibits smooth probabilistic gating behavior around 0. Its continuous curvature facilitates optimization in state-of-the-art architectures, making it the standard choice in **GPT, BERT**, and modern Vision Transformers.
- **Cons:** Slightly higher computational cost than ReLU.

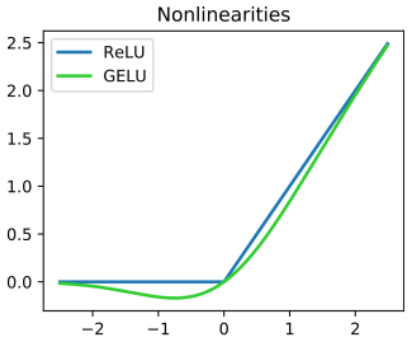


Figure 1.5: Illustration of GELU.

Note

TL;DR — Practical Guidelines for Activation Functions:

- Use **ReLU** as your standard baseline, monitoring learning rates to avoid dead neurons.
- Explore **Leaky ReLU** or **GELU** for improved gradient flow and smoother optimization.
- Avoid **Sigmoid/Tanh** for intermediate hidden layers of deep networks, reserving them strictly for output gates or bounded representations (such as in LSTM cells).

—

1.5 Weight Initialization and Signal Propagation

Properly initializing the weights of a neural network is crucial. With poor initialization, training can fail entirely before it starts due to signal collapse or numerical explosion.

1.5.1 The Pitfalls of Naive Initialization

- **Constant Initialization ($W =$ constant):** If all weights are initialized to the same value (e.g., $W = 0$), all neurons in a given layer will produce identical outputs and receive identical gradients during backpropagation. The network fails to *break symmetry* and cannot learn independent features.

Small Random Noise and Vanishing Gradients

A common naive approach is initializing weights with small random numbers, such as `W = 0.01 * np.random.randn(D, H)`, where weights are sampled from a standard normal distribution scaled by a tiny standard deviation of 0.01.

In deep networks, this naive scaling causes complete signal extinction:

- **Step 1: The Forward Pass (Shrinking Activations).**
  The histograms in Figure 1.6 illustrate the distribution of **activations** across successive layers. Because the weights are scaled by 0.01, the output of each layer is strictly smaller in variance than its input. Across sequential layers, this repeated multiplication causes the signal energy to exponentially decay. By Layer 6, activations have completely collapsed to 0 (std $= 0.05$).
- **Step 2: The Backward Pass (Vanishing Gradients).**
  During backpropagation through a linear layer $y = Wx$, the local gradient with respect to weights is $\frac{\partial y}{\partial W} = x$. Therefore, the gradient of the Loss with respect to W is:

$$\frac{\partial L}{\partial W} = x^T \frac{\partial L}{\partial y}$$

The weight update is directly proportional to the magnitude of the input activations x. Because the forward pass collapsed the activations to 0, the weight gradient evaluates to $\frac{\partial L}{\partial W} \approx 0 \cdot \frac{\partial L}{\partial y} = 0$, bringing learning to a complete standstill.

1.5.2 Variance Scaling Methods: The Solution

To guarantee that signal variance remains stable across deep layers, we scale initial weights **dynamically** based on the fan-in (D_{in}) of each neuron:

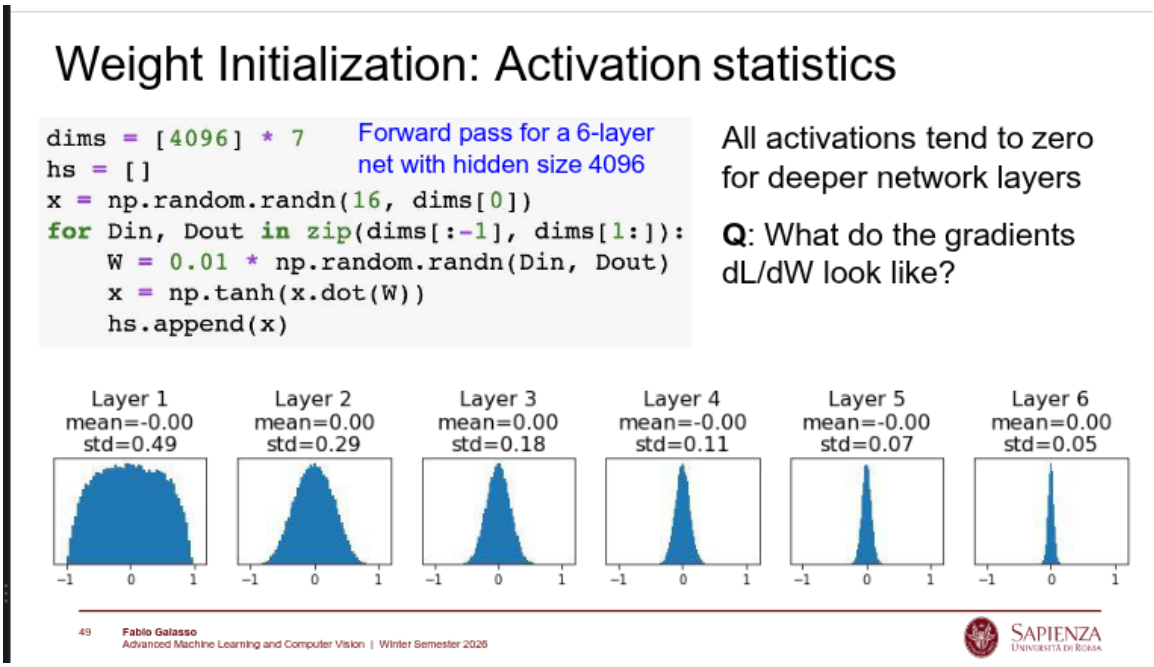


Figure 1.6: Distribution of activations x across 6 layers with $W \sim \mathcal{N}(0, 0.01)$ and Tanh.

Key Word

1. Xavier (Glorot) Initialization

Designed for symmetric, zero-centered activation functions like **Tanh** or **Sigmoid**. The variance of activations is kept constant across layers by drawing weights from a distribution with standard deviation:

$$\text{std} = \frac{1}{\sqrt{D_{in}}}$$

2. Kaiming (He) Initialization (He et al., 2015) [31]

Xavier initialization fails with **ReLU** because ReLU zeroes out all negative values, effectively halving the variance at each step. Kaiming initialization compensates by doubling the variance, setting the standard deviation to:

$$\text{std} = \sqrt{\frac{2}{D_{in}}}$$

This is the standard and mandatory initialization when training deep ReLU networks.

—

1.6 Gradient-Based Optimization Algorithms

From a theoretical standpoint, our objective is to minimize the total loss over the entire dataset:

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W) + \lambda R(W)$$

The exact gradient requires summing over all N training examples:

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

Computing this **Full Batch** gradient requires evaluating millions of samples at every parameter update, which is computationally prohibitive.

Key Word

Stochastic Gradient Descent (SGD): Instead of the full sum, SGD approximates the true gradient over a small random **minibatch** of examples (e.g., 32, 64, or 128). While this introduces stochastic variance, it allows frequent, computationally cheap updates that accelerate convergence.

—

1.7 Regularization in Deep Neural Networks

The objective of **Regularization** is to prevent models from overfitting training data by controlling model capacity. We add a parameter complexity penalty $R(W)$ to the empirical loss:

$$L_{total}(W,X) = \underbrace{L(W,X)}_{\text{Data Loss}} + \underbrace{\lambda R(W)}_{\text{Regularization Term}}$$

Note

Role of the Hyperparameter λ:
- $\lambda = 0$: The model focuses solely on fitting training data, risking **Overfitting**.
- $\lambda \to \infty$: The penalty dominates, forcing all weights toward zero and causing **Underfitting**.

Classic mathematical penalties include:

- **L2 Regularization (Weight Decay / Ridge):** $R(W) = \sum W^2$. Penalizes large weights quadratically, encouraging diffuse representations across all neurons.
- **L1 Regularization (Lasso):** $R(W) = \sum |W|$. Drives less relevant weights exactly to zero, performing automatic feature selection.

1.7.1 Dropout

Introduced by Srivastava et al. [50], **Dropout** is a stochastic regularization technique where individual neurons are randomly deactivated during training with probability p (commonly $p = 0.5$).

- **Prevents Co-adaptation:** Neurons cannot rely on the specific activations of neighboring units, forcing them to learn robust, self-reliant features.
- **Ensemble Effect:** Dropout implicitly trains an exponential ensemble of shared-weight subnetworks (2^N sub-architectures for an N-neuron layer).

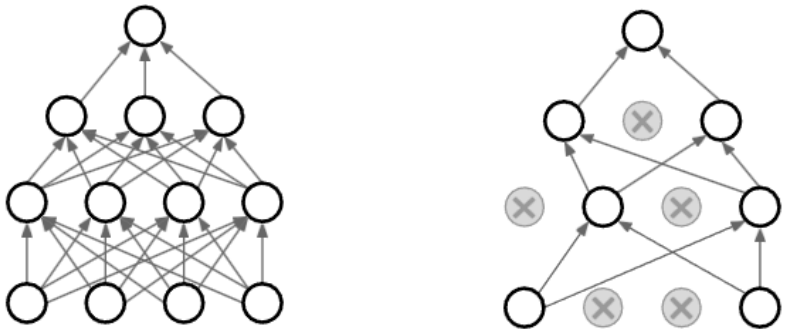


Figure 1.7: Illustration of Dropout during training.

Inverted Dropout

Because dropping neurons during training reduces the active sum of signals, the expected output at test time (when all neurons are active) would be scaled by $\frac{1}{1-p}$. To eliminate any scaling overhead at inference time, modern implementations use **Inverted Dropout** during training:

$$h_{\text{train}} = \frac{\text{mask} \odot x}{1-p}$$

- **Numerator (mask $\odot x$):** Drops a random fraction p of the activations to zero.
- **Denominator ($1-p$):** Inflates the surviving activations, ensuring that the expected activation value during training matches the deterministic magnitude at test time ($h_{\text{test}} = x$).

Example

Numerical Example of Inverted Dropout ($p = 0.5$):
Given input $x = [10, 20, 30, 40]$ (Sum $= 100$) and mask $[1, 0, 1, 0]$:

$$\text{Masked output: } [10, 0, 30, 0] \quad (\text{Sum drops to } 40)$$

$$h_{\text{train}} = \frac{[10, 0, 30, 0]}{0.5} = [20, 0, 60, 0] \quad (\text{Scaled sum } = 80 \approx 100)$$

1.7.2 The Common Pattern of Modern Regularization

Dropout illustrates the universal recipe of modern deep learning regularization:

1. **Training:** Inject stochasticity or random noise.
2. **Testing:** Marginalize or average out the randomness to yield deterministic predictions.

—

1.8 Feature Normalization Techniques

To stabilize signal distributions and accelerate optimization in deep networks, activations are explicitly normalized across intermediate layers.

1.8.1 Batch Normalization

Introduced by Ioffe and Szegedy [37], Batch Normalization (BN) normalizes activations across the minibatch dimension, forcing each feature channel k to zero mean and unit variance:

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}] + \varepsilon}}$$

where $\mathrm{E}[x^{(k)}]$ and $\mathrm{Var}[x^{(k)}]$ are the minibatch mean and variance, and ε prevents division by zero.

Learnable Scale and Shift Parameters

To prevent rigid normalization from limiting network capacity, two learnable parameters (γ and β) restore representational flexibility:

$$y^{(k)} = \gamma^{(k)}\widehat{x}^{(k)} + \beta^{(k)}$$

If beneficial, the optimizer can recover the identity mapping by setting $\gamma^{(k)} = \sqrt{\mathrm{Var}[x^{(k)}]}$ and $\beta^{(k)} = \mathrm{E}[x^{(k)}]$.

Training versus Test Phase

During training, statistics are calculated over the current minibatch. During inference, fixed global running averages computed during training are used, allowing BN to be mathematically folded into preceding linear layers with zero inference overhead.

1.8.2 Alternative Normalization Methods

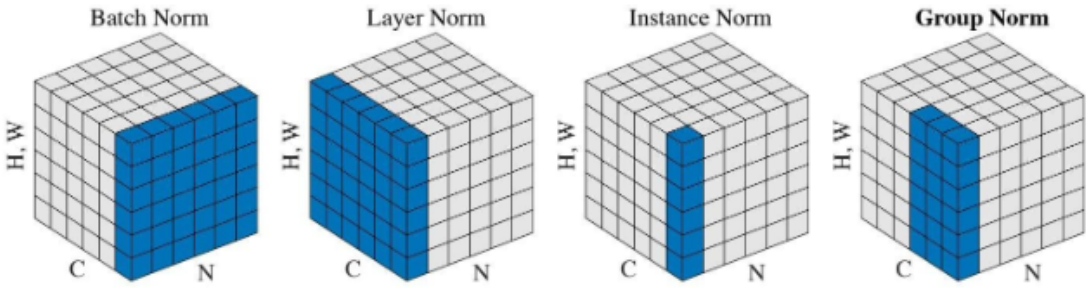


Figure 1.8: Comparison of different normalization methods across batch (N), channel (C), and spatial (H, W) dimensions.

The choice among normalization methods depends on the axes across which statistics are aggregated (Figure 1.8):

Batch Norm (BN)

Aggregates across the batch dimension N and spatial dimensions (H, W) independently for each channel C [37]. Ideal for vision models with large minibatches.

Layer Norm (LN)

Proposed by Ba et al. [1], LN aggregates across all channels C and spatial dimensions for each individual sample N. Independent of batch size, making it the standard for RNNs and Transformers.

Instance Norm (IN)

Proposed by Ulyanov et al. [53], IN aggregates strictly across spatial dimensions (H, W) for each single channel of each single sample. Widely used in generative style transfer.

Group Norm (GN)

Introduced by Wu and He [58], GN divides the channel dimension C into distinct groups and aggregates across each group per sample. Provides batch-size independence while preserving channel specialization.

—

1.9 Chapter Summary and Conceptual Graph

The foundations developed in this chapter form an interconnected chain where each new concept resolves a specific mathematical failure mode or constraint introduced by earlier building blocks:

Key Insight

The Logical Connections Between Nodes:

- **From Node 1 to 2:** A single linear layer cannot approximate complex non-linear functions; we must stack layers into deep networks, which in turn requires Backpropagation to compute parameter gradients.
- **From Node 2 to 3:** Stacking linear transformations collapses into a single linear map ($W_2W_1x = W'x$); activation functions are strictly required to introduce non-linear expressivity.
- **From Node 3 to 4:** Deep non-linear networks are prone to signal decay at initialization. Naive random noise collapses activations to zero, causing vanishing gradients. Variance scaling (He/Xavier) dynamically stabilizes signal spread.
- **From Node 4 to 5:** With stable gradients established, evaluating the loss over millions of examples is computationally prohibitive; Stochastic Gradient Descent (SGD) enables rapid iterative updates via minibatches.
- **From Node 5 to 6:** High-capacity deep models easily overfit training data. Regularization (weight decay and Inverted Dropout) controls capacity by penalizing weights and enforcing distributed representations.
- **From Node 6 to 7:** Throughout deep optimization, intermediate layer distributions continuously drift; Batch, Layer, and Group Normalization re-center activations to maintain high learning rates and stable convergence.

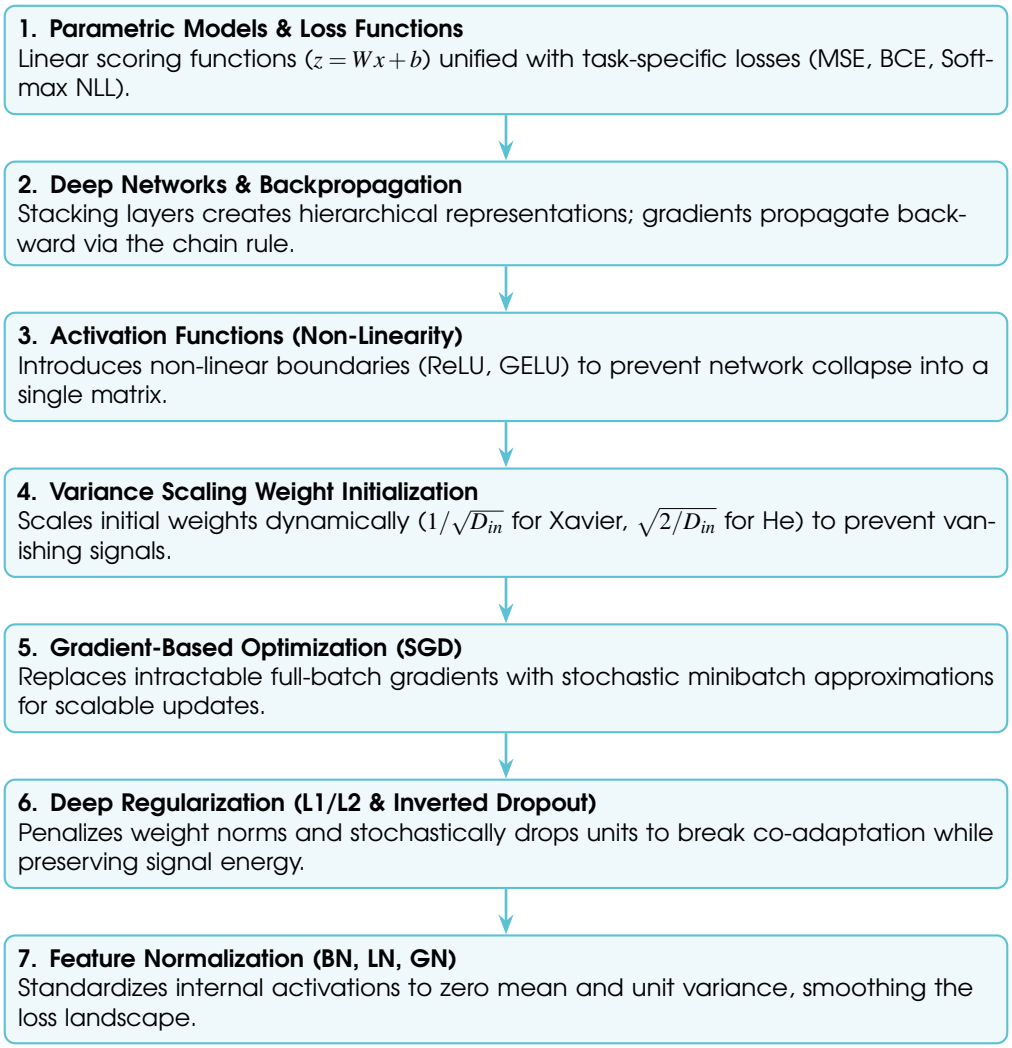
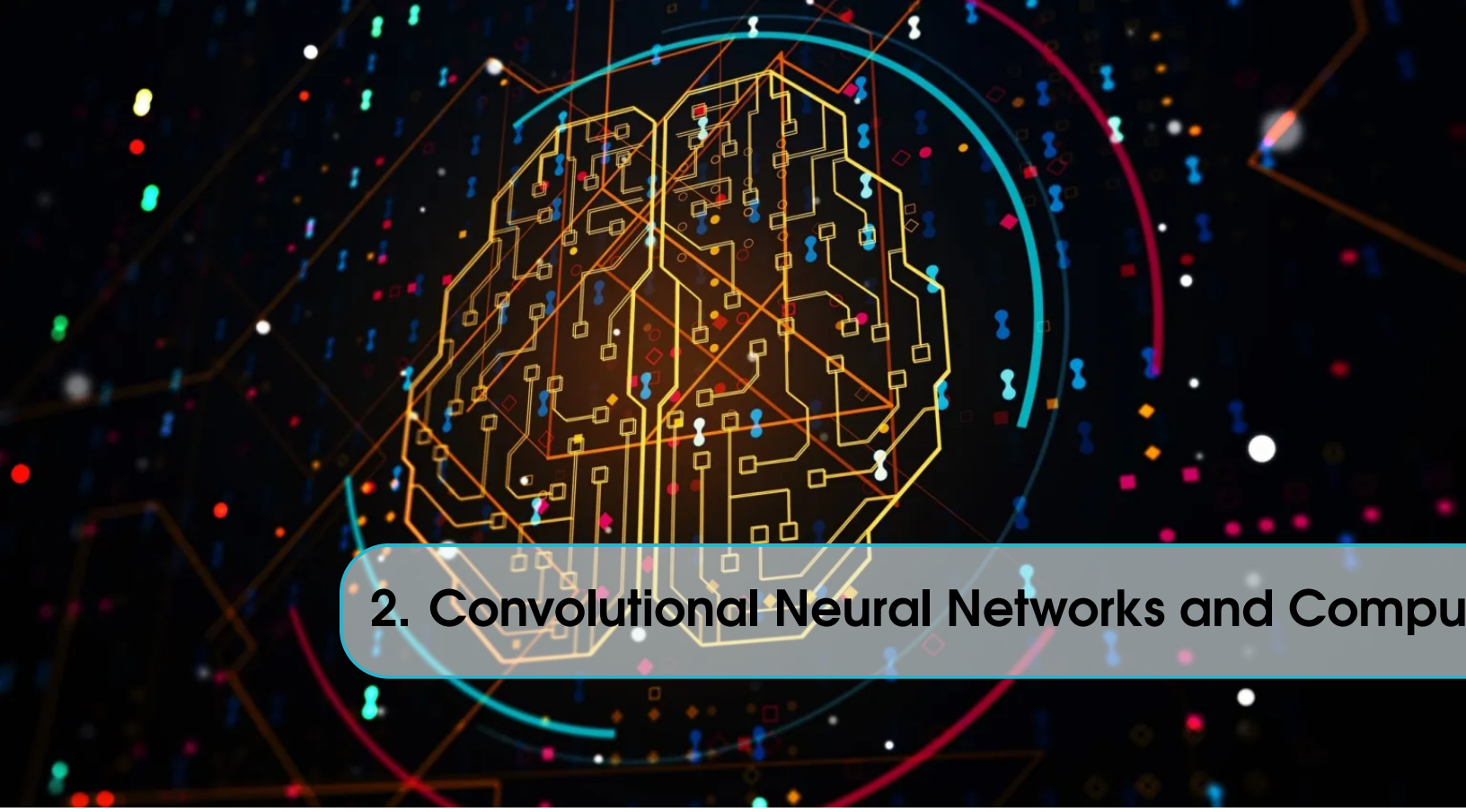


Figure 1.9: Conceptual roadmap of deep learning foundations and optimization.



2. Convolutional Neural Networks and Compu

2.1 Image Representation and the Imaging Process

In computer vision, the fundamental objective is to interpret and reason about visual inputs. Digital images are typically formed via an optical camera model (such as the pinhole camera model) and subsequently digitized, as shown in Figure 2.1. This chapter explores convolutional architectures, classical vision backbones, and data augmentation based on the course lecture slides [22].

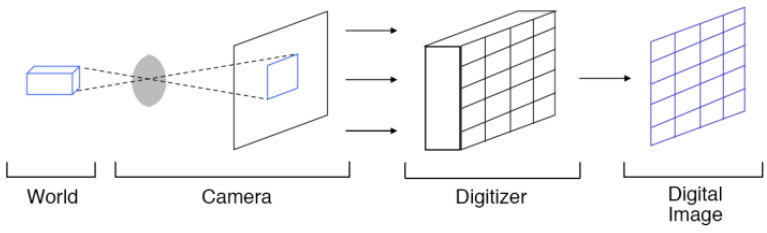


Figure 2.1: Illustration of the pinhole camera imaging process.

Note

The Digital Image Tensor: A digital camera digitizes incoming light into a 3D array of numbers with dimensions $(H \times W \times C)$, where H is the image height, W is the image width, and C represents the color channels (typically $C = 3$ for Red, Green, and Blue). Each pixel intensity is represented by an integer in $[0, 255]$ or normalized as a floating-point value in $[0, 1]$.

—

2.2 Motivations for Convolutions

2.2.1 The Problem with Fully Connected Layers

In standard Fully Connected (Dense) layers, every input unit is wired to every output unit. When applied directly to image data, this architecture exhibits severe failure modes:

- **Parameter Explosion & Overfitting:** Consider a modest 1000×1000 RGB image (3×10^6 input values). Connecting this input to a hidden layer of only 10^3 neurons requires over 3×10^9 parameters (3 billion weights). Storing and optimizing this volume of parameters requires unfeasible memory and inevitably triggers catastrophic overfitting.
- **Destruction of Spatial Topology:** Flattening an image into a 1D vector completely ignores spatial relationships, treating adjacent pixels with the same prior relevance as pixels located at opposite ends of the image.

2.2.2 Exploiting Image Structure: Locality and Stationarity

To overcome these structural limitations, **Convolutional Neural Networks (CNNs)** leverage two intrinsic physical properties of visual data:

1. **Local Spatial Correlation:** Spatially proximal pixels exhibit strong statistical correlation, whereas distant pixels are relatively uncorrelated. Instead of connecting each neuron globally to the entire image, a neuron connects only to a small local receptive field.
2. **Translation Stationarity (Invariance):** Visual features (such as edges, corners, and textures) are statistically invariant to spatial translation. A pattern detector useful in the top-left corner is equally meaningful in the bottom-right corner. Consequently, the same set of parameters can be shared across all spatial positions.

—

2.3 The Core Building Block: The Convolutional Layer

A Convolutional Layer replaces dense matrix multiplication with a local sliding operation, illustrated in Figure 2.2:

- **The Kernel (Filter):** A small matrix of learnable weights (e.g., 3×3 or 5×5). It serves as a specialized feature detector.
- **The Sliding Window:** The filter slides systematically across the spatial grid of the input. At each position, it computes the dot product (sum of element-wise multiplications) between its weights and the underlying local patch.

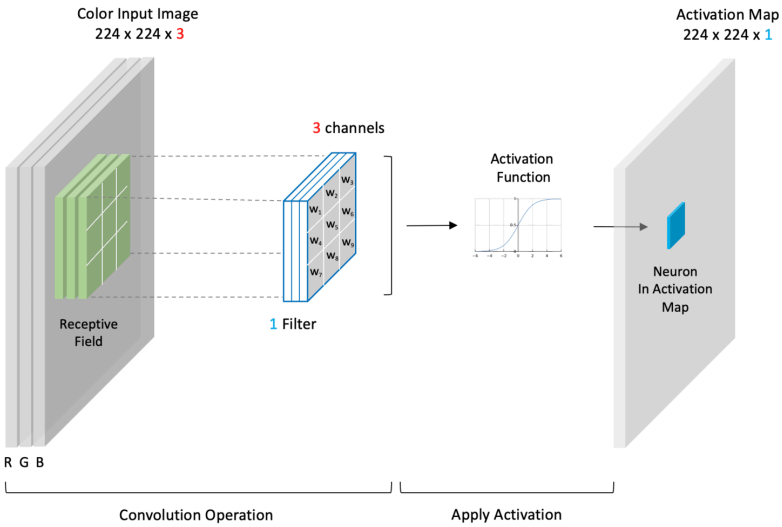


Figure 2.2: Illustration of a single convolutional layer operating on an image.

- **The Feature Map:** The resulting output grid, illustrated in Figure 2.3. High values indicate strong alignment between the learned filter pattern and that specific image region.

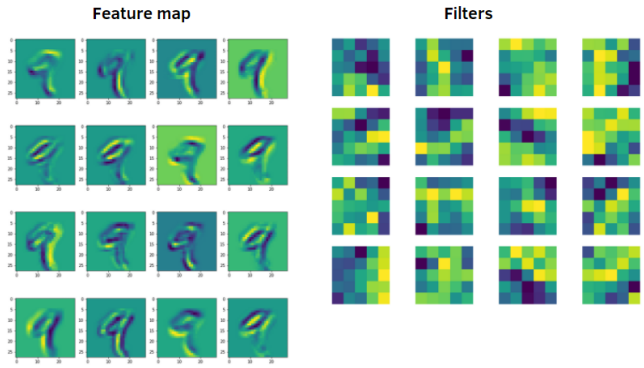


Figure 2.3: Multiple feature maps produced by convolving distinct filters over an input.

2.3.1 Key Hyperparameters of the Convolution

The behavior and spatial dimensions of a convolutional layer are governed by three primary hyperparameters:

- **Stride (S):** The step size with which the kernel moves across the image. A stride of $S = 1$ moves pixel-by-pixel, while $S > 1$ downsamples the output grid.
- **Padding (P):** Adding zero-valued borders around the input image boundary to preserve spatial dimensions and ensure edge pixels receive equal computational attention.
- **Channels (C):** Real images have depth (e.g., $C_{in} = 3$ for RGB). A kernel must match the depth of the input, yielding a 3D tensor of shape $(C_{in} \times K_h \times K_w)$. Using C_{out} distinct kernels produces C_{out} output feature

maps.

2.3.2 Formal Mathematical Definition

Formally, the discrete 2D convolution of an image I with a 2D filter g at spatial location $[m,n]$ is defined as:

$$f[m,n] = (I * g)[m,n] = \sum_k \sum_l I[m-k, n-l]\, g[k,l] \tag{2.1}$$

where k and l index the spatial extent of the kernel.

Key Word

Convolution versus Cross-Correlation in Deep Learning

In classical signal processing, the index subtraction $I[m-k,n-l]$ implies flipping the kernel g by 180°. Deep learning libraries (e.g., PyTorch) compute **Cross-Correlation** instead:

$$f[m,n] = (I \star g)[m,n] = \sum_k \sum_l I[m+k, n+l]\, g[k,l]$$

Because kernel weights $g[k,l]$ are learned from scratch via backpropagation, the network simply learns pre-flipped weights directly, rendering cross-correlation and convolution mathematically equivalent for optimization.

—

2.4 Classic Image Processing Filters

Before inspecting multi-channel deep representations, consider how classical handcrafted kernels manipulate local spatial neighborhoods to extract specific visual features.

Example

1. Smoothing Filter (Noise Reduction):

Replacing each pixel with a weighted neighborhood average dampens noise and sharp outliers:

2	3	3
3	**20**	2
3	2	3

Smoothing →

2	3	3
3	**3**	2
3	2	3

The central anomaly (20) is effectively smoothed down to 3.

Example

2. Vertical Edge Detection Filter:
Filters with opposing positive and negative signs detect sharp directional variations in pixel intensity:

$$g_{\text{vertical}} = \begin{array}{|c|c|c|}\hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline \end{array}$$

Transposing g_{vertical} yields a horizontal edge detector $g_{\text{horizontal}} = g_{\text{vertical}}^T$.

Example

3. Step-by-Step 2D Convolution Example (Stride 1, Pad 1):
Convolving a 3×3 image I with a vertical edge filter g with padding $P = 1$ pads the input with zeros into a 5×5 grid I_{padded}:

$$I_{\text{padded}} = \begin{array}{|c|c|c|c|c|}\hline 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 8 & 5 & 2 & 0 \\ \hline 0 & 7 & 5 & 3 & 0 \\ \hline 0 & 9 & 4 & 1 & 0 \\ \hline 0 & 0 & 0 & 0 & 0 \\ \hline \end{array}$$

Applying the convolution with stride $S = 1$ yields:

$$\underset{g[k,l]}{\begin{array}{|c|c|c|}\hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline \end{array}} \otimes \underset{\text{(conceptually)}}{I_{\text{padded}}} = \underset{f[m,n]}{\begin{array}{|c|c|c|}\hline -10 & 10 & 10 \\ \hline -14 & 18 & 14 \\ \hline -9 & 12 & 9 \\ \hline \end{array}}$$

Accounting for the 180° Kernel Flip:
Flipping g by 180° produces:

$$g_{\text{flipped}} = \begin{array}{|c|c|c|}\hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline \end{array}$$

- **Center Value $f[1,1] = 18$:** Flipped kernel overlaps fully with the original 3×3 image pixels:

$$(1 \cdot 8 - 1 \cdot 2) + (1 \cdot 7 - 1 \cdot 3) + (1 \cdot 9 - 1 \cdot 1) = 6 + 4 + 8 = 18$$

- **Top-Left Value $f[0,0] = -10$:** Flipped kernel overlaps with the top-left boundary of I_{padded}:

$$(1 \cdot 0 - 1 \cdot 5) + (1 \cdot 0 - 1 \cdot 5) = -10$$

—

2.5 Output Shape and Parameter Counting for 3D Volumes

In deep vision models, inputs are 3D tensors $(C_{\text{in}} \times H \times W)$, and each layer applies a bank of C_{out} distinct filters.

2.5.1 Spatial Output Size Formula

Given an input spatial width W, kernel spatial size K, padding P, and stride S, the output spatial width W_{out} is given by:

$$W_{\text{out}} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1 \tag{2.2}$$

Example

Numerical Example of Parameter Counting:
Suppose the input volume is a $32 \times 32 \times 3$ image, and we apply 10 filters of size 5×5 (with stride $S = 1$, padding $P = 2$):

- **Output spatial size:** $\lfloor (32 - 5 + 2 \cdot 2)/1 \rfloor + 1 = 32 \implies (32 \times 32)$.
- **Output volume size:** $32 \times 32 \times 10$.
- **Number of parameters:** Each filter has $5 \times 5 \times 3 = 75$ weights plus 1 bias $= 76$ parameters. For 10 filters: $76 \times 10 =$ **760** parameters.

—

2.6 The Pooling Layer

Convolutional neural networks interleave convolutional layers with **Pooling layers** to perform spatial down-sampling.

2.6.1 Purpose and Properties of Pooling

- **Spatial Dimensionality Reduction:** Reduces spatial width and height, reducing computational complexity in deeper layers.
- **Parameter Control:** Shrinking feature map resolutions minimizes the number of parameters needed when eventually transitioning to fully connected classification heads.
- **Translation Invariance:** Pooling provides slight invariance to local translations; small shifts in the input do not alter the pooled maximum or mean.
- **Channel Independence:** Pooling operates independently on each depth slice without mixing channels. If the input has C channels, the output has exactly C channels.

2.6.2 Max Pooling

Max Pooling slides a window across each channel and outputs strictly the **maximum value** in each receptive window.

Example

Max Pooling Example (2×2 filter, Stride 2):
Applying Max Pooling to a 4×4 slice:

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

$\xrightarrow{\text{Max Pool } 2\times2, S=2}$

6	8
3	4

Spatial dimensions are halved, discarding 75% of activations and retaining only the most prominent local features.

2.6.3 Average Pooling

Average Pooling computes the **arithmetic mean** over each local window:

Example

Average Pooling Example (2×2 filter, Stride 2):
Applying Average Pooling to a 4×4 slice:

1	1	2	4
4	6	6	8
3	2	1	0
1	2	3	4

$\xrightarrow{\text{Average Pool } 2\times2, S=2}$

3	5
2	2

—

2.7 Visualizing Convolutional Networks

Key Insight

Visualizing what deep CNNs learn (e.g., via Deconvolutional Networks by Zeiler and Fergus) reveals a clear hierarchical decomposition:

- **Layer 1:** Learns elementary low-level visual primitives (oriented edges, basic color gradients).
- **Layer 2:** Combines primitives into textures, corners, and simple geometric patterns.
- **Layer 3 and Deeper:** Assembles textures into semantically complex object parts (e.g., wheels, faces) and eventually full objects.

—

2.8 Vision Regularization: Data Augmentation

While weight decay acts on model parameters and Dropout acts on activations, **Data Augmentation** acts directly on the input data. It artificially expands training dataset diversity by applying label-preserving

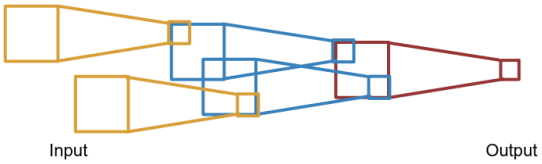


Figure 2.4: Hierarchical feature representations learned across deep convolutional layers.

transformations, embodying the standard modern regularization paradigm: introducing stochasticity at training and evaluating on fixed standard inputs at test time.

2.8.1 Classic Spatial and Color Augmentations

- **Horizontal Flips:** A simple, highly effective transformation for general object recognition.
- **Random Crops and Scales:** During training, random patches are extracted at varying scales (e.g., ResNet resizes the short side between 256 and 480 pixels before extracting a 224×224 crop). At test time, predictions are averaged over a fixed set of crops (e.g., 10 crops: 4 corners + center, and their horizontal flips).
- **Color Jitter:** Perturbing brightness, contrast, and saturation to render the model invariant to illumination changes.

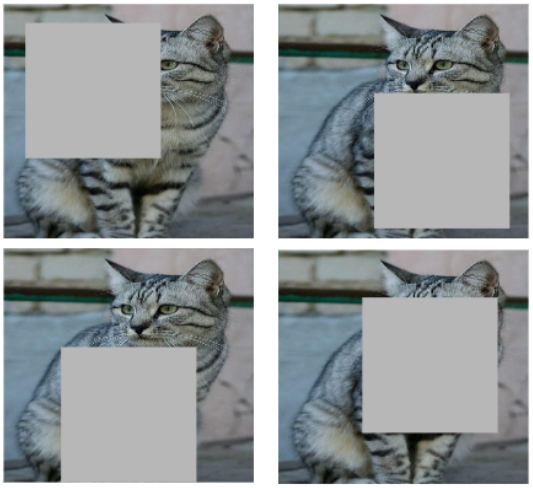


Figure 2.5: Example of Cutout.

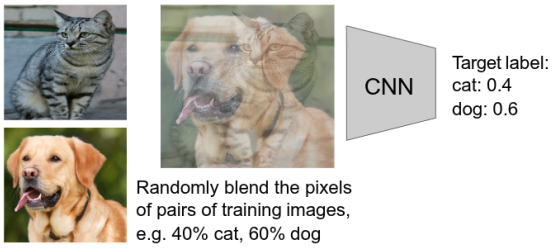


Figure 2.6: Example of Mixup.

2.8.2 Advanced Stochastic Augmentations

- **Cutout:** Introduced by DeVries and Taylor [16], square image patches are randomly masked out (set to zero) during training. This forces the network to avoid over-relying on a single dominant feature (such as an animal's face) and learn distributed contextual cues. Unmodified images are evaluated at test time.
- **Mixup:** Proposed by Zhang et al. [60], blends pairs of images and their corresponding one-hot labels linearly. For instance, blending 40% of a cat image with 60% of a dog image produces a target training label of $[0.4\,\text{cat}, 0.6\,\text{dog}]$. At test time, standard unblended images are used.

—

2.9 Transfer Learning in Computer Vision

2.9.1 The Fundamental Concept (Why Use It?)

Training deep Convolutional Neural Networks from scratch requires massive datasets (e.g., millions of annotated images). On small target datasets, training from scratch leads to severe overfitting.

The solution is **Transfer Learning**: leveraging a **Source model** pre-trained on a gigantic dataset (e.g., ImageNet) and transferring its learned visual representations to a **Target model** for a specific, smaller task.

2.9.2 Practical Implementation

Using standard backbones like VGG [49] or ResNet [30], transfer learning proceeds in two stages:

- **1. Preserving Convolutional Feature Extractors:** Early and middle convolutional layers are kept frozen. They act as universal feature extractors, having already learned edge, texture, and shape primitives common to all natural visual scenes.
- **2. Replacing the Classification Head:** The final Fully Connected layers are removed and replaced with new layers configured for the target dataset's classes. Only this new head (and optionally fine-tuned upper conv layers) is trained.

2.9.3 Pervasiveness: The Modern Standard

Transfer learning is standard across all computer vision applications:

- **Object Detection:** Frameworks like Fast R-CNN utilize ImageNet-pre-trained CNN backbones to extract spatial features before predicting bounding boxes.
- **Image Captioning:** Pre-trained CNN backbones produce dense feature vectors that initialize or condition recurrent sequence decoders.

Key Word

The Model Zoo

All major deep learning frameworks provide a **Model Zoo** (e.g., `torchvision.models` in PyTorch) containing pre-trained weights for state-of-the-art vision architectures. These models can be downloaded instantly and fine-tuned for custom downstream tasks.

2.10 Chapter Summary and Conceptual Graph

The concepts developed in this chapter establish how the inductive biases of natural visual data lead from raw pixels to deep hierarchical vision systems:

Key Insight

The Logical Connections Between Nodes:

- **From Node 1 to 2:** Viewing digital images as raw flattened arrays reveals why generic architectures fail: high spatial resolutions lead directly to parameter explosion and loss of geometry.
- **From Node 2 to 3:** Recognizing the failure of dense connections motivates identifying domain-specific inductive biases: pixels exhibit local correlation and patterns are invariant to spatial shifts.
- **From Node 3 to 4:** Locality and stationarity directly dictate the mathematical structure of the Convolutional Layer: local receptive fields with shared kernel weights.
- **From Node 4 to 5:** While convolutions preserve spatial structure, deep networks need downsampling to grow the effective receptive field and limit computation, achieved through parameter-free Pooling layers.
- **From Node 5 to 6:** Interleaving convolutions and pooling enables deep networks to naturally learn hierarchical representations, moving from oriented edges in early layers to semantic concepts in deep layers.
- **From Node 6 to 7:** Deep vision models with high expressive capacity easily memorize training images; Data Augmentation (including Cutout and Mixup) enforces invariance to photometric and spatial perturbations.
- **From Node 7 to 8:** Because training deep CNNs from scratch requires millions of images, Transfer Learning allows small custom datasets to inherit universal visual features from large-scale pre-trained Model Zoos.

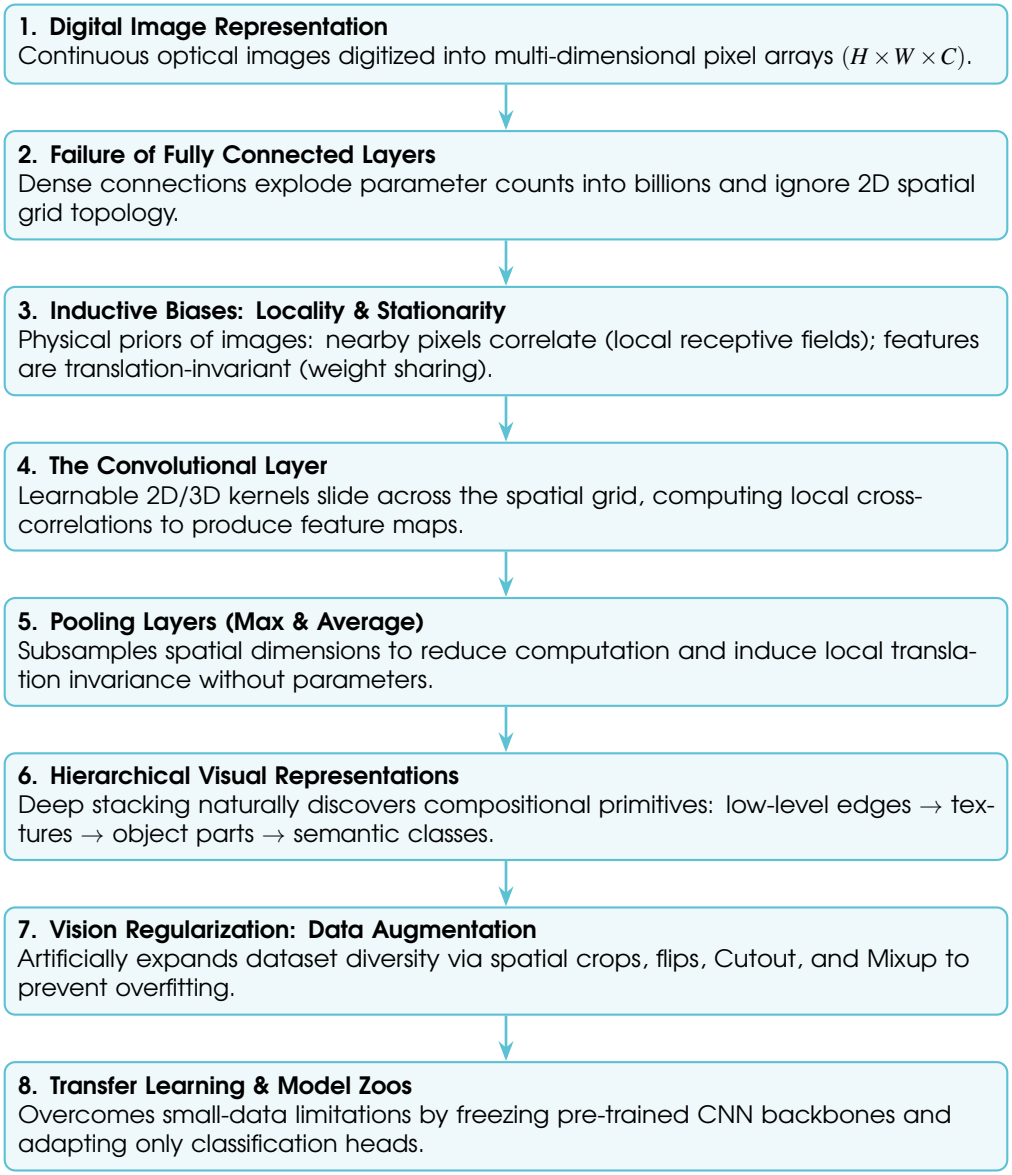


Figure 2.7: Conceptual roadmap of convolutional neural networks and computer vision.

3. Sequence Models

3.1 Introduction to Sequence Modeling

Standard feedforward architectures (such as multi-layer perceptrons and traditional convolutional neural networks) operate strictly under a **one-to-one** paradigm: a fixed-dimensional input vector $x \in \mathbb{R}^D$ is mapped to a fixed-dimensional output $y \in \mathbb{R}^K$. While effective for static tasks like single-image classification, this formulation fails when dealing with data that unfolds over time or possesses an intrinsic sequential nature, such as text, audio, video streams, or financial time series. This chapter covers sequence modeling paradigms, recurrent neural networks, and gating mechanisms based on the course lecture slides [25].

Key Word

The Sequence Modeling Challenge

In sequential tasks, either the input, the output, or both can have arbitrary and variable lengths:

$$\{x_1, x_2, \ldots, x_T\} \quad \longrightarrow \quad \{y_1, y_2, \ldots, y_{T'}\}$$

A sequence model must be able to preserve temporal order, handle variable-length sequences without retraining, and share learned parameters across different time steps.

3.1.1 Taxonomy of Sequence Tasks

Depending on whether the input and output consist of single vectors or sequences of vectors, sequential problems are traditionally classified into five primary architectural paradigms, illustrated in Figure 3.1.

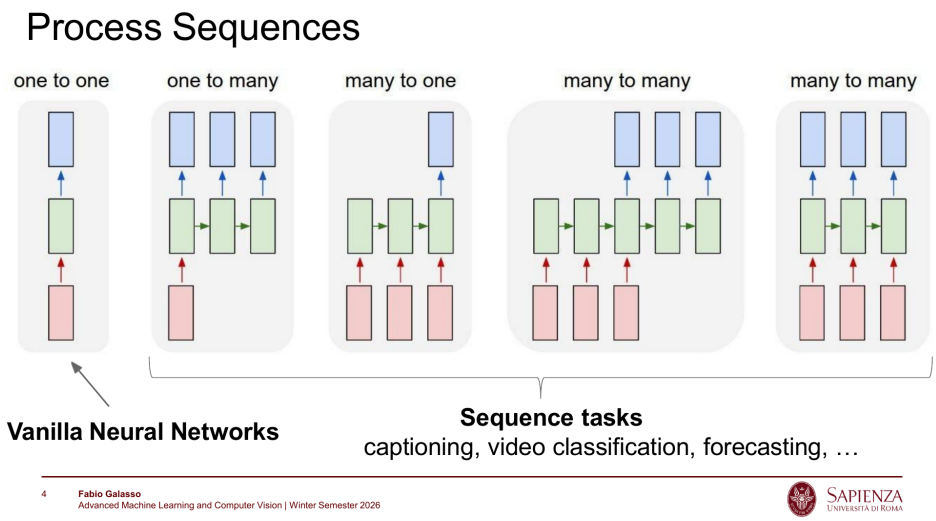


Figure 3.1: Architectural taxonomy of sequence processing tasks (from course slides). Pink boxes denote inputs, green boxes represent internal processing units, and blue boxes denote outputs.

- **One-to-One:** The conventional neural network setup without temporal recurrence. A single input produces a single output (e.g., standard image classification).
- **One-to-Many:** A single fixed-size input generates a sequential output of variable length. A prominent example is **Image Captioning**, where a single visual input (e.g., a photo of a dog) produces an output sequence of textual tokens (*"A happy brown dog"*).
- **Many-to-One:** A temporal sequence of inputs is processed and summarized into a single static prediction. Typical examples include **Action Prediction / Video Classification** (where a temporal sequence of consecutive video frames is mapped to a single discrete action label like *"Jumping"*) and **Sentiment Analysis** (a sequence of words mapped to a positive or negative score).
- **Many-to-Many (Synchronized / Aligned):** Both input and output are sequences of equal length, evaluated in lockstep at every time step. An example is **Frame-level Video Classification**, where an action category or segmentation map is predicted synchronously for each incoming frame.
- **Many-to-Many (Asynchronous / Sequence-to-Sequence):** The input sequence and output sequence have independent lengths ($T \neq T'$). This paradigm powers **Machine Translation** (translating an English sentence into Italian) and **Video Captioning** (ingesting a sequence of video frames to output a descriptive sentence, e.g., *"A dog jumps over a hurdle"*).

3.1.2 Forecasting as a Core Sequence Problem

A critical subset of sequence tasks is **Forecasting**: given a historical trajectory or sequence of observations up to the current time t, predict future observations:

$$(x_1, x_2, \ldots, x_t) \quad \longrightarrow \quad (\hat{x}_{t+1}, \hat{x}_{t+2}, \ldots)$$

Example

Real-World Applications of Forecasting:

- **Trajectory Forecasting:** Predicting the future spatial paths of pedestrians and vehicles, essential for collision avoidance in autonomous driving and human-robot interaction.
- **Long-term Tracking across Occlusions:** Estimating the motion of targets when temporarily blocked by obstacles.
- **Physical and Financial Time Series:** Forecasting weather patterns, seismic alerts, critical pressure thresholds in industrial power plants, and financial market trends.

—

3.2 Temporal Convolutional Networks

Before exploring recurrent architectures, sequence modeling can be approached using **Convolutional Neural Networks** applied along the temporal dimension, commonly known as **Temporal Convolutional Networks (TCN)**.

3.2.1 One-Dimensional Convolutions on Sequences

While standard computer vision uses 2D convolutions over spatial grids, temporal data is structured as a 1D sequence over discrete time. The 1D discrete convolution of an input sequence f with a learnable filter g is formulated as:

$$(f * g)(i) = \sum_k g(k)\, f(i-k) \tag{3.1}$$

Note

Piece-by-Piece Mathematical Breakdown of Equation 3.1:

- i: The discrete temporal index (time step) at which the output feature is evaluated.
- f: The 1D input sequence signal across time (e.g., raw temporal measurements or feature embeddings).
- g: The learnable convolutional kernel (filter) of fixed size K.
- k: The kernel offset index running over the temporal window of the filter ($k \in \{0, \ldots, K-1\}$).
- $g(k)$: The learnable parameter (weight) at relative offset k within the kernel.
- $f(i-k)$: The input value sampled k time steps prior to time step i.
- $\sum_k$: The summation accumulating the weighted products across the entire receptive span of the kernel.

3.2.2 Causal and Dilated Convolutions

When applying convolutions to sequential forecasting, two structural constraints arise:

Causal Convolutions

Standard convolutions compute outputs using a symmetric window that looks both backward and forward in time. For temporal prediction, accessing future time steps ($i+k$ with $k>0$) causes an illegal information leak (*lookahead*). Therefore, the convolution must be strictly **causal**: the calculation at time step t depends solely on inputs x_s where $s \leq t$.

Dilated Convolutions and Receptive Field

A fundamental limitation of standard causal convolutions is that the **receptive field** (the maximum historical context the network can observe) grows only *linearly* with the depth of the network:

$$\text{Receptive Field} = 1 + L(K-1)$$

where L is the number of layers and K is the kernel size. Capturing long-term temporal dependencies would therefore require an impractical number of layers.

To overcome this limitation, TCNs introduce **Dilated Causal Convolutions**, as illustrated in Figure 3.2. For a 1D sequence f and kernel g, a dilated convolution with dilation factor d is defined as:

$$(f *_d g)(i) = \sum_k g(k)\, f(i - d \cdot k) \tag{3.2}$$

Note

Piece-by-Piece Mathematical Breakdown of Equation 3.2:

- d: The **dilation factor**, which determines the spacing between kernel taps. For $d=1$, Equation 3.2 reduces to a standard convolution. For $d>1$, the filter samples inputs every d steps, skipping $d-1$ intermediate values.
- $i - d \cdot k$: The temporal sampling index, stretching the effective reach of the kernel backwards in time without adding extra parameters.

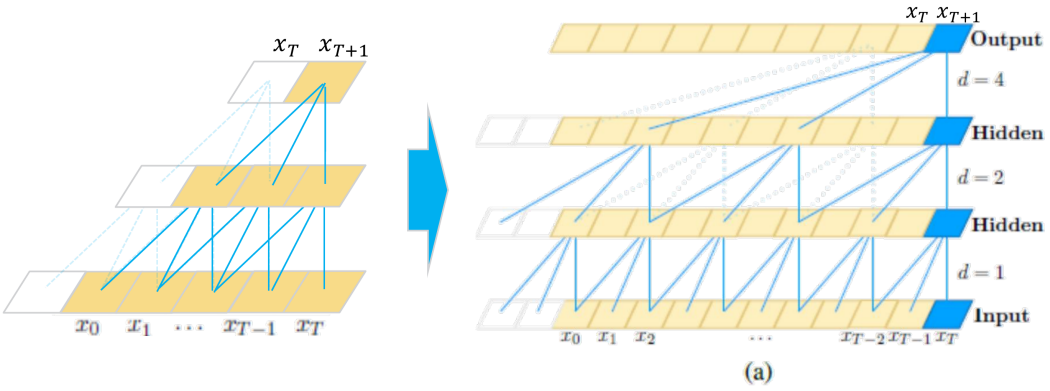


Figure 3.2: Architectural comparison between standard causal convolutions and dilated causal convolutions (Bai et al. [3]). Left: Standard causal convolution where receptive field grows linearly with depth. Right: Dilated causal convolution with exponentially doubling dilation factors ($d = 1, 2, 4$), achieving exponential receptive field expansion without gaps in the input history.

Comparative Architectural Analysis: Standard vs. Dilated Causal Convolutions

Figure 3.2 directly visualizes the paradigm shift from standard causal convolutions (left) to the dilated architecture proposed by Bai et al. [3] (right):

- **Left Panel — Standard Causal Convolution (Linear Interaction Distance):**
  - *Strict Causality:* Time flows from left to right ($x_0 \to x_1 \cdots \to x_T$). Predictions evaluated at time t must never access future observations (x_{t+1}, x_{t+2}). Consequently, the convolutional kernel is strictly causal: connections from upper layers descend exclusively vertically or diagonally towards the left (past), ensuring zero lookahead.
  - *The Receptive Field Bottleneck:* With a standard kernel size K and no dilation ($d = 1$), each additional layer increases the receptive field by only $K - 1$ past time steps:

$$\text{Receptive Field} = 1 + L(K - 1)$$

    As visible in the narrow triangular pyramid on the left, capturing a historical horizon of N time steps requires $L \approx N/(K - 1)$ layers. For long temporal sequences (e.g., $N = 1000$), an impractically deep network of hundreds of layers would be required, introducing severe computational costs and optimization difficulties.
- **Right Panel — Dilated Causal Convolution (Exponential Receptive Field):** Instead of growing the filter size or stacking hundreds of layers, the network keeps small, efficient kernels (e.g., $K = 2$ parameters per channel) while exponentially doubling the dilation factor d at each successive layer:
  1. **Input Layer (bottom):** Represents the raw discrete input sequence $(x_0, x_1, \ldots, x_T)$.
  2. **First Hidden Layer ($d = 1$):** Standard stride-1 causal convolution. Each node aggregates 2 consecutive values (x_t and x_{t-1}).
  3. **Second Hidden Layer ($d = 2$):** The dilation factor doubles to $d = 2$. The filter spans 2 positions spaced by 2 time steps (sampling t and $t - 2$, skipping the intermediate point).
  4. **Output Layer ($d = 4$):** The dilation factor doubles again to $d = 4$. The filter samples features spaced 4 time steps apart (at t and $t - 4$).

- **Tracing the Blue Receptive Tree (Top-Right Node at x_T):** Following the blue highlighted connections backwards in Figure 3.2 illustrates how exponential dilation covers the past:
  - At layer $d = 4$, the top-right output node branches into **2 features** (at T and $T-4$).
  - At layer $d = 2$, each of these two features branches into 2 nodes at stride 2, yielding **4 features** $(T, T-2, T-4, T-6)$.
  - At layer $d = 1$, each of these four features branches into 2 adjacent nodes, covering **8 consecutive input tokens** $(x_{T-7}, x_{T-6}, \ldots, x_T)$.

  With only 3 convolutional layers and $K = 2$, the receptive field spans 8 full time steps.

Key Word

Why Dilation is Logarithmic and Continuous (w/o Gaps)

1. Logarithmic Depth Requirement: Because the effective receptive span doubles with every layer $(1, 2, 4, 8, \ldots)$, the receptive field scales exponentially with depth:

$$\text{Receptive Field} = 1 + (K-1)\sum_{l=0}^{L-1} 2^l = 1 + (K-1)(2^L - 1)$$

To observe a temporal context of length N, the required network depth scales **logarithmically**:

$$L \approx \log_2(N)$$

For instance, while a standard causal convolution would require ≈ 1000 layers to observe 1000 historical steps, a dilated TCN achieves this with just $\lceil \log_2(1000) \rceil = 10$ layers!

2. Coverage Without Gaps (*w/o gaps*): Although individual higher-level filters skip intermediate activations, the hierarchical doubling of dilation factors ($d_l = 2^l$) ensures that every single discrete input token within the receptive span contributes to the final prediction. There are no temporal holes or unobserved steps in the input history.

3.2.3 Trade-offs of Temporal Convolutional Networks

Key Insight

Advantages and Disadvantages of TCNs:

- **Pros:**
  - **Parallelization:** Unlike recurrent architectures, convolutions can be computed concurrently across all time steps during training, drastically accelerating training times.
  - **Local Dependencies:** Convolutions excel at modeling localized temporal features and patterns.
- **Cons:**
  - **Fixed Maximum Memory:** The receptive field is strictly bounded by the network depth and dilation schedule; historical information beyond this window is permanently lost.
  - **Inflexibility at Test Time:** Handling variable sequence lengths or streaming evaluation requires managing historical buffers, whereas recurrent models update a single internal state vector dynamically.

3.3 Recurrent Neural Networks

The primary and historically dominant paradigm for processing sequential data is the **Recurrent Neural Network (RNN)**. Instead of relying on expanding spatial filters, an RNN introduces an internal **hidden state** that acts as a dynamic memory buffer.

3.3.1 The Recurrence Principle

At each time step t, the network processes an incoming input x_t while retaining a memory vector of past computations, denoted as the **hidden state** h_{t-1}. The state is updated via a recurrence formula:

$$h_t = f_W(h_{t-1}, x_t) \tag{3.3}$$

When output predictions are required, the current hidden state is mapped to an output vector:

$$y_t = f_{W_{hy}}(h_t) \tag{3.4}$$

Note

Piece-by-Piece Breakdown of Equations 3.3 and 3.4:

- t: The current discrete time step index ($t = 1, 2, \ldots, T$).
- h_t: The **new hidden state** vector at time t, capturing the historical context up to the current moment.
- h_{t-1}: The **previous hidden state** vector from time step $t-1$ (with h_0 typically initialized to zero).
- x_t: The **input vector** fed into the network at time step t.
- f_W: A parameterized non-linear state transition function. **Crucial property:** *the exact same function f_W and the exact same set of parameters W are reused identically across every time step.*
- y_t: The predicted output vector (e.g., class logits) generated at time step t.
- $f_{W_{hy}}$: The output mapping function parameterized by weights W_{hy}.

3.3.2 The Vanilla RNN Formulation

In the classic formulation (often called the **Vanilla RNN** or **Elman RNN**), the transition function applies affine transformations followed by a hyperbolic tangent (tanh) non-linearity:

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t) \tag{3.5}$$

$$y_t = W_{hy} h_t \tag{3.6}$$

Note

Piece-by-Piece Mathematical Breakdown of Equation 3.5:

- W_{hh}: The **hidden-to-hidden weight matrix**. It transforms the previous memory vector h_{t-1}, governing how historical context propagates into the future.
- W_{xh}: The **input-to-hidden weight matrix**. It projects the incoming input vector x_t into the hidden feature space.
- tanh: The hyperbolic tangent activation function. It squashes state values strictly into the interval

$(-1, 1)$, preventing activations from growing unbounded across successive steps.

- W_{hy}: The **hidden-to-output weight matrix**. It projects the internal hidden representation h_t into the target output space.

3.3.3 Unrolling the Computational Graph

To understand how an RNN is trained, the recurrence is unrolled across time into a directed acyclic computational graph, as shown in Figure 3.3.

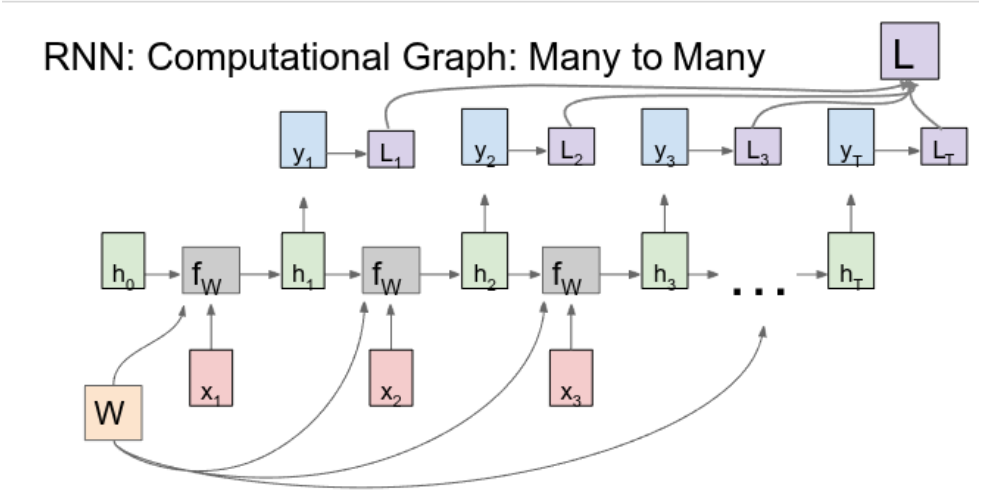


Figure 3.3: Unrolled computational graph of a recurrent neural network across time steps $t = 1, \ldots, T$. Notice that the weight matrix W is reused identically at every time step.

At each time step t, the network receives input x_t, updates its hidden state h_t, and optionally emits an output y_t. If a target label y_t^* is available, a local loss L_t is computed. The total loss across the full sequence of length T is the accumulated sum:

$$L = \sum_{t=1}^{T} L_t$$

3.3.4 Sequence-to-Sequence: Synchronized vs. Encoder-Decoder Mappings

When modeling sequences with inputs and outputs, two distinct structural paradigms emerge from the computational graph, as introduced in the lecture slides (Slides 41–46):

Synchronized Many-to-Many (Slide 41)

In a synchronized recurrence, the network ingests an input x_t and immediately emits a corresponding output y_t at every time step t. While effective for tasks with strict 1:1 temporal alignment (e.g., frame-level video classification or part-of-speech tagging), this architecture fundamentally fails for tasks such as **Machine Translation**:

- **Length Disparity ($T_{\text{in}} \neq T_{\text{out}}$):** Natural languages rarely have matching word counts (e.g., English *"I am hungry"* [3 words] $\longrightarrow$ Italian *"Ho fame"* [2 words]). Synchronous evaluation cannot handle length discrepancies.

- **Structural Word Reordering:** Grammar and syntax vary across languages. In English, adjectives precede nouns (*"the black cat"*), whereas in Italian they follow (*"il gatto nero"*). A synchronized network would be forced to predict the first translated word before even observing the subsequent subject.

Sequence-to-Sequence: Many-to-One + One-to-Many (Slides 45–46)

To resolve these limitations, Sutskever et al. [51] (and Cho et al. [12]) proposed the **Sequence-to-Sequence (Seq2Seq)** framework, decomposing the task into two decoupled recurrent modules operating in cascade, illustrated in Figure 3.4:

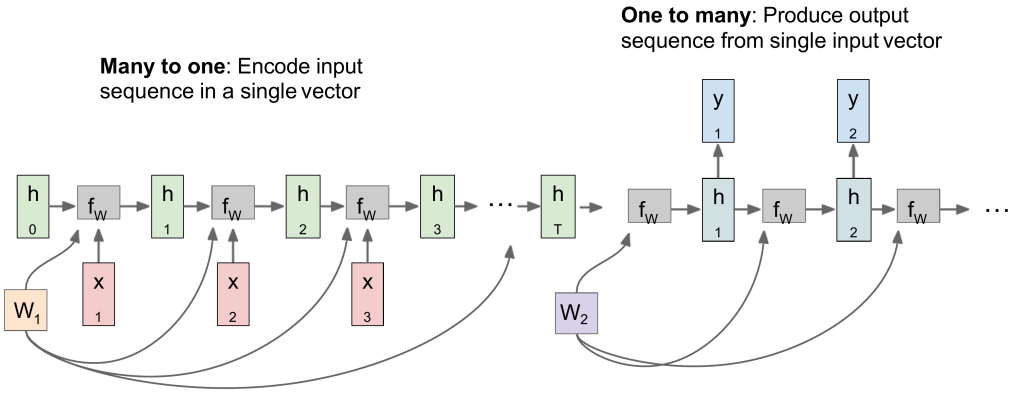


Figure 3.4: Sequence-to-Sequence architecture (Sutskever et al. [51], from course slides). The Encoder (Many-to-One with weights W_1) compresses the source sequence into bottleneck state h_T. The Decoder (One-to-Many with weights W_2) generates the target sequence from h_T until an end-of-sequence token is emitted.

1. **Encoder — Many-to-One (Parameterized by W_1):**
   The encoder consumes the entire input sequence $x_1, \ldots, x_T$ step-by-step without producing intermediate outputs. Its sole purpose is to accumulate the full contextual meaning of the input into the final hidden state vector h_T, known as the **context** or **bottleneck vector**.
2. **Decoder — One-to-Many (Parameterized by W_2):**
   The decoder receives h_T as its initial hidden state and autoregressively generates the output sequence $y_1, y_2, \ldots, y_{T'}$. Generation terminates dynamically when the model emits an end-of-sequence token $\langle \text{END} \rangle$, allowing T' to differ arbitrarily from T.

Key Word

Why Two Distinct Weight Sets ($W_1 \neq W_2$)?

The encoder weights W_1 and decoder weights W_2 are explicitly decoupled:

- W_1 learns temporal dynamics, grammar, and semantic representations of the **source** domain (e.g., parsing English syntax).
- W_2 learns the vocabulary distributions, syntax rules, and generation dynamics of the **target** domain (e.g., generating fluent Italian sentences).

3.3.5 Case Study: Character-Level Language Modeling

To see an RNN in action, consider a character-level language model trained to predict the sequence *"hello"* from a vocabulary of four letters: $\{h, e, l, o\}$ (Slides 47–53; Karpathy [38]). At each step, the network receives the current character and predicts a probability distribution over the vocabulary for the next character.

Training Phase: Teacher Forcing

During training, the complete sequence *"hello"* is known in advance. The model is trained using **Teacher Forcing** (also called *Ground Truth Forcing*), illustrated in Figure 3.5.

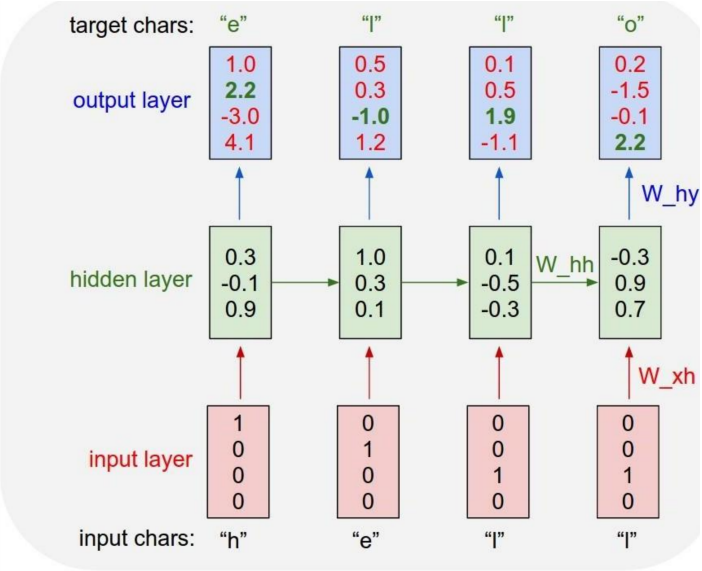


Figure 3.5: Training phase of the character-level RNN with Teacher Forcing (Slide 49). At each step, the ground-truth character from the dataset is fed as input, regardless of what the network predicted at the previous step.

As shown in Figure 3.5:

- **Network Flow:** Each character is represented as a one-hot vector in the input layer (4 red nodes), processed by the recurrent hidden layer (3 green nodes), and projected to raw scores (logits) in the output layer (4 blue nodes).
- **Teacher Forcing:** At each step t, the input is strictly the ground-truth character from the target word:
  - Feed *'h'* $\longrightarrow$ target is *'e'*
  - Feed *'e'* $\longrightarrow$ target is *'l'*
  - Feed *'l'* $\longrightarrow$ target is *'l'*
  - Feed *'l'* $\longrightarrow$ target is *'o'*

  Even if the network makes an incorrect prediction at step 1, step 2 is still fed the true character *'e'*. This prevents errors from accumulating during training.
- **Memory Disambiguates Consecutive Letters:** Notice steps 3 and 4: both receive the identical input vector for *'l'*, but step 3 must predict *'l'* while step 4 must predict *'o'*. A static feedforward network cannot tell them apart. The RNN succeeds because its hidden state acts as memory: step 3 remembers the past context $[h \to e \to l]$, whereas step 4 remembers $[h \to e \to l \to l]$.

Test Phase: Autoregressive Generation

At test time, the model must generate text autonomously without ground-truth answers. It operates in an **Autoregressive Loop**, shown in Figure 3.6.

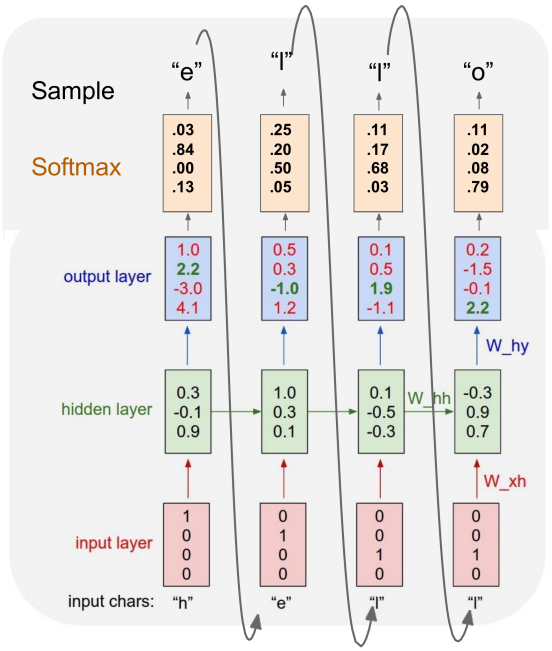


Figure 3.6: Test phase of the character-level RNN (Slide 53). Characters sampled from the output distribution are fed back recursively as inputs for the next time step (curved loop arrows).

The generation proceeds step by step:

- **Recursive Feedback:** Because there is no teacher at test time, the character produced at step t is fed back as the input for step $t+1$, as traced by the curved arrows in Figure 3.6.
- **Sampling from Softmax:**
  1. **Seed Prompt ($t=1$):** We provide an initial character, $x_1 =$ *'h'*. The network computes output scores and converts them to probabilities via Softmax (yielding $P(e) = 0.84$). We sample token *'e'*.
  2. **Step $t=2$:** The sampled *'e'* is fed back as input x_2. Combining it with the internal hidden state, the network predicts probabilities and samples *'l'* ($P(l) = 0.50$).
  3. **Step $t=3$:** The sampled *'l'* is fed back as input x_3, yielding another sampled *'l'* ($P(l) = 0.68$).
  4. **Step $t=4$:** The second *'l'* is fed back as input x_4. Thanks to the internal memory accumulated over time, the network recognizes the completed prefix and samples *'o'* ($P(o) = 0.79$).

Key Word

Training vs. Testing at a Glance

- **Training (Teacher Forcing):** Inputs come from the real dataset. Errors do not compound because the ground truth is provided at every step.
- **Testing (Autoregressive):** Inputs come from the model's own previous outputs. The network generates sequences recursively on its own.

3.3.6 Case Study: Image Captioning (CNN + RNN)

Another foundational application of recurrent models is **Image Captioning** (Slides 112–123, pioneered by Vinyals et al. [55], Karpathy and Fei-Fei [39], and Donahue et al. [18]): given an input image, the network must generate a fluent sentence describing its visual content. This represents a canonical **One-to-Many** task that unifies Computer Vision (CNN) and Natural Language Processing (RNN).

How CNN and RNN Interact

The architecture combines a Convolutional Neural Network (as a visual feature extractor) and a Recurrent Neural Network (as a sequential language generator), as shown in Figure 3.7.

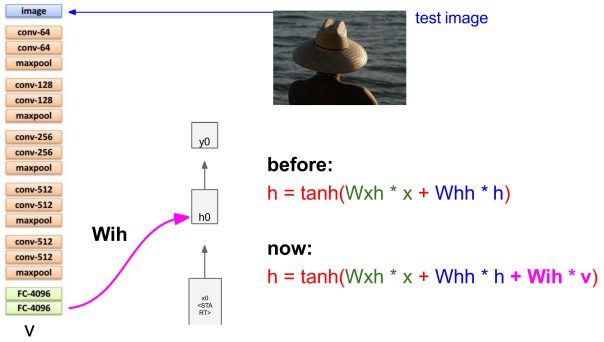


Figure 3.7: Interaction between CNN and RNN for Image Captioning (Slide 118). The CNN extracts visual feature vector v from a pre-trained layer, which is injected into the RNN recurrence via weight matrix W_{ih}.

The interaction between the two networks proceeds as follows:

- **Visual Feature Extraction (CNN):** The input image is fed into a convolutional network (such as a pre-trained VGG-16). Crucially, the final classification layer (which predicts ImageNet class scores with Softmax) is discarded. Instead, we extract the dense feature vector $v \in \mathbb{R}^{4096}$ from the penultimate fully connected layer (`FC-4096`), capturing the global semantic content of the image.
- **The Recurrence Equation (Slide 118):** While a standard RNN computes $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1})$, in Image Captioning the visual feature vector v is added as a dedicated third input term:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + W_{ih}v)$$

Note

What Learns vs. What Remembers (Parameters vs. Context):

- **The Learnable Weights (What changes during training):** The matrices W_{xh}, W_{hh}, W_{ih} (alongside output projection W_{hy}) are the actual parameters of the model. During training via BPTT, backpropagation updates **only these weight matrices**. They learn the optimal transition rules: how to interpret words, how to advance memory, and how to project image features. At test time, all W matrices are frozen.
- **The Hidden State (h_t):** The hidden state does **not** contain learnable weights. It is an activation vector acting as temporary working memory (functioning like a continuous Markovian state) that compresses the visual context v and the sequence of words generated so far. It updates

dynamically at every time step and resets to $\vec{0}$ for each new image.

- **The Inputs (x_t and v):** Vector v is the static visual embedding anchoring the language to the image, while x_t is the text input ($x_0 =$ <START> at $t = 0$, followed by the word sampled at step $t-1$).

Autoregressive Caption Generation

At inference time, the unrolled RNN generates the descriptive sentence word by word, as illustrated in Figure 3.8.

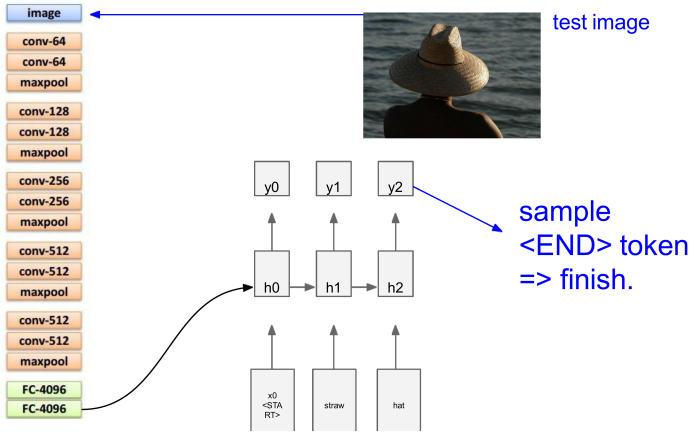


Figure 3.8: Unrolled test-time generation of an image caption (Slide 123). The network generates words sequentially from a starting token until emitting the special <END> token.

At each step, the hidden state h_t projects raw scores (logits) y_t, which are converted into a probability distribution over the word dictionary via Softmax. Generation unfolds sequentially:

1. **Initialization ($t = 0$):** Given visual vector v, initial memory $h_{-1} = \vec{0}$, and start token $x_0 =$ <START>, the network computes h_0 and predicts distribution y_0. The most probable word, *"straw"*, is sampled.
2. **Sequential Steps ($t = 1, 2, \ldots$):** The newly sampled word is fed back as the input for the next time step alongside memory h_t and visual vector v:
   - Feed $x_1 =$ *"straw"* $\longrightarrow$ computes $h_1 \longrightarrow$ samples *"hat"*.
   - Feed $x_2 =$ *"hat"* $\longrightarrow$ computes $h_2 \longrightarrow$ samples <END>.
3. **Stopping Criterion:** Sampling the special token <END> signals that the sentence is complete, terminating the autoregressive loop with final caption *"straw hat"*.

3.4 Backpropagation Through Time and Gradient Flow

Training an RNN requires computing the gradient of the total loss L with respect to the shared weight matrices W. This process is known as **Backpropagation Through Time (BPTT)** [57] (Slides 141–144).

3.4.1 BPTT and Gradient Flow in Vanilla RNNs

BPTT proceeds in two phases:

1. **Forward Pass:** The input sequence is processed step by step, computing intermediate hidden states h_t and loss terms L_t.
2. **Backward Pass:** Gradients are backpropagated backward in time from step $t = T$ to $t = 1$, accumulating gradient contributions to the shared weights at every step.

When backpropagating from hidden state h_t to previous state h_{t-1}, the gradient is multiplied by the recurrent weight matrix W_{hh}^T at each step (Slides 145–147).

3.4.2 Exploding and Vanishing Gradients

Backpropagating gradients across multiple time steps involves repeated multiplication by W_{hh} (neglecting the non-linearity for simplicity). As illustrated in Figure 3.9, this repeated multiplication leads to two failure modes (Slide 157):

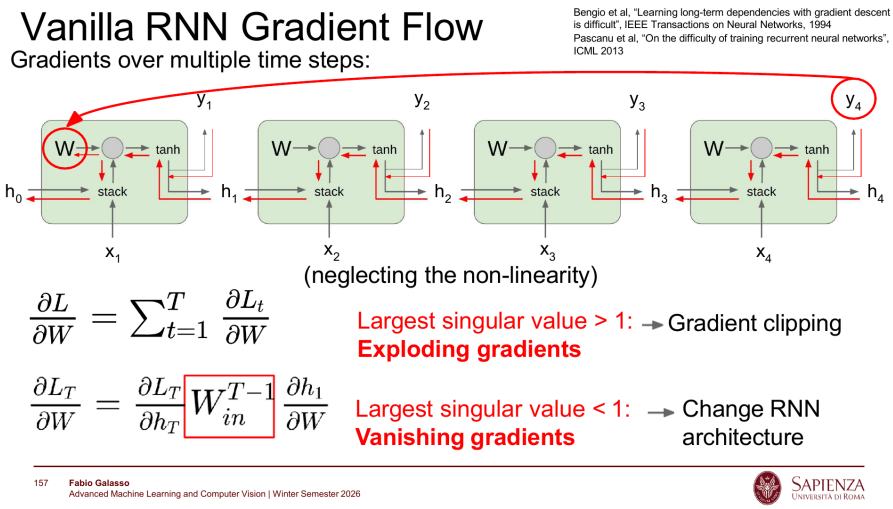


Figure 3.9: Gradient flow across multiple time steps in a Vanilla RNN (Slide 157). Repeated multiplication by W_{hh} inevitably causes either exploding or vanishing gradients.

- **Exploding Gradients (Largest singular value > 1):** Gradients grow exponentially with sequence length, causing numerical instability.
  Solution: *Gradient Clipping* (Pascanu et al., 2013) [45]. When the gradient norm $\|g\|$ exceeds a threshold τ, it is clipped:
  $$g \longleftarrow \frac{\tau}{\|g\|} g \quad \text{if } \|g\| > \tau$$
- **Vanishing Gradients (Largest singular value < 1):** Gradients decay exponentially toward zero as they travel back in time. Early weights receive negligible updates, making it practically impossible to learn long-term dependencies.
  Solution: Vanishing gradients cannot be fixed by gradient clipping; they require redesigning the architecture with additive interactions $\longrightarrow$ **LSTM**.

3.4.3 Vanilla RNN Trade-offs

The strengths and limitations of Vanilla RNNs can be summarized as follows (Slide 158):

Key Insight

Vanilla RNN Trade-offs (Slide 158):

- **Advantages:**
  - Can process input sequences of arbitrary length.
  - Model parameter size remains strictly constant regardless of input length.
  - Same weights are applied at every time step, providing temporal symmetry.
  - In theory, computation at step t can use information from many steps back.
- **Disadvantages:**
  - Recurrent computation is slow and cannot be parallelized across time.
  - In practice, vanishing gradients make it difficult to access information from many steps back.

3.5 Long Short-Term Memory

To resolve the fatal vanishing gradient problem inherent in Vanilla RNNs, Hochreiter and Schmidhuber [36] introduced the **Long Short-Term Memory (LSTM)** network.

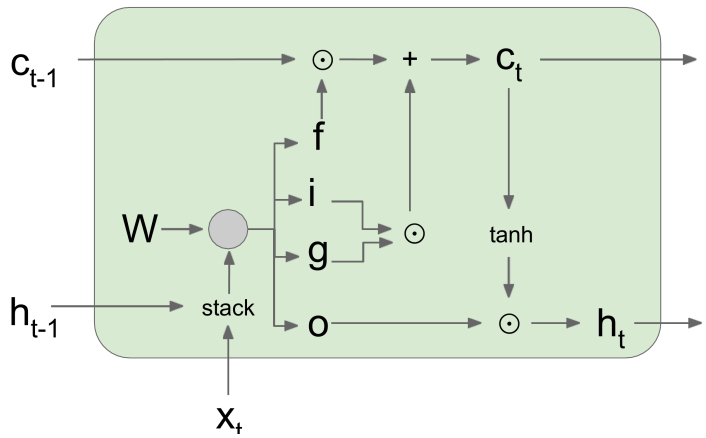
3.5.1 The Dual-State Architecture

While a Vanilla RNN maintains only a single hidden state h_t, an LSTM explicitly decouples memory into two complementary internal state vectors:

1. **Hidden State (h_t):** The short-term working state, used to emit immediate predictions and interface with adjacent layers.
2. **Cell State (c_t):** The long-term persistent memory. It functions as an uninterrupted information highway where memories can flow through time with minimal linear attenuation.

3.5.2 The LSTM Cell Architecture

Figure 3.10 illustrates the internal structure of the LSTM cell and its governing equations.



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Figure 3.10: The LSTM cell diagram and its update equations (from course slides). At time step t, the cell stacks h_{t-1} and x_t to compute four gates (f, i, g, o), updating the cell state c_t additively and producing the hidden state h_t.

3.5.3 Detailed Breakdown of Cell Components

To understand how information flows through the cell in Figure 3.10, we can trace each input, parameter, gate, and intermediate operation:

- **Inputs to the Cell:**
  - x_t: The input vector at the current time step t (e.g., word embedding or extracted visual feature vector).
  - h_{t-1}: The hidden state from the previous time step $t-1$, providing short-term context.
  - c_{t-1}: The cell state from the previous time step $t-1$, carrying long-term memory along the upper horizontal highway.
- **Vector Stacking and Weight Matrix W:**
  - $\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$ (**stack**): Concatenates h_{t-1} and x_t into a single composite vector.
  - W: The learnable weight matrix of the cell. Multiplying W by the stacked input computes all four gate activations simultaneously in a single linear transformation:

$$\begin{pmatrix} f \\ i \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

- **The Four Gating Mechanisms (f, i, g, o):**
  - f (**Forget Gate**, σ): *Whether to erase cell memory*. Uses the Sigmoid activation $\sigma(z) \in [0,1]$. Acting as a soft valve, an activation close to 0 clears that memory dimension, whereas 1 preserves it completely.
  - i (**Input Gate**, σ): *Whether to write to the cell*. Uses Sigmoid $\sigma(z) \in [0,1]$. It acts as a filter deciding which portions of the incoming information are relevant enough to be stored.
  - g (**Gate Gate / Candidate State**, tanh): *What candidate information to write*. Uses $\tanh(z) \in (-1,1)$ to propose new candidate values to be written into the cell state.
  - o (**Output Gate**, σ): *How much of the cell to reveal*. Uses Sigmoid $\sigma(z) \in [0,1]$ to control how much of the internal cell state is exposed to the outside as the working hidden state h_t.
- **Element-Wise Operations and State Updates:**
  - $\odot$ (**Hadamard Product**): Pointwise vector multiplication used for gating. Multiplying by a $[0,1]$ gate value smoothly scales each dimension.
  - $f \odot c_{t-1}$: Forget gate modulation. It selectively suppresses outdated memories from the previous cell state.
  - $i \odot g$: Input gate modulation. It scales the candidate information g by the input gate i, letting only selected features enter the memory.
  - $+$ (**Additive Interaction**): Merges the preserved past memory with the new candidate memory:

$$c_t = f \odot c_{t-1} + i \odot g$$

    This addition is the core innovation of the LSTM: it bypasses the repeated multiplicative weight transformations of vanilla RNNs.
  - $\tanh(c_t)$: Squashes the accumulated cell state values into $(-1,1)$ before filtering for output.
  - $h_t = o \odot \tanh(c_t)$: Produces the current hidden state h_t, which is both routed to the next time step $t+1$

and emitted upwards to produce predictions or feed higher layers.

3.5.4 Why LSTM Solves the Vanishing Gradient Problem

The fundamental reason LSTMs succeed where Vanilla RNNs fail lies in the gradient flow along the cell state highway, illustrated in Figure 3.11.

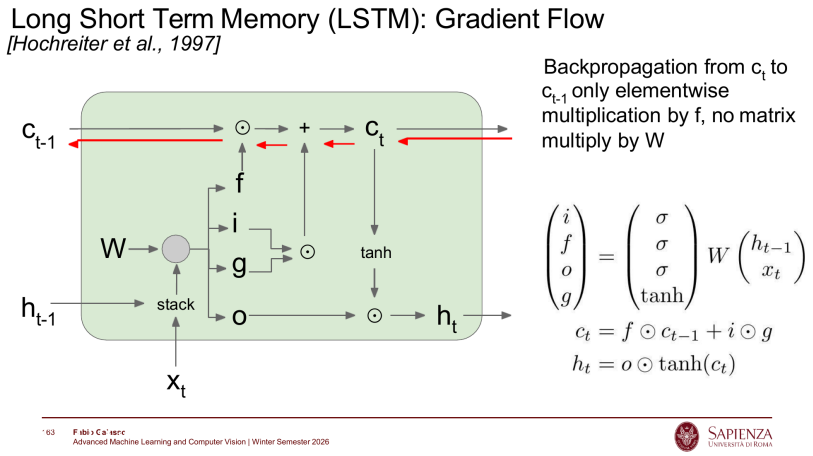


Figure 3.11: Uninterrupted gradient flow along the cell state highway in an LSTM (from course slides). Backpropagation from c_t to c_{t-1} involves only element-wise multiplication by f, avoiding matrix multiplications by W.

Key Insight

The Uninterrupted Gradient Highway: Why LSTM Overcomes Vanishing Gradients

In a Vanilla RNN, backpropagating through time repeatedly multiplies error gradients by the recurrent weight matrix W_{hh}^T and saturating tanh derivatives, causing the signal to decay exponentially towards zero ($W_{hh}^T \cdot W_{hh}^T \cdots \to 0$).

In an LSTM, backpropagating from c_t to c_{t-1} along the upper cell state highway yields $\frac{\partial c_t}{\partial c_{t-1}} = f$:

- **No Weight Matrix Multiplications:** Backpropagation along the highway involves only an element-wise multiplication by the forget gate vector f, with **no matrix multiplication by** W. There is no risk of exponential degradation caused by repeated matrix powers.
- **Selective Memory Retention ($f \approx 1$):** The forget gate outputs values in $[0,1]$. If the network learns that an early piece of context is critical and sets $f \approx 1$, the local gradient derivative is practically 1.
- **Lossless Flow Across Hundreds of Steps:** Multiplying by 1 repeatedly preserves the error magnitude:

$$1 \times 1 \times \cdots \times 1 = 1$$

  The gradient signal flows backwards through dozens or hundreds of time steps virtually intact!
- **The ResNet Analogy:** Just as ResNets use additive identity skip connections ($x + F(x)$) to bypass deep spatial layers, LSTMs use an additive cell state ($f \odot c_{t-1} + i \odot g$) to bypass long temporal sequences.

3.6 Chapter Summary and Conceptual Graph

The concepts developed in this chapter trace the evolution of sequence models from parallel temporal convolutions to recurrent memory networks and additive gradient highways:

Key Insight

The Logical Connections Between Nodes:

- **From Node 1 to 2:** Handling variable-length sequences with temporal ordering prompts using 1D convolutions; enforcing causality prevents future lookahead, while dilation expands context without sacrificing parallel training.
- **From Node 2 to 3:** Despite their parallel speed, TCNs have a fixed receptive field buffer; Recurrent Neural Networks introduce true dynamic memory via an evolving hidden state h_t with parameter sharing.
- **From Node 3 to 4:** Training recurrent graphs requires unrolling them through time and calculating loss gradients backwards across all time steps via Backpropagation Through Time (BPTT).
- **From Node 4 to 5:** Unrolling across long horizons exposes BPTT to repeated multiplications by W_{hh}^{T-1}. If the spectral radius exceeds 1, gradients explode (controlled via Gradient Clipping); if less than 1, gradients vanish exponentially.
- **From Node 5 to 6:** Because vanishing gradients represent an architectural failure of Vanilla RNNs, Hochreiter and Schmidhuber redesigned the memory cell: separating working state h_t from a dedicated cell state c_t.
- **From Node 6 to 7:** Regulating the cell state via input, forget, and output gates (i, f, o) and applying purely additive updates $(+)$ establishes an uninterrupted gradient highway, allowing LSTMs to preserve long-range dependencies across hundreds of steps.

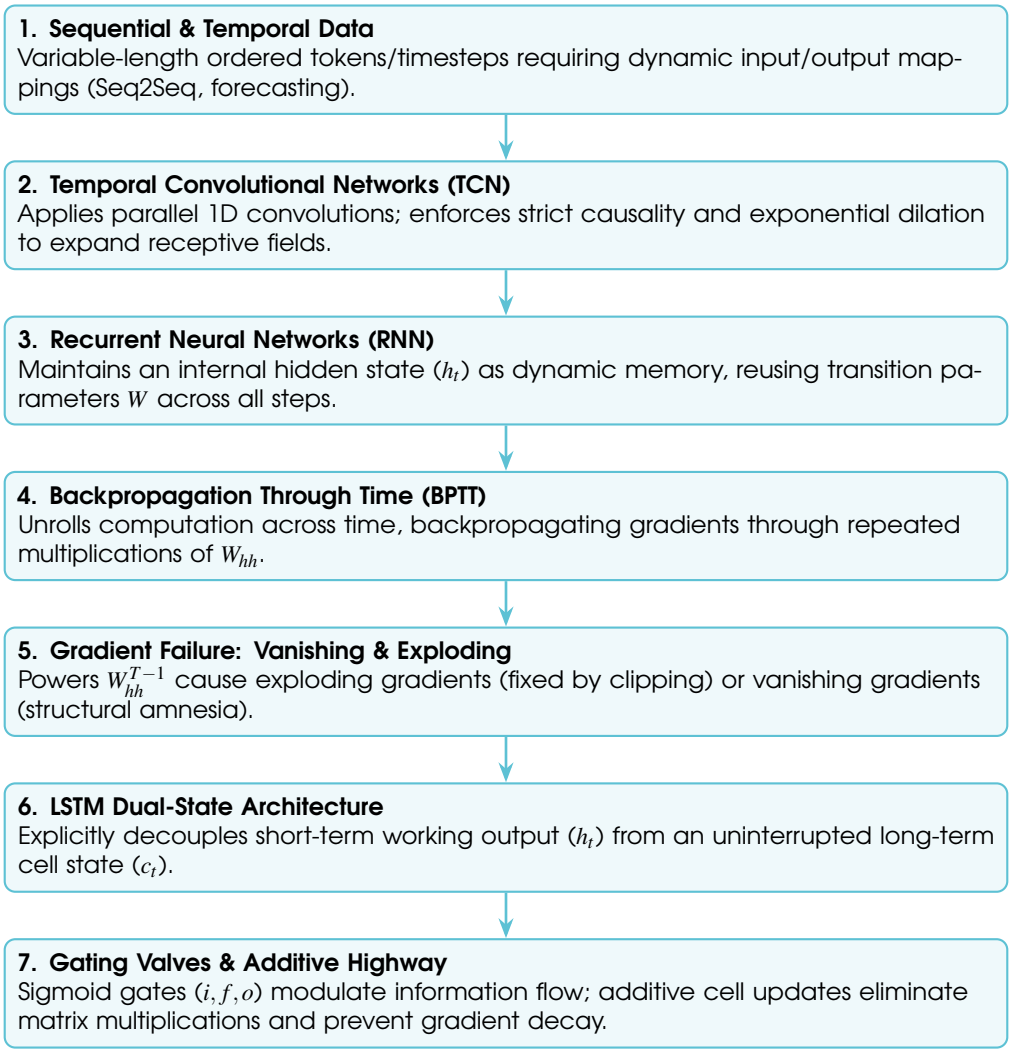


Figure 3.12: Conceptual roadmap of sequence modeling, recurrent dynamics, and LSTMs.

4. Attention Mechanisms and Transformer Netw

4.1 Introduction: Transcending the Recurrent Bottleneck

In Chapter 3, we explored sequential modeling through the lens of recurrence. While architectures such as Long Short-Term Memory (LSTM) alleviate the vanishing gradient problem via an additive cell state highway, recurrent systems remain fundamentally constrained by their temporal formulation:

1. **The Information Bottleneck:** In standard sequence-to-sequence networks, the encoder squashes an arbitrary-length input sequence $\{x_1, \ldots, x_T\}$ into a single fixed-dimensional bottleneck vector $c = h_T$. As sequence length grows ($T \geq 30$), compressing rich sentence context into a static vector causes catastrophic information loss.
2. **Sequential Computational Dependency ($O(n)$):** Recurrent computations are strictly serialized across time: hidden state h_t cannot be computed until h_{t-1} is fully evaluated. Consequently, training cannot be parallelized across sequence length on modern hardware accelerators.
3. **Linear Interaction Path Length ($O(n)$):** Information transfer between distant tokens must propagate step-by-step through intermediate hidden states, increasing susceptibility to signal attenuation and gradient decay.

This chapter details the paradigm shift that overcame these structural limits: the invention of the **Attention Mechanism** and the emergence of the **Transformer** architecture, based on the course lecture modules by Prof. Fabio Galasso [26].

4.2 From Recurrence to Attention

4.2.1 The Information Bottleneck in Vanilla Seq2Seq

As formalized by Sutskever et al. [51], a standard Sequence-to-Sequence (Seq2Seq) model maps an input sequence $x = (x_1, \ldots, x_T)$ into an output sequence $y = (y_1, \ldots, y_{T'})$ using an Encoder-Decoder framework:

$$h_t = f_W(x_t, h_{t-1}) \quad \text{(Encoder)} \tag{4.1}$$

$$s_t = g_U(y_{t-1}, s_{t-1}, c) \quad \text{(Decoder)} \tag{4.2}$$

where:

- f_W denotes the recurrent transition function of the encoder (e.g. an RNN, LSTM, or GRU cell) parameterized by its collection of learnable weights W.
- g_U denotes the recurrent transition function of the decoder, parameterized by its own set of learnable weights U.
- $c = h_T$ is the static context vector passed from the final encoder step to the decoder.

When translating long documents ($T \geq 30$), forcing the entire semantic representation of the source sentence through the single fixed-dimensional vector h_T constitutes an unsustainable **information bottleneck**.

4.2.2 The Bahdanau Additive Attention Mechanism

To resolve this limitation, Bahdanau et al. [2] introduced the **Attention Mechanism**. Rather than relying on a static context vector, the decoder dynamically computes a **customized context vector** c_t at every decoding time step t, as illustrated in Figure 4.1.

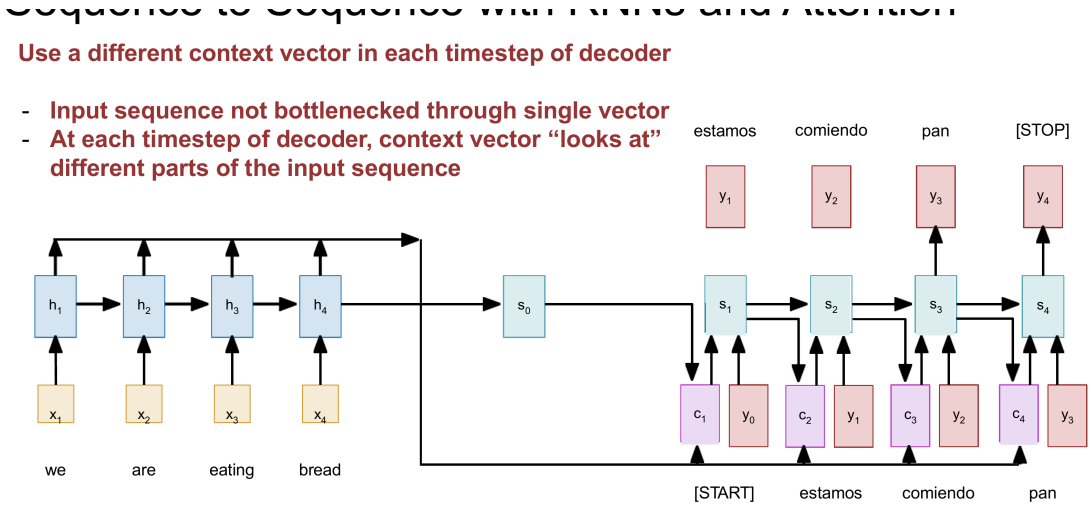


Figure 4.1: Unrolled Sequence-to-Sequence architecture with dynamic attention (from course slides [26]). At each decoding step t, the decoder generates a customized context vector c_t that selectively attends to different parts of the input sequence.

The attention mechanism operates via four sequential steps:

1. **Alignment Scores ($e_{t,i}$):** The model evaluates how well the current decoder state s_{t-1} matches each encoder

hidden state h_i:

$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i) \tag{4.3}$$

where f_{att} is parameterized as a small feed-forward Multi-Layer Perceptron (MLP) trained jointly with the rest of the network.

2. **Attention Weights ($\alpha_{t,i}$):** The scalar alignment scores are normalized across all input positions $i \in \{1, \dots, T\}$ via a Softmax operation:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{k=1}^{T} \exp(e_{t,k})}, \quad \text{such that } 0 < \alpha_{t,i} < 1 \text{ and } \sum_{i=1}^{T} \alpha_{t,i} = 1 \tag{4.4}$$

The weight $\alpha_{t,i}$ represents the probability or relative importance assigned to input token x_i when generating output word y_t.

3. **Dynamic Context Vector (c_t):** The context vector is computed as a weighted average over all encoder hidden states:

$$c_t = \sum_{i=1}^{T} \alpha_{t,i} h_i \tag{4.5}$$

4. **Decoder Recurrent Update:** The new decoder state and output token probabilities are computed conditioning on c_t:

$$s_t = g_U(y_{t-1}, s_{t-1}, c_t), \quad P(y_t \mid y_{<t}, x) = \text{softmax}(W_v s_t) \tag{4.6}$$

Note

End-to-End Differentiability: Every operation in the attention formulation (affine projections, Softmax, weighted averaging) is fully differentiable. No explicit target supervision is needed for the attention weights $\alpha_{t,i}$: backpropagation through the cross-entropy translation loss trains the attention mechanism to discover linguistic alignments automatically.

4.2.3 Step-by-Step Walkthrough: Dynamic Alignment at Decoding Step $t = 2$

To fully appreciate the operational mechanics of the attention mechanism, we trace the concrete translation example presented by Prof. Galasso [26], based on Bahdanau et al. [2]: translating the English sentence *"we are eating bread"* into Spanish (*"estamos comiendo pan"*).

Figure 4.2 details the exact state of the network at decoding step $t = 2$, where the model generates the target word *"comiendo"*.

The decoding step unfolds across five distinct computational stages:

1. **Source Encoding ($T_x = 4$):** The input tokens $x_1 =$ *"we"*, $x_2 =$ *"are"*, $x_3 =$ *"eating"*, and $x_4 =$ *"bread"* are processed sequentially by the recurrent encoder, generating contextual hidden states $h_1, h_2, h_3, h_4 \in \mathbb{R}^d$. Crucially, unlike vanilla Seq2Seq, **all four hidden states are retained in memory** throughout the decoding phase.
2. **Decoder Prior State at Step $t = 1$:** At step $t = 1$, the decoder received the initial start token $y_0 =$ `[START]` and initial context c_1. It generated the first Spanish token $y_1 =$ *"estamos"* (translating the subject and auxiliary verb *"we are"*), updating its recurrent hidden state to s_1. State s_1 now encodes the working memory that the plural first-person continuous subject has been established and an active verb is required next.

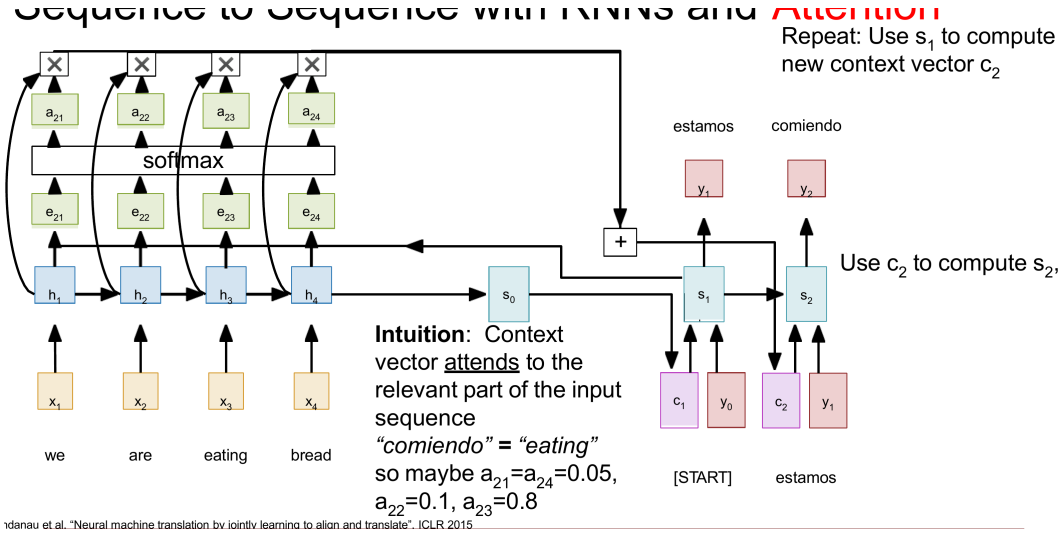


Figure 4.2: Detailed walkthrough of the Attention mechanism at decoding step $t = 2$ (from course slides [26, 2]). The decoder utilizes hidden state s_1 to evaluate alignment scores across all encoder hidden states $h_1, \ldots, h_4$. After Softmax normalization, the model concentrates 80% of its attention weight on h_3 (*"eating"*), synthesizing context vector c_2 to predict $y_2 =$ *"comiendo"*.

3. **Querying Encoder States via Alignment Scores ($e_{2,i}$):** To discover which English word corresponds to the upcoming action, decoder state s_1 acts as a **query** against each encoder state h_i:

$$e_{2,i} = f_{\text{att}}(s_1, h_i) = v_a^\top \tanh(W_a s_1 + U_a h_i), \quad \text{for } i \in \{1,2,3,4\} \tag{4.7}$$

   The feed-forward scoring network evaluates the semantic compatibility between what the decoder needs next (s_1) and what each source position represents (h_i).

4. **Softmax Normalization and Weight Distribution ($\alpha_{2,i}$):** The raw scalar alignment scores are exponentiated and normalized via Softmax to yield a strict probability distribution:

$$\alpha_{2,i} = \frac{\exp(e_{2,i})}{\sum_{k=1}^{4} \exp(e_{2,k})} \tag{4.8}$$

   As highlighted in the course slides, the learned weights reflect the underlying linguistic correspondence:
   - $\alpha_{2,1} = 0.05$ (weight assigned to *"we"*)
   - $\alpha_{2,2} = 0.10$ (weight assigned to *"are"*)
   - $\alpha_{2,3} = \mathbf{0.80}$ **(dominant attention concentrated on *"eating"*)**
   - $\alpha_{2,4} = 0.05$ (weight assigned to *"bread"*)

   Because the model must produce the Spanish gerund *"comiendo"*, the alignment network concentrates 80% of the probability mass directly on token x_3 (*"eating"*).

5. **Dynamic Context Synthesis (c_2) and the Recurrent Update Function g_U:** The customized context vector c_2 is synthesized as the weighted linear combination:

$$c_2 = 0.05\,h_1 + 0.10\,h_2 + \mathbf{0.80\,h_3} + 0.05\,h_4 \;\approx\; h_3 \tag{4.9}$$

Vector c_2 functions as a soft, differentiable spotlight: irrelevant past tokens (*"we", "are"*) and future objects (*"bread"*) are heavily suppressed, channeling pure semantic information about the eating action into the decoder.

The decoder state is then updated via the recurrent transition operator:

$$s_2 = g_U(y_1, s_1, c_2) \tag{4.10}$$

To understand the concrete mechanics of g_U, note that it integrates three distinct sources of information:

- y_1: The embedding of the previously generated token (*"estamos"*).
- s_1: The previous recurrent hidden state (carrying decoder past history).
- c_2: The dynamic context vector synthesized by the attention mechanism.

If the decoder is implemented as a standard recurrent layer, the transition function g_U expands explicitly as:

$$s_2 = \tanh\Big(\underbrace{U_{ss}\, s_1}_{\text{previous state}} + \underbrace{U_{ys}\, y_1}_{\text{previous token}} + \underbrace{U_{cs}\, c_2}_{\text{attention context}} + b_U \Big) \tag{4.11}$$

where $U = \{U_{ss}, U_{ys}, U_{cs}, b_U\}$ denotes the collection of learnable projection matrices and bias terms of the decoder cell. In gated implementations (such as the Gated Recurrent Unit used by Bahdanau et al.), g_U represents the gate equations (reset and update gates) operating over the concatenated vector $[y_1; s_1; c_2]$.

Conditioned on the updated state s_2, the linear vocabulary head projects s_2 through Softmax to emit:

$$y_2 = \text{"comiendo"} \tag{4.12}$$

Key Insight

Why Vanilla Seq2Seq Fails Where Attention Succeeds:

In a vanilla Seq2Seq network without attention, the decoder at step $t = 2$ would have access only to the static bottleneck vector $c = h_4$ (the state produced after processing *"bread"*). Because h_4 is separated from *"eating"* by subsequent recurrent steps, its representation suffers from recency bias and information decay. The decoder is forced to remember the entire sentence structure from a single vector.

With attention, the model bypasses temporal distance entirely: at time step $t = 2$, it establishes a direct, $\mathcal{O}(1)$ highway straight to h_3, retrieving the exact information needed without memory degradation.

4.2.4 Interpreting Attention Matrices and Word Reordering

A remarkable consequence of the attention mechanism is its intrinsic interpretability. By visualizing the matrix of attention weights $\alpha_{t,i} \in \mathbb{R}^{T' \times T}$, one can directly observe how the model aligns source and target tokens, as shown in Figure 4.3.

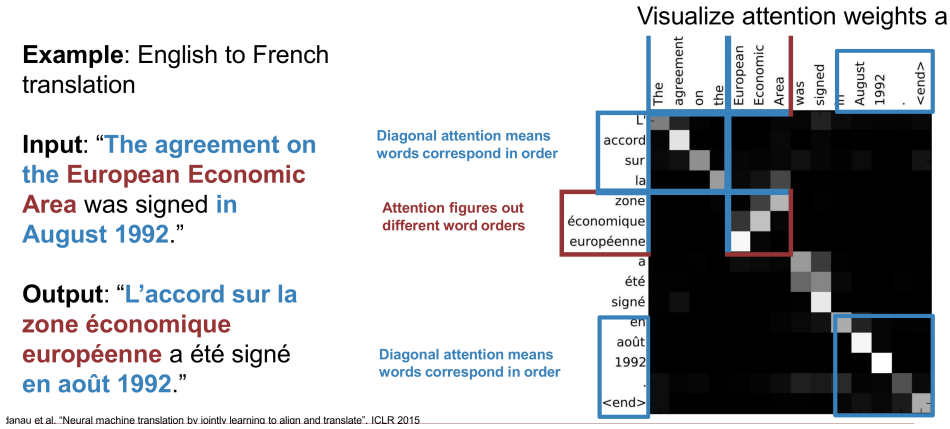


Figure 4.3: Visualization of attention alignment weights for English-to-French translation (Bahdanau et al. [2], from course slides). Strong diagonal values indicate sequential correspondence, while off-diagonal elements show grammatical word reordering.

Key Word

Monotonic Alignments versus Non-Monotonic Reordering

As illustrated in the attention matrix of Figure 4.3, when languages share syntactic ordering, attention concentrates along the main diagonal. However, when syntactic structures differ (e.g., English *"European Economic Area"* $\longrightarrow$ French *"zone économique européenne"*), the attention matrix exhibits an off-diagonal transpose: the model correctly looks ahead to translate *"zone"* before returning to attend to *"économique"* and *"européenne"*.

Key Insight

The Unordered Set Property (Slide 27):

Notice an unexpected theoretical property of the attention decoder: it does not rely on the fact that the encoder vectors h_i form a temporally ordered sequence. The decoder treats them purely as an **unordered set of feature vectors** $\{h_i\}$!

This insight proves revolutionary: because attention operates over arbitrary unordered sets, the same mechanism can be directly applied to sets of spatial image features, audio tokens, or graph nodes.

4.2.5 Spatial Attention in Computer Vision: Show, Attend and Tell

Xu et al. [59] generalized Bahdanau's sequence attention to multi-modal computer vision in the celebrated *"Show, Attend and Tell"* architecture, illustrated in Figure 4.4.

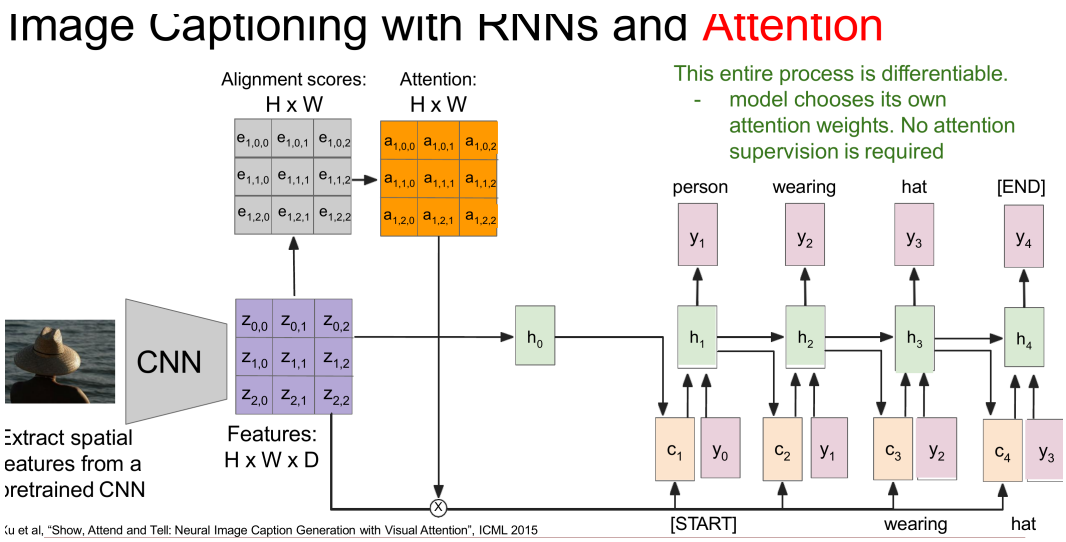


Figure 4.4: Image captioning with spatial 2D visual attention (Xu et al. [59], from course slides). Instead of a single image vector, the decoder computes 2D attention maps over a grid of spatial feature vectors $z_{i,j}$, dynamically focusing on salient image regions for each word.

Rather than extracting a single global vector (such as `FC-4096` from VGG in Chapter 3), a convolutional backbone extracts a 3D spatial tensor of feature vectors $z \in \mathbb{R}^{H \times W \times D}$. At each time step t:

1. Scalar alignment scores are computed across the 2D grid: $e_{t,i,j} = f_{\text{att}}(h_{t-1}, z_{i,j})$.
2. Spatial attention weights are computed via 2D Softmax: $\alpha_{t,i,j} = \frac{\exp(e_{t,i,j})}{\sum_{u,v} \exp(e_{t,u,v})}$.
3. A localized visual context vector is accumulated: $c_t = \sum_{i=1}^{H} \sum_{j=1}^{W} \alpha_{t,i,j} z_{i,j}$.

As the decoder outputs words sequentially (*"person"* $\longrightarrow$ *"wearing"* $\longrightarrow$ *"hat"*), the attention maps visibly shift their spatial focus across the corresponding regions of the input image.

4.3 Self-Attention and Scaled Dot-Product Mechanics

4.3.1 From General Attention to Self-Attention

While standard attention computes interactions between two distinct sequences (decoder states querying encoder states), **Self-Attention** computes interactions between elements within the **same** sequence. Each token is updated by comparing itself against all other tokens in the input.

4.3.2 The Query, Key, Value (Q, K, V) Formulation

Given an input matrix $X \in \mathbb{R}^{N \times D}$ representing N tokens of dimension D, the self-attention layer linearly projects each vector into three distinct roles through learnable parameter matrices:

- **Query** ($Q = XW_Q \in \mathbb{R}^{N \times D_k}$)**:** Represents what the current token is looking for.
- **Key** ($K = XW_K \in \mathbb{R}^{N \times D_k}$)**:** Represents the identity or content descriptor against which queries are compared.
- **Value** ($V = XW_V \in \mathbb{R}^{N \times D_v}$)**:** The actual informational content retrieved and blended into the updated representation.

The pairwise alignment score between token j (query) and token i (key) is evaluated via their dot product,

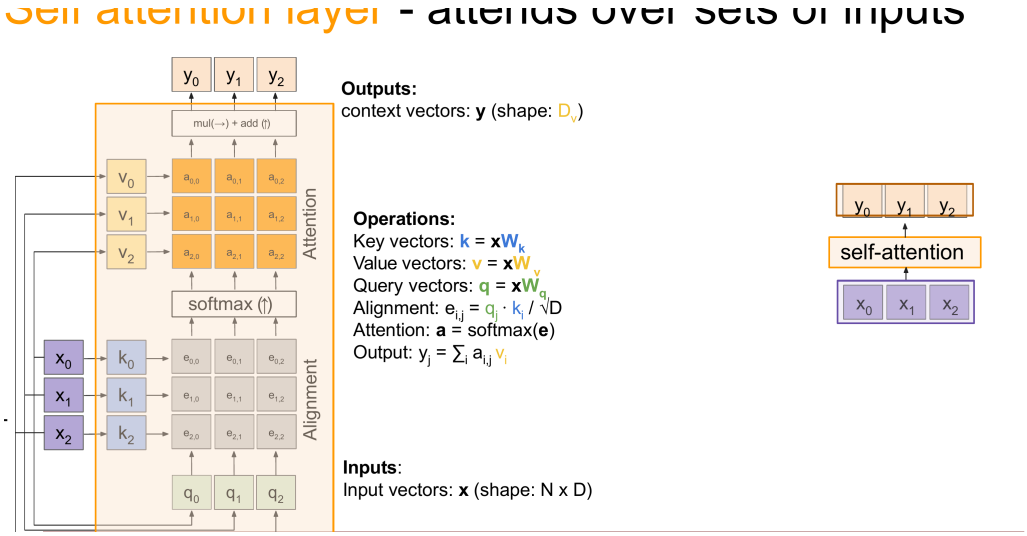


Figure 4.5: The Self-Attention layer structure (from course slides). Input vectors are projected into Queries, Keys, and Values. Alignment scores are scaled, normalized via Softmax, and multiplied by Value vectors to produce updated context vectors.

scaled by the square root of the key dimension D_k:

$$e_{i,j} = \frac{q_j^T k_i}{\sqrt{D_k}} \tag{4.13}$$

In compact matrix notation, the **Scaled Dot-Product Attention** introduced by Vaswani et al. [54] is evaluated simultaneously across all tokens:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{D_k}}\right)V \tag{4.14}$$

Note

Why Divide by $\sqrt{D_k}$?

Assume the components of query q and key k are independent random variables with zero mean and unit variance. Their dot product $q^T k = \sum_{m=1}^{D_k} q_m k_m$ has mean 0 and variance D_k.

For large dimensions (e.g., $D_k = 64$ or $D_k = 128$), the variance of the dot products becomes substantial ($D_k \gg 1$), resulting in large positive or negative logits. Passing large inputs into Softmax forces its outputs to saturate near 0 or 1, causing the local gradient of Softmax to collapse exponentially toward zero. Dividing by $\sqrt{D_k}$ normalizes the variance back to 1, preserving healthy gradient flow throughout training.

4.3.3 Maximum Path Length and Computational Complexity

The structural dominance of self-attention over recurrent and convolutional layers stems from its **constant path length** and **unconstrained parallelizability**, illustrated in Figure 4.6.

Table 4.1 synthesizes the asymptotic properties from course Slide 45 (Vaswani et al. [54]):

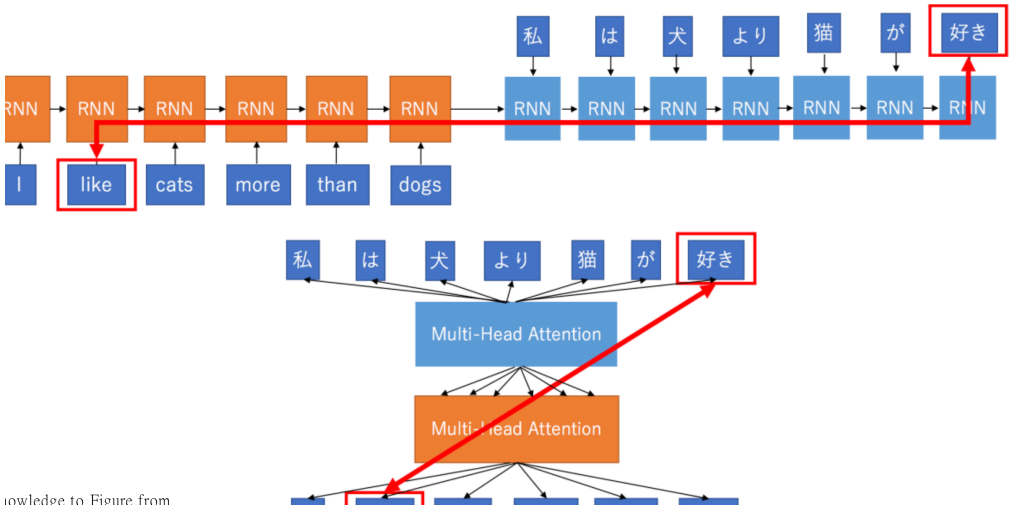


Figure 4.6: Maximum path length comparison between RNN and Self-Attention (from course slides). In an RNN, connecting position 1 to position n requires $O(n)$ sequential hops. In Self-Attention, any pair of tokens communicates directly in a single $O(1)$ constant step.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent (RNN/LSTM)	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$

Table 4.1: Comparative analysis of Self-Attention, Recurrence, and Convolutions.

- **Sequential Operations ($O(1)$ vs $O(n)$):** While an RNN must evaluate n sequential steps to process n tokens, Self-Attention computes all $n \times n$ attention scores in parallel via matrix multiplications.
- **Maximum Path Length ($O(1)$ vs $O(n)$):** The shortest path that gradient signals must travel between any two arbitrary positions is strictly 1. There is no intermediate chain of recurrent weights, eliminating structural vanishing gradients across long temporal horizons.
- **Memory Scalability Trade-off ($O(n^2 \cdot d)$):** Because self-attention computes an $n \times n$ attention matrix, memory consumption grows quadratically with sequence length n. While efficient when $n < d$ (e.g., $n = 512, d = 1024$), ultra-long sequences require specialized sparse or linear attention formulations.

4.4 Positional Encodings, Masking, and Multi-Head Attention

4.4.1 Permutation Equivariance: Why Pure Attention Ignores Order

A fundamental property of the self-attention formula (Equation 4.14) is that it is **permutation equivariant**:

$$\text{Attention}(PQ, PK, PV) = P\,\text{Attention}(Q, K, V) \tag{4.15}$$

for any permutation matrix P. If we shuffle the order of the input tokens, the output tokens are shuffled in the exact same manner without changing their computed values. Because natural language and physical signals depend critically on temporal or spatial order, pure self-attention requires an explicit mechanism to inject positional awareness.

4.4.2 Positional Encodings (PE)

To convey token order, a positional vector $p_j \in \mathbb{R}^D$ is added directly to each input embedding x_j:

$$\widetilde{x}_j = x_j + p_j \tag{4.16}$$

Course Slide 50 formalizes the four core desiderata for any positional encoding function $\text{pos}(j)$:

1. It must output a unique encoding for each time step j.
2. The geometric distance between any two time steps $(j, j+k)$ must remain consistent across sentences of varying lengths.
3. It must generalize effortlessly to longer sequence lengths at test time, producing bounded activation values.
4. It must be deterministic.

Two primary paradigms exist for generating p_j:

- **Learned Positional Embeddings:** A trainable lookup table of shape $(T_{\max} \times D)$ is optimized alongside model parameters. While flexible, it cannot generalize beyond the maximum sequence length $T_{\max}$ observed during training.
- **Sinusoidal Positional Encoding (Vaswani et al. [54]):** Pre-computed using sine and cosine functions of varying frequencies, illustrated in Figure 4.7:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/D}}\right) \tag{4.17}$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/D}}\right) \tag{4.18}$$

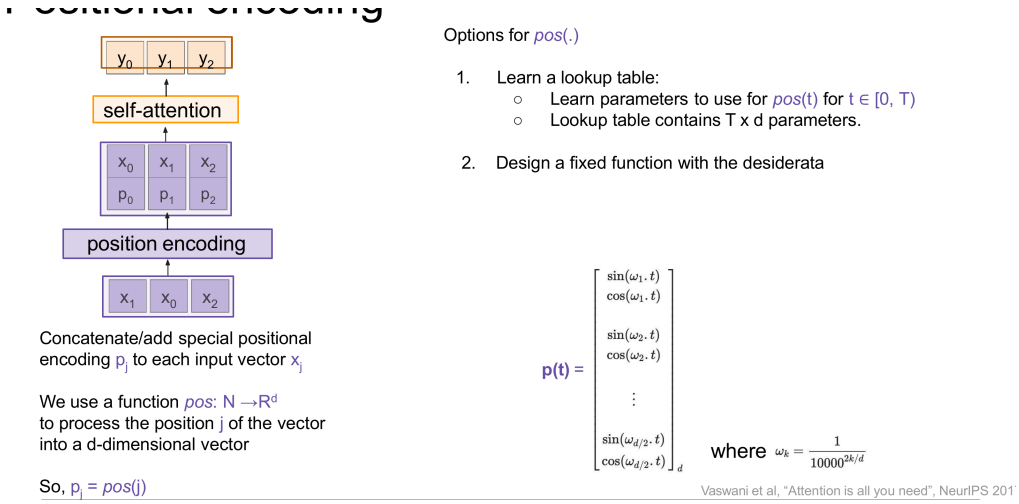


Figure 4.7: Sinusoidal Positional Encoding (Vaswani et al. [54], from course slides). Each position t is mapped to a combination of sinusoidal waves across frequencies, allowing the model to attend by relative distances.

where pos is the token position and $i \in \{0, \ldots, D/2-1\}$ indexes the channel dimension.

Key Word

The Linear Relative Transformation Property

Because $\sin(\alpha+\beta) = \sin\alpha\cos\beta + \cos\alpha\sin\beta$ and $\cos(\alpha+\beta) = \cos\alpha\cos\beta - \sin\alpha\sin\beta$, for any fixed offset k, the positional encoding at position $pos+k$ can be represented as a linear transformation of the encoding at position pos:

$$PE_{(pos+k)} = M_k \cdot PE_{(pos)}$$

where $M_k \in \mathbb{R}^{D\times D}$ is a deterministic rotation matrix. This allows self-attention to easily learn to attend based on **relative positional offsets**.

4.4.3 Masked Self-Attention (Causal Attention)

In autoregressive generation tasks (such as language modeling or autoregressive translation decoding), token y_t must be generated using only past context $y_{<t}$ and must never access future tokens $y_{t+1}, \ldots, y_T$.

In an RNN, causality is naturally enforced by sequential step-by-step execution. To preserve causality in the Transformer **without sacrificing parallel training**, we apply a **Causal Mask** to the alignment scores, as illustrated in Figure 4.8.

Before evaluating the Softmax function, an upper-triangular mask matrix M is added to the scaled dot products:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \le i \quad \text{(past and current tokens)} \\ -\infty & \text{if } j > i \quad \text{(future tokens)} \end{cases} \tag{4.19}$$

$$\text{MaskedAttention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{D_k}} + M\right)V \tag{4.20}$$

Because $\exp(-\infty) = 0$, the attention weights assigned to future positions evaluate to exactly zero. Conse-

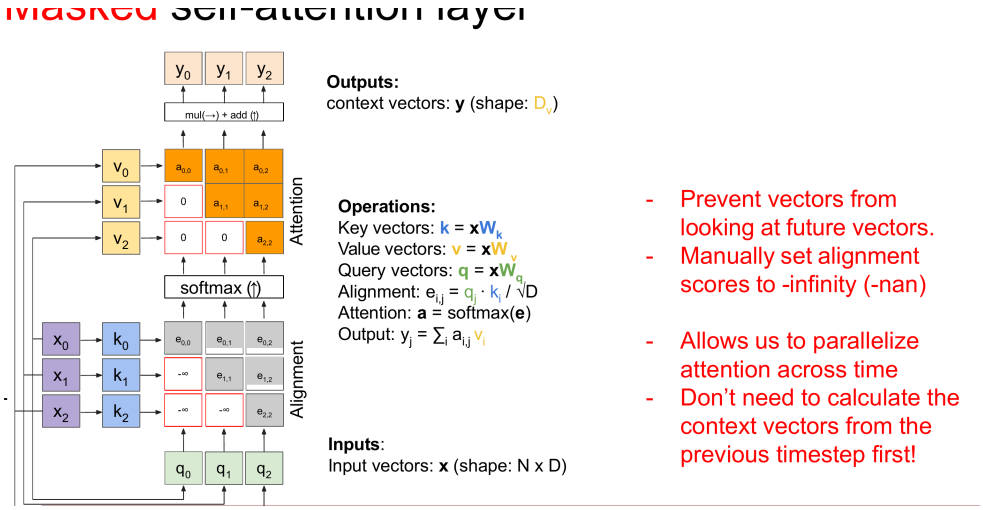


Figure 4.8: Masked Self-Attention with causal masking (from course slides). Setting future alignment entries to $-\infty$ forces their Softmax probabilities to zero, preventing information leakage while enabling full training parallelization across time.

quently, during training, the network can evaluate all T output steps in a single parallel matrix multiplication without unrolling a recurrent loop.

4.4.4 Multi-Head Self-Attention (MHSA)

In single-head attention, computing a single set of attention weights forces the network to average over all relational signals. To provide representational richness, Vaswani et al. [54] introduced **Multi-Head Self-Attention (MHSA)**, shown in Figure 4.9.

Key Insight

Analogy to Convolutional Filters in CNNs (Slide 59):
In a Convolutional Neural Network, a layer does not learn a single 2D filter; it learns an entire bank of 64 or 128 distinct kernels to simultaneously detect edges, textures, colors, and shapes.
Multi-Head Attention applies the exact same philosophy to attention: by learning H distinct sets of (W_i^Q, W_i^K, W_i^V) projections, individual heads specialize in distinct relationship patterns (e.g., Head 1 tracks immediate syntactic dependencies, Head 2 tracks pronoun co-reference, and Head 3 tracks long-range semantic topics).

For H attention heads, the input dimension D is partitioned into head dimension $D_k = D/H$:

$$\text{head}_i = \text{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right), \quad i \in \{1,\ldots,H\} \tag{4.21}$$

where $W_i^Q \in \mathbb{R}^{D\times D_k}$, $W_i^K \in \mathbb{R}^{D\times D_k}$, and $W_i^V \in \mathbb{R}^{D\times D_v}$. The individual outputs are concatenated and linearly projected via output matrix $W_O \in \mathbb{R}^{HD_v\times D}$:

$$\text{MultiHead}(Q,K,V) = \text{Concat}\left(\text{head}_1,\ldots,\text{head}_H\right)W_O \tag{4.22}$$

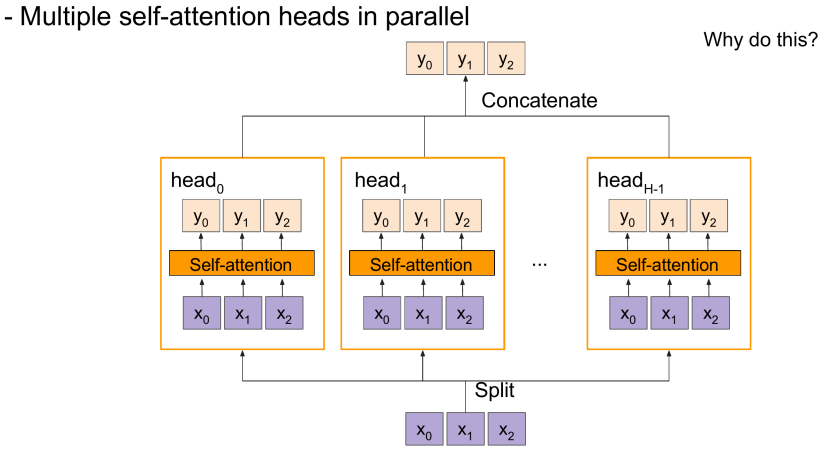


Figure 4.9: Multi-Head Self-Attention layer (from course slides). Input queries, keys, and values are split across H parallel heads, evaluated independently in separate subspaces, concatenated, and projected through output matrix W_O.

Because each head operates on vectors of reduced dimension D/H, the total computational complexity of MHSA is identical to that of single-head attention on the full dimension D.

4.5 The Transformer Architecture

4.5.1 The Transformer Encoder Block

The complete Transformer Encoder is constructed by stacking N identical modular blocks ($N = 6$ in the original architecture), detailed in Figure 4.10.

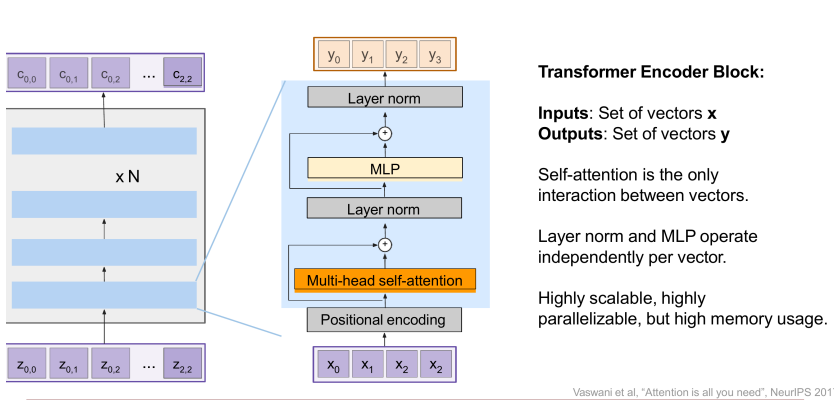


Figure 4.10: The Transformer Encoder Block (Vaswani et al. [54], from course slides). Composed of Multi-Head Self-Attention followed by a position-wise MLP, each wrapped with residual connections and Layer Normalization.

Each encoder block contains two primary sub-layers:

1. **Sub-layer 1 (Multi-Head Self-Attention):** Computes all-to-all token interactions. Crucially, self-attention

is the **only** component in the encoder where information is exchanged across different tokens.

2. **Sub-layer 2 (Position-wise Feed-Forward Network):** An MLP applied identically and independently to each token vector:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{4.23}$$

Typically, the inner dimension expands to $4D$ before projecting back to D (e.g., $D = 512 \rightarrow 2048 \rightarrow 512$). Every sub-layer is wrapped by a **Residual Connection** followed by **Layer Normalization** (Chapter 1):

$$x_{\text{out}} = \text{LayerNorm}(x + \text{SubLayer}(x)) \tag{4.24}$$

Residual connections prevent gradient dissipation across deep stacks, while Layer Normalization stabilizes the activation distributions of individual tokens independently of batch size.

4.5.2 The Transformer Decoder Block

The Transformer Decoder stacks N decoder blocks to generate output sequences autoregressively, detailed in Figure 4.11.

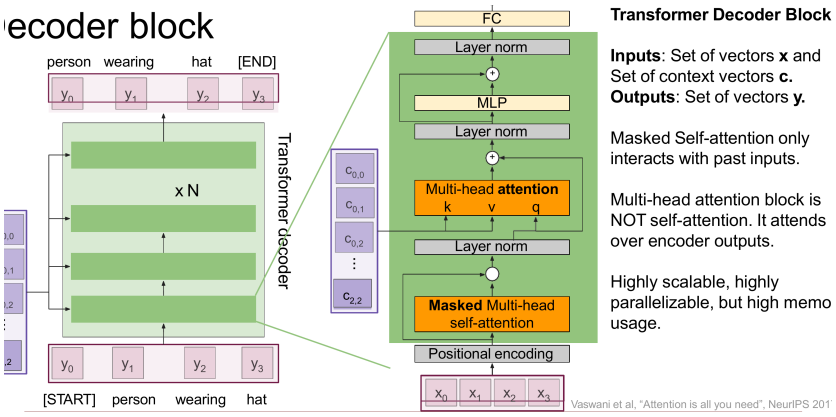


Figure 4.11: The Transformer Decoder Block (Vaswani et al. [54], from course slides). Contains Masked Multi-Head Self-Attention (causal), Cross-Attention over encoder outputs, and a position-wise MLP.

Each decoder block contains three sub-layers:

1. **Masked Multi-Head Self-Attention:** Computes interactions exclusively among previously generated target tokens using the causal mask ($M_{i,j} = -\infty$ for $j > i$).
2. **Multi-Head Cross-Attention:** Fuses encoder and decoder representations. Unlike self-attention, the **Queries** (Q) originate from the preceding decoder layer, while the **Keys** (K) and **Values** (V) are drawn from the final output of the Encoder stack c.
3. **Position-wise Feed-Forward Network (MLP):** Projects each token vector independently, followed by a final linear classifier and Softmax over the target vocabulary.

4.5.3 Comparative Summary: RNNs versus Transformers

Course Slide 71 provides a side-by-side synthesis of the two paradigms, summarized in Figure 4.12 and Table 4.2.

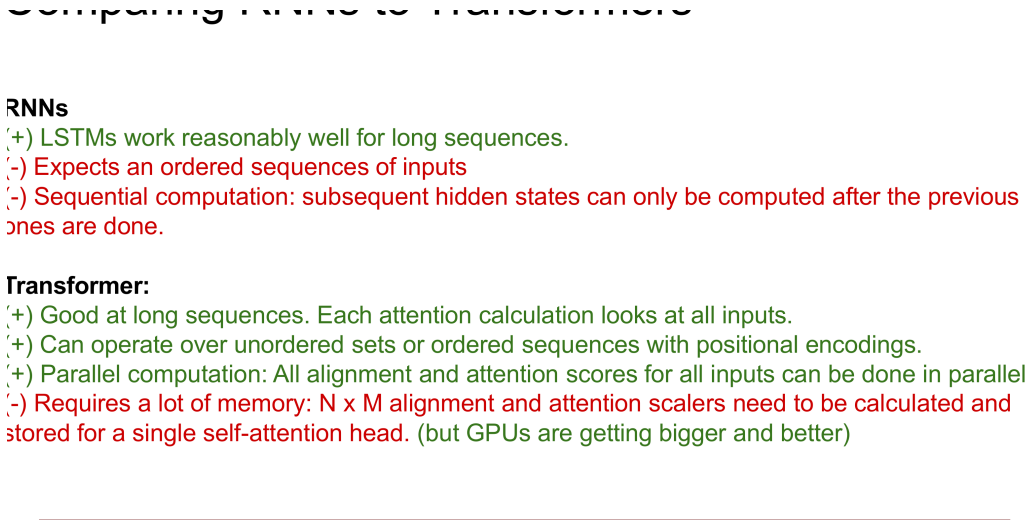


Figure 4.12: Comparison of RNNs and Transformers (Slide 71). Highlighting parallelization benefits, sequence length handling, and memory trade-offs.

Property	**Recurrent Networks (RNN / LSTM)**	**Transformer Networks**
Temporal Processing	Strictly sequential (h_t requires h_{t-1}).	Fully parallel across all time steps.
Long-Range Context	Suffers from exponential signal decay.	Direct $O(1)$ constant path length between any pair.
Input Structure	Inherently expects ordered temporal streams.	Operates on unordered sets; order is injected via PE.
Hardware Utilization	Poor GPU scaling due to sequential loops.	Massive parallel scaling on modern matrix accelerators.
Memory Footprint	Linear memory complexity $O(n)$.	Quadratic memory scaling $O(n^2)$ for attention maps.

Table 4.2: Comprehensive architectural trade-off comparison between RNNs and Transformers.

4.6 Transformers in Computer Vision

4.6.1 Visual Self-Similarity and Non-Local Filtering

Before Transformers dominated Computer Vision, the core mathematical intuition behind self-attention was widely studied under the concept of **visual self-similarity**:

- **Self-Similarity in Textures and Natural Images:** As demonstrated by Efros and Leung [20], natural textures and scenes exhibit pervasive recurring motifs across distant pixel coordinates.
- **Non-Local Means (Buades et al. [5]):** In classical image denoising, the Non-Local Means filter updates pixel i by computing a weighted average over all pixels j in the image:

$$u(i) = \sum_j w(i,j)v(j), \quad \text{where } w(i,j) \propto \exp\left(-\frac{\|v(N_i) - v(N_j)\|^2}{h^2}\right) \tag{4.25}$$

  Mathematically, Non-Local Means is identical to self-attention without learnable projections: queries and keys are local image patches, and values are pixel intensities!

4.6.2 The Evolution of Vision-to-Language Pipelines

The integration of attention and Transformers progressively transformed vision-to-language models across four distinct generations (Slides 84–86):

1. **CNN + Vanilla RNN (Chapter 3):** A convolutional backbone extracts a single global vector v, which initializes a recurrent decoder.
2. **CNN + Attentive RNN (Xu et al. [59]):** The recurrent decoder dynamically computes 2D spatial attention maps over convolutional feature grids.
3. **CNN + Transformer (Image Transformer):** The recurrent language model is replaced with a Transformer decoder, while the visual backbone remains a CNN.
4. **Pure Transformers (From Pixels to Language):** Convolutions and recurrences are completely removed. The visual image and output text are processed entirely within a unified Transformer architecture.

4.6.3 Vision Transformers (ViT)

Dosovitskiy et al. [19] proved that pure Transformer architectures could match or surpass Convolutional Networks on large-scale computer vision benchmarks with the introduction of the **Vision Transformer (ViT)**, illustrated in Figure 4.13.

The ViT formulation treats image patches as words in a sentence:

1. **Patch Extraction and Linear Embedding:** An image $x \in \mathbb{R}^{H\times W\times C}$ is partitioned into a sequence of $N = \frac{HW}{P^2}$ non-overlapping square patches of size $P \times P$ (typically $P = 16$). Each flattened patch $x_p^i \in \mathbb{R}^{P^2C}$ is linearly projected to dimension D via a trainable embedding matrix E:

$$z_0 = \left[x_{\text{class}}; x_p^1 E; x_p^2 E; \ldots; x_p^N E\right] + E_{\text{pos}} \tag{4.26}$$

   where $E_{\text{pos}} \in \mathbb{R}^{(N+1)\times D}$ is a learnable 1D positional embedding tensor.
2. **The Classification Token (`[CLS]`):** Analogous to BERT, a learnable token embedding x_{class} is prepended to the patch sequence. Because self-attention allows all tokens to interact, the final state of the `[CLS]` token aggregates global visual context and serves as the input to the classification head.

Vision Transformers vs. ResNets

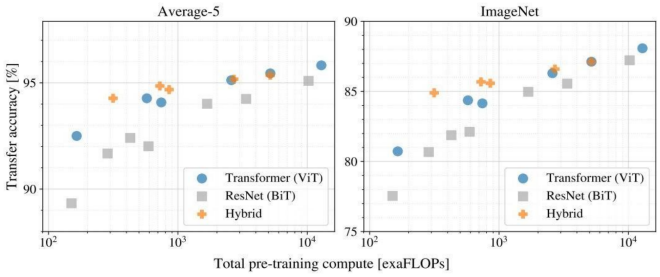


Figure 5: Performance versus cost for different architectures: Vision Transformers, ResNets, and hybrids. Vision Transformers generally outperform ResNets with the same computational budget. Hybrids improve upon pure Transformers for smaller model sizes, but the gap vanishes for larger models.

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ArXiv 2020
Colab link to an implementation of vision transformers

Figure 4.13: Vision Transformers (ViT) versus ResNets (Dosovitskiy et al. [19], from course slides). An image is segmented into 16×16 non-overlapping patches, linearly projected into embeddings, prepended with a `[CLS]` token, and processed by a standard Transformer encoder.

3. **Transformer Encoder Stack:** The sequence of $N+1$ token vectors is processed through standard Transformer encoder blocks with Layer Normalization and Multi-Head Self-Attention.

Key Word

Inductive Bias: ViT versus ResNet

A central pedagogical takeaway from course Slide 87 is the contrasting nature of their **inductive biases**:

- **Convnets (ResNets):** Possess strong, hard-coded visual inductive biases: **two-dimensional locality** (adjacent pixels interact first) and **translation equivariance** (the same convolution filter operates across all spatial positions).
- **Vision Transformers (ViT):** Have virtually **zero visual inductive bias**. The model does not inherently know that patch 1 is adjacent to patch 2, nor does it know that patches form a 2D grid; it must learn all geometric relationships from scratch via positional embeddings and attention weights.

The Scaling Regime: Because ViT has little inductive bias, when trained on small or medium datasets (e.g., ImageNet-1K with 1.3M images), it tends to overfit and performs worse than ResNet. However, when pre-trained on massive datasets (such as Google's JFT-300M with 300 million images), ViT's superior capacity and global receptive field from layer 1 decisively outperform ResNets!

Modern hierarchical adaptations have since generalized ViTs to multi-scale vision tasks:

- **DETR (Carion et al. [7]):** Treats object detection as a direct set-prediction problem using Transformer encoder-decoders and bipartite Hungarian matching, eliminating anchor boxes and Non-Maximum Suppression (NMS).
- **Swin Transformer (Liu et al. [40]):** Computes self-attention within local shifted windows, reducing computational complexity from quadratic $O((HW)^2)$ to linear $O(HW)$ while preserving multi-scale feature

hierarchies suitable for dense segmentation.

- **Multiscale Vision Transformers (MViT) (Fan et al. [21]):** Progressively reduces spatial resolution while expanding channel dimensions across stages, optimizing compute for high-resolution video and image recognition.

4.7 Pre-training, BERT, and Model Compression

4.7.1 The "ImageNet Moment" for Natural Language Processing

In Computer Vision, transfer learning using ImageNet pre-training had been the gold standard since AlexNet (Chapter 2). For years, Natural Language Processing lacked an equivalent universal pre-training paradigm, relying instead on shallow static embeddings (Word2Vec, GloVe).

In 2018, the Transformer enabled the true **"ImageNet Moment for NLP"** (Slide 62):

1. **Pre-training Phase:** Train a massive Transformer model on hundreds of gigabytes of unannotated text scraped from the internet using self-supervised objectives.
2. **Fine-Tuning Phase:** Adapt the pre-trained weights to specific downstream tasks (classification, question answering, named entity recognition) with a small task-specific head, requiring minimal labeled data.

4.7.2 BERT: Bidirectional Encoder Representations from Transformers

Prior pre-trained models (such as OpenAI GPT) relied on standard autoregressive language modeling: predicting token x_t given left-to-right context $x_{<t}$. However, language understanding is intrinsically **bidirectional**: words derive their meaning from both preceding and succeeding context.

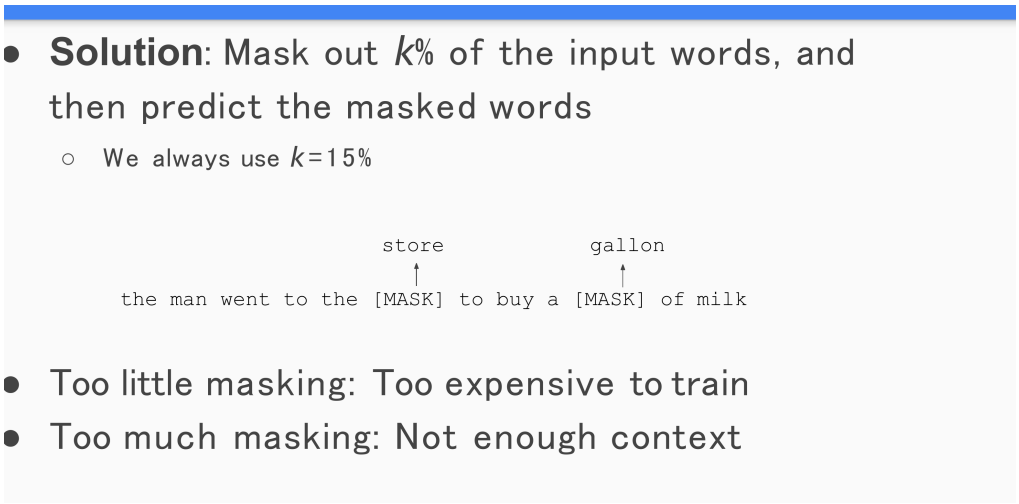


Figure 4.14: BERT Masked Language Model (MLM) pre-training (Devlin et al. [15], from course slides). 15% of input tokens are masked, and the model predicts the original words using full bidirectional context.

Why could standard language models not be bidirectional? In a multi-layer bidirectional Transformer, tokens in upper layers would trivially "see themselves" through intermediate connections, reducing the next-word prediction task to trivial copying.

To solve this dilemma, Devlin et al. [15] introduced **BERT** and the **Masked Language Model (MLM)** objective, illustrated in Figure 4.14:

1. **15% Masking Rule:** Exactly 15% of the input tokens are chosen at random. Of these chosen tokens:
   - 80% are replaced with the special mask token `[MASK]`.
   - 10% are replaced with a random word.
   - 10% are kept unchanged.
2. **Bidirectional Context Processing:** The corrupted sequence is fed through a deep Transformer encoder, where every token attends freely to all other tokens in both directions.
3. **Selective Cross-Entropy Loss:** A classification head over the vocabulary predicts the original identity of the masked tokens. The training loss considers **only** the predictions at the masked positions:

$$L_{\text{MLM}} = -\sum_{i \in \text{masked}} \log P(x_i | \widetilde{X}) \tag{4.27}$$

Note

Why Exactly 15% Masking?

Course Slide 92 emphasizes this fundamental optimization trade-off:

- *Too little masking ($< 15\%$):* Too few gradient signals are generated per sentence, making pre-training excessively slow and computationally prohibitive.
- *Too much masking ($> 15\%$):* Insufficient contextual clues remain in the corrupted sentence, preventing the network from learning meaningful syntactic and semantic dependencies.

Alongside MLM, BERT was trained on a secondary objective: **Next Sentence Prediction (NSP)**, where the model predicts whether two paired input sentences appeared consecutively in the original text or were randomly paired.

4.7.3 ELECTRA: Efficient Pre-training as Discriminators

While BERT achieved state-of-the-art results, computing loss on only 15% of tokens represents substantial sample inefficiency. Clark et al. [13] resolved this limitation with **ELECTRA**, training text encoders as **discriminators** rather than generators.

Key Word

Replaced Token Detection (RTD)

ELECTRA consists of two networks trained jointly:

1. **Generator (Small MLM):** Predicts masked tokens in the standard 15% corrupted sequence, producing plausible word substitutions.
2. **Discriminator (Target Transformer Encoder):** Instead of predicting word identities over a 30,000-word vocabulary, the discriminator solves a binary classification task for **every single token**: predicting whether each token is **original** or has been **replaced** by the generator.

Key Advantage: Because binary classification is evaluated over the entire sequence, **100% of the tokens contribute to the loss gradient** at every step (rather than just 15% in BERT). Consequently, ELECTRA achieves comparable or superior downstream accuracy with less than 25% of BERT's training compute.

4.7.4 Knowledge Distillation and Model Compression

While massive Transformer models achieve outstanding accuracy, their billions of parameters introduce prohibitive latency and memory footprints, hindering deployment in real-time edge devices and low-latency production microservices.

Course Slide 99 highlights the primary technique for solving this operational bottleneck: **Knowledge Distillation** (pioneered by Bucilă et al. [6], Hinton et al. [35], and extended to adversarial compression by Belagiannis, Farshad, and Galasso [4]).

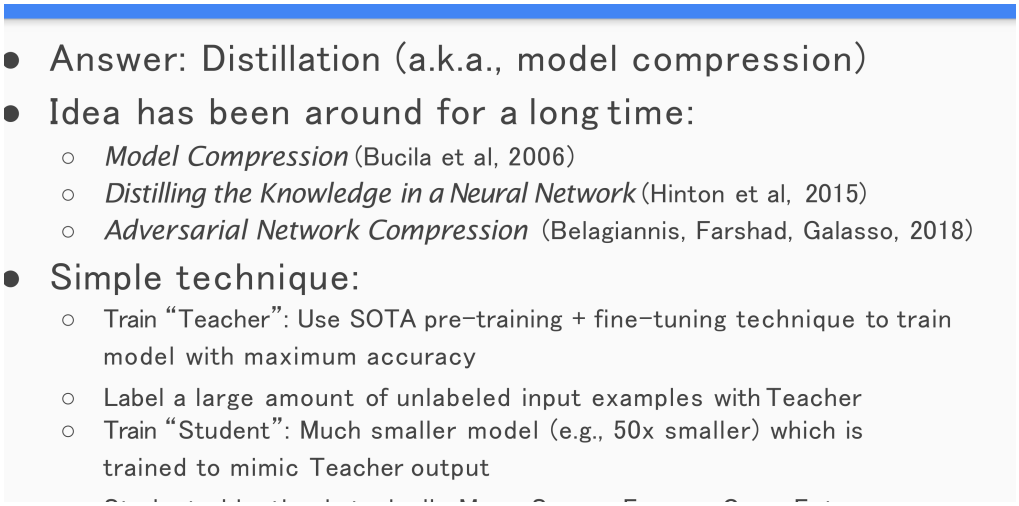


Figure 4.15: Knowledge Distillation for Transformer model compression (Slide 99). A high-capacity Teacher network generates soft probability distributions over unlabeled data, supervising a compact Student model.

The distillation protocol proceeds through three distinct phases:

1. **Train the Teacher Network:** A large, high-capacity Transformer (e.g., BERT-Large) is trained using standard pre-training and task fine-tuning to achieve maximum accuracy.
2. **Soft Target Generation:** The frozen Teacher evaluates a massive corpus of unlabeled data, generating soft probability distributions over classes:
$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{4.28}$$
where $T > 1$ is the distillation **temperature**, which softens output distributions to reveal the *"dark knowledge"* (relative probabilities assigned to incorrect classes).
3. **Train the Student Network:** A much smaller Student model (e.g., 50x smaller, such as DistilBERT or compact BERT [52]) is trained to mimic the Teacher's soft outputs by minimizing Kullback-Leibler (KL) divergence or Cross-Entropy:
$$L_{\text{distill}} = \alpha L_{\text{hard}}(y_{\text{student}}, y_{\text{true}}) + (1-\alpha) T^2 \,\text{KL}\,(q_{\text{student}} \,\|\, q_{\text{teacher}}) \tag{4.29}$$

Empirically (Slide 100), distilling a Teacher into a smaller model yields substantially higher accuracy than training the smaller model from scratch directly on labeled data. Knowledge distillation allows production systems to retain over 97% of BERT's performance while operating at a fraction of the inference latency.

4.8 Chapter Summary and Conceptual Graph

The concepts developed in this chapter establish the architectural foundations of modern attention and Transformer networks, tracing the path from recurrent bottlenecks to pure attention and self-supervised foundation models:

Key Insight

The Logical Connections Between Nodes:

- **From Node 1 to 2:** The fixed-dimensional bottleneck vector $c = h_T$ of Vanilla Seq2Seq degrades on long sequences; Bahdanau attention resolves this by computing dynamic, content-based alignments $c_t = \sum_i \alpha_{t,i} h_i$.
- **From Node 2 to 3:** Recognizing that attention operates over unordered sets allows extending attention to spatial 2D convolutional grids, enabling models to dynamically focus on image regions during caption generation.
- **From Node 3 to 4:** Eliminating recurrent transitions entirely, Self-Attention allows tokens within the same sequence to interact directly via Queries, Keys, and Values with a constant $O(1)$ maximum path length.
- **From Node 4 to 5:** Because self-attention is permutation equivariant, Positional Encodings (sinusoidal or learned) are added to inject temporal ordering, while Causal Masking ($-\infty$) enforces autoregressive validity in parallel.
- **From Node 5 to 6:** Wrapping Multi-Head Self-Attention with residual connections, Layer Normalization, and point-wise MLPs creates the complete Transformer Encoder and Decoder blocks.
- **From Node 6 to 7:** Partitioning images into 16×16 patches translates 2D vision into token sequences, enabling Vision Transformers (ViT) to outperform ResNets at scale despite minimal hard-coded inductive biases.
- **From Node 7 to 8:** Transformers unlock massive self-supervised pre-training via Masked Language Modeling (BERT) and discriminative Replaced Token Detection (ELECTRA), with Knowledge Distillation enabling real-time production deployment.

Figure 4.16: Conceptual roadmap of Attention Mechanisms, Transformer Networks, ViT, and Pre-training.

5. Self-Supervised Learning

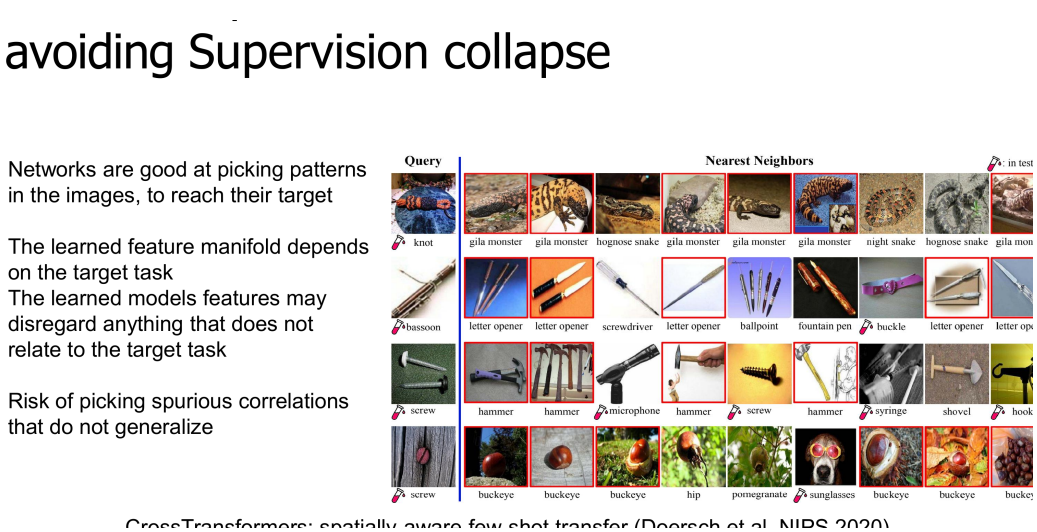
5.1 Introduction: Transcending the Supervised Bottleneck

Throughout Chapters 1 to 4, our models operated predominantly under the **supervised learning paradigm**. In supervised deep learning, deep architectures require massive volumes of human-annotated data (such as ImageNet with 1.28 million images across 1000 categories) to discover rich representations. However, full supervision suffers from two critical, systemic limitations:

1. **Annotation Cost and Unscalability:** Manual labeling is prohibitively expensive, time-consuming, error-prone, and ethically constrained. While human labelers cannot feasibly annotate billions of images or videos, modern cameras and internet repositories generate billions of raw, unlabeled visual samples daily.
2. **Supervision Collapse and Task-Specific Manifolds:** A deep network trained strictly with supervised cross-entropy optimizes solely for discriminative decision boundaries relevant to the specific target classes. Consequently, the network discards any visual information, texture, geometry, or spatial context that is uninformative for the target task. If the training dataset contains spurious correlations (e.g., cows always appearing on green grass, boats always on open water), the supervised model relies on background context rather than genuine object semantics.

To resolve this fundamental tension, **Self-Supervised Learning (SSL)** re-frames representation learning: rather than relying on human annotations, the training signal is derived directly from the intrinsic structure of the unlabeled data itself.

This chapter formalizes the principles, mathematical formulations, and evolutionary milestones of self-



CrossTransformers: spatially-aware few-shot transfer (Doersch et al. NIPS 2020)

Figure 5.1: Supervision collapse in deep neural representations (from course slides [24]). When trained solely on class labels, the learned manifold is constricted to features that discriminate between target classes, ignoring broader semantic structures and risking the exploitation of spurious shortcuts.

supervised visual representation learning, following the curriculum designed by Prof. Fabio Galasso [24].

—

5.2 Foundations of Self-Supervised Learning

5.2.1 The Pretext-Downstream Paradigm

The canonical self-supervised learning pipeline operates across two decoupled stages, as illustrated in Figure 5.2:

1. **Pretext Task (Self-Supervised Pre-training):** The model is trained on a massive unlabeled dataset $\mathscr{D}_{\text{unlabeled}} = \{x_1, x_2, \ldots, x_N\}$. An artificial supervisory objective (a *pretext task*) is defined automatically from the data without human intervention. The primary goal is not the pretext task itself, but rather forcing the feature encoder $f_\theta(\cdot)$ to extract rich, generalizable visual representations.
2. **Downstream Task (Transfer Learning):** The pre-trained encoder $f_\theta(\cdot)$ is transferred to target tasks of interest (such as ImageNet classification, object detection, or instance segmentation) using a smaller labeled dataset $\mathscr{D}_{\text{downstream}} = \{(x_i, y_i)\}_{i=1}^{M}$.

5.2.2 Evaluation Protocols for Learned Representations

Evaluating the quality of an unsupervised representation requires standardized empirical protocols:

1. **Pretext Task Performance:** Measuring validation accuracy or loss directly on the pretext task. While useful for monitoring convergence, pretext performance often correlates poorly with downstream utility: networks can invent clever low-level *shortcuts* (such as exploiting lens chromatic aberration or edge continuity) that achieve near-perfect pretext accuracy while learning degenerate semantic representations.
2. **Linear Evaluation Protocol (Linear Probing):** The gold standard for representation quality. The pre-trained feature extractor f_θ is completely frozen. A single linear classifier (a linear projection followed by

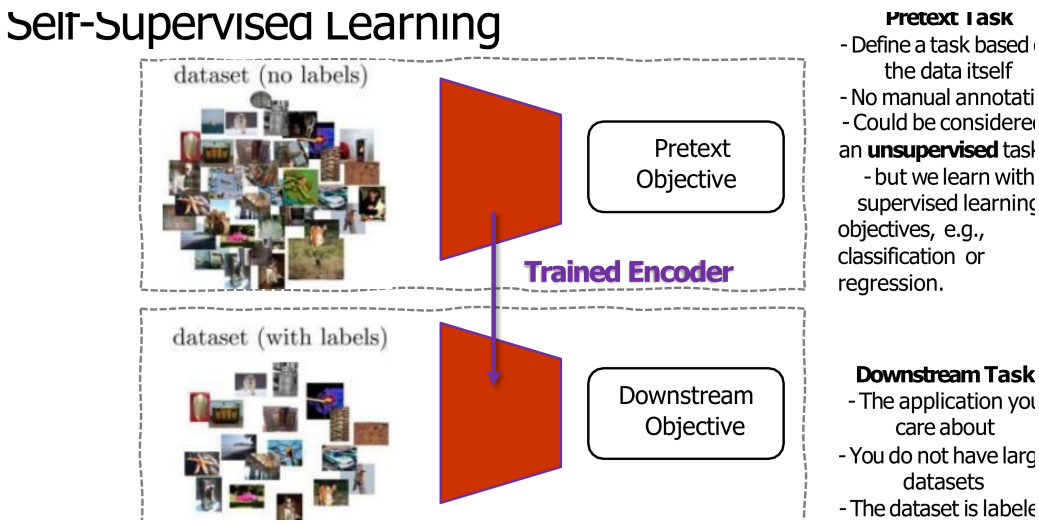


Figure 5.2: The two-stage Self-Supervised Learning framework (from course slides [24]). Left: The feature encoder f_θ is trained on an unlabeled pretext objective. Right: The learned representation is frozen or fine-tuned on target downstream tasks.

Softmax) is trained on top of the frozen representations $h = f_\theta(x)$ on a target benchmark (e.g., ImageNet-1K):

$$\hat{y} = \text{Softmax}(W^\top f_\theta(x) + b) \tag{5.1}$$

Because the backbone weights θ cannot adapt, high linear probing accuracy proves that the pre-trained features are **linearly separable** and encode rich semantic concepts natively.

3. **Fine-Tuning Protocol:** The entire network (backbone f_θ plus a newly initialized task head) is fine-tuned end-to-end on the downstream dataset, typically using a smaller learning rate for the backbone layers. This measures the efficacy of the representation as a weight initialization strategy.
4. **Semi-Supervised Benchmark:** The pre-trained model is fine-tuned on extreme low-data regimes, such as 1% or 10% of labeled ImageNet examples. Superior representations yield competitive performance even when labels are drastically scarce.

—

5.3 Heuristic Pretext Tasks

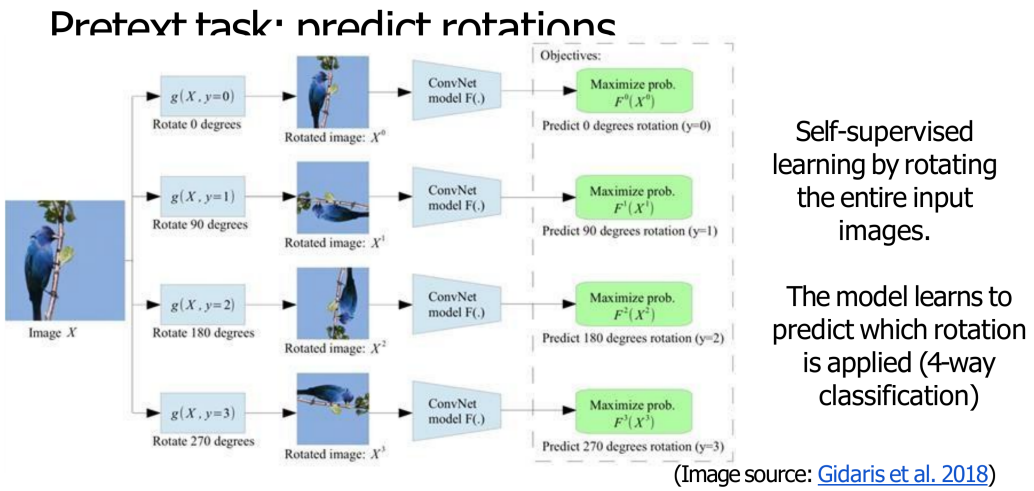
Early self-supervised methods relied on clever, domain-specific heuristic transformations where the input image is perturbed and the network is tasked with predicting the perturbation parameters.

5.3.1 Predicting Geometric Rotations

Gidaris et al. [28] proposed training a convolutional network to predict discrete geometric rotations applied to an image. Given an unrotated, upright image x, a transformation operator $R_y(x)$ rotates the image by

$y \in \{0°, 90°, 180°, 270°\}$ (represented as a 4-class categorical label $y \in \{0, 1, 2, 3\}$), as depicted in Figure 5.3:

$$\mathscr{L}_{\text{rot}} = -\sum_{k=0}^{3} \mathbb{I}_{[y=k]} \log \left(\frac{\exp(w_k^\top f_\theta(R_y(x)))}{\sum_{j=0}^{3} \exp(w_j^\top f_\theta(R_y(x)))} \right) \tag{5.2}$$



(Image source: Gidaris et al. 2018)

Figure 5.3: Rotation prediction pretext task (from course slides [24, 28]). The network receives an image rotated by 0°, 90°, 180°, or 270° and must classify the applied angle.

Key Insight

Why Rotation Prediction Induces Semantic Understanding:
A model cannot recognize whether a dog or a building is upside-down (180°) by inspecting isolated, low-level pixel statistics alone. To determine canonical orientation, the network must identify high-level visual entities, localize object parts (e.g., eyes, wheels, sky, ground), and reason about physical gravity.

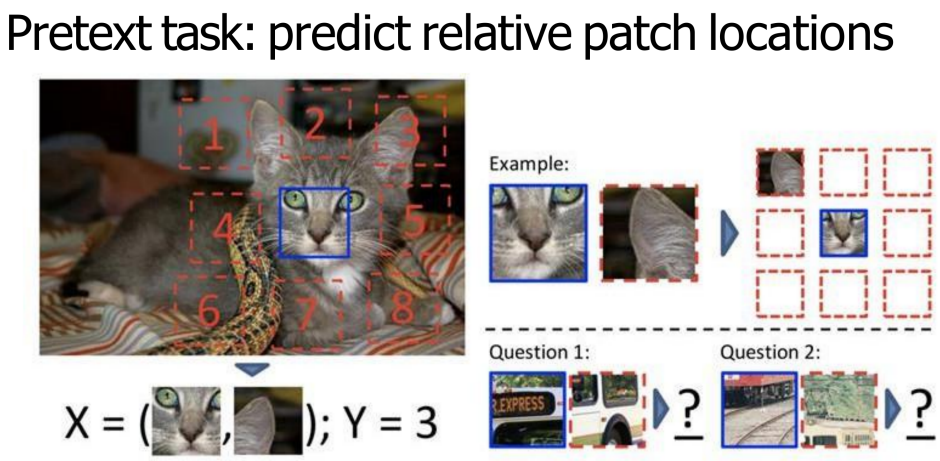
5.3.2 Spatial Context: Relative Patch Location and Jigsaw Puzzles

Relative Patch Localization

Doersch et al. [17] formulated an unsupervised spatial context task. As shown in Figure 5.4, two random patches are extracted from a 3×3 grid: a central patch and one of eight peripheral neighbors. The network must predict the relative spatial position $y \in \{1, \ldots, 8\}$.

To prevent the model from exploiting trivial shortcuts, two key design safeguards are mandatory:

- **Random Gap and Jitter:** If patches were strictly adjacent, the network could trivially align continuous edge contours across patch boundaries. Random spatial gaps between patches eliminate edge continuity shortcuts.
- **Color Jittering and Aberration Removal:** Optical camera lenses exhibit *chromatic aberration*—a radial distortion where color channels separate slightly toward image borders. Networks quickly learn to evaluate the radial color shift to deduce absolute patch position! Converting patches to grayscale or randomly dropping color channels eliminates this hardware shortcut.

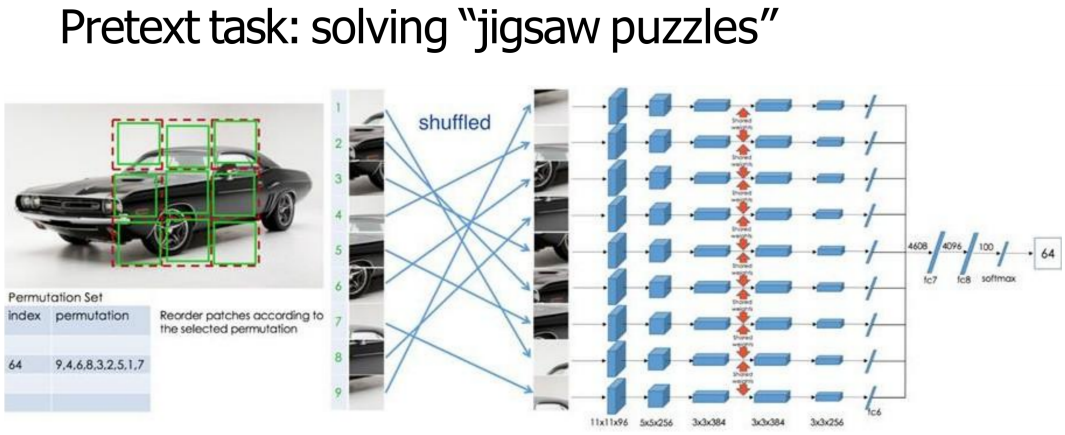


(Image source: Doersch et al., 2015)

Figure 5.4: Relative patch location pretext task (from course slides [24, 17]). The network takes two patches (one central, one random neighbor) and predicts which of the eight spatial positions the neighbor occupies.

Solving Jigsaw Puzzles

Noroozi and Favaro [42] extended spatial context prediction to an entire 3×3 grid of 9 patches, shown in Figure 5.5. Rather than predicting single pair relations, the 9 patches are permuted according to a pre-defined permutation set $\mathscr{P} \subset S_9$ (typically $|\mathscr{P}| = 100$ or 1000 permutations chosen with maximum Hamming distance out of $9! = 362{,}880$ possibilities).



(Image source: Noroozi & Favaro, 2016)

Figure 5.5: Solving Jigsaw Puzzles with Context-Free Networks (CFN) (from course slides [24, 42]). 9 randomly shuffled patches pass through weight-shared convolutional towers before a fully connected classifier predicts the permutation index.

The architecture, termed a **Context-Free Network (CFN)**, processes each patch independently through weight-shared Siamese convolutional backbones before concatenating the features into an MLP classifier. This ensures the convolutional kernels extract modular, position-agnostic part representations.

5.3.3 Context Encoders: Feature Learning by Inpainting

Pathak et al. [46] introduced **Context Encoders**, where an encoder-decoder network reconstructs a missing central image region (e.g., a 64×64 masked region within a 128×128 image), as illustrated in Figure 5.6.

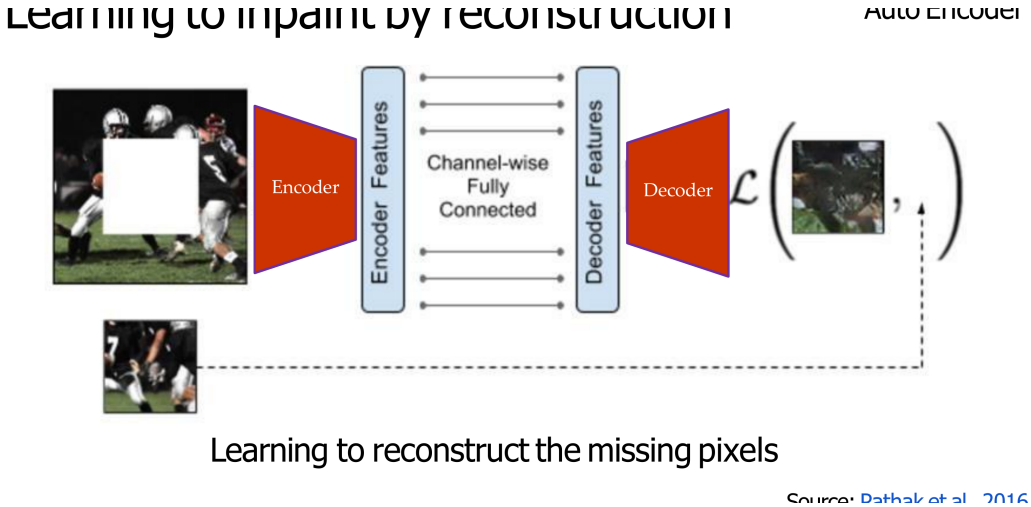


Figure 5.6: Context Encoders for image inpainting (from course slides [24, 46]). An encoder compresses the unmasked context into a latent representation, and a decoder reconstructs the missing pixel patch via a joint reconstruction and adversarial loss.

Training with standard L_2 pixel reconstruction loss alone:

$$\mathscr{L}_{\text{recon}} = \|\hat{M} \odot (x - f_\theta((1-\hat{M}) \odot x))\|_2^2 \tag{5.3}$$

penalizes variance and forces the network to output the blurry expected average of all plausible completions. To generate sharp, semantically realistic structures, Pathak et al. combined the reconstruction loss with a Generative Adversarial Network (GAN) adversarial objective:

$$\mathscr{L}_{\text{total}} = \mathscr{L}_{\text{recon}} + \lambda_{\text{adv}} \mathscr{L}_{\text{adv}} \tag{5.4}$$

where an auxiliary discriminator D_ϕ distinguishes real unmasked image patches from synthesized completions.

5.3.4 Image and Video Colorization

Colorful Image Colorization and Split-Brain Autoencoders

Zhang et al. [61] formulated colorization as a pretext task: given the grayscale luminance channel $L \in \mathbb{R}^{H \times W \times 1}$ in the CIELAB color space, predict the associated color channels $ab \in \mathbb{R}^{H \times W \times 2}$. Because regression directly to continuous a, b values causes desaturated, brownish predictions, Zhang et al. quantized the ab space into 313 discrete bins and applied class rebalancing to overweight rare, vibrant colors.

Zhang et al. [62] subsequently generalized this into **Split-Brain Autoencoders**, shown in Figure 5.7. Rather than reconstructing identical inputs, the input channels are split into disjoint subsets X_1 (e.g., L) and X_2

(e.g., *ab*). Subnetwork f_1 predicts X_2 from X_1, while subnetwork f_2 predicts X_1 from X_2:

$$\mathscr{L}_{\text{split}} = \mathscr{L}(f_1(X_1), X_2) + \mathscr{L}(f_2(X_2), X_1) \tag{5.5}$$

By forcing cross-channel prediction, the network cannot learn identity shortcuts and must construct dense, cross-modal semantic feature spaces.

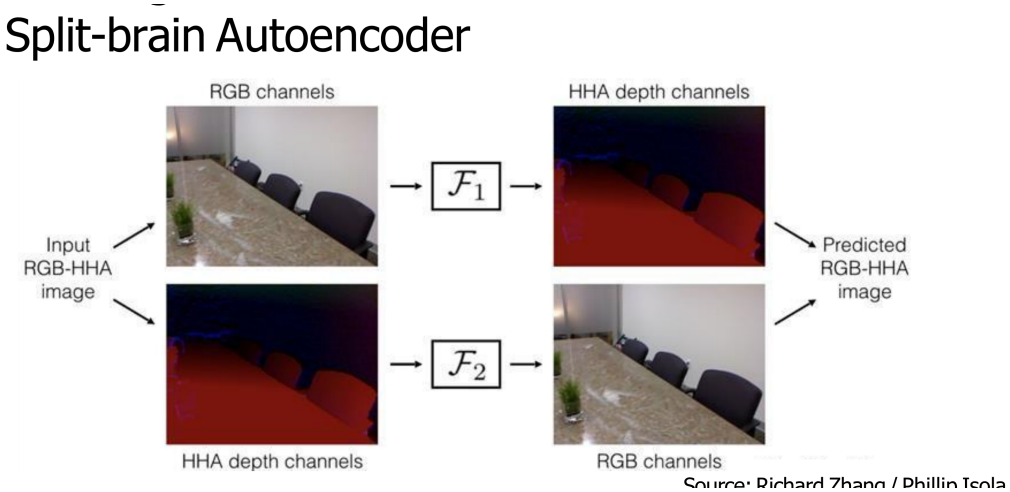


Figure 5.7: Split-Brain Autoencoder architecture (from course slides [24, 62]). Channels are partitioned into complementary subsets, forcing subnetworks to predict missing channels from observed channels without shortcutting.

Video Colorization and Emergence of Space-Time Tracking

Vondrick et al. [56] leveraged temporal coherence in unlabeled video sequences. In continuous video, objects maintain consistent color over time. Given a reference frame with known color and subsequent target frames in grayscale, the network must colorize the target frames by attending to the reference frame.

The model maps reference patches j to embeddings g_j and target patches i to embeddings f_i. A Softmax attention map A_{ij} evaluates spatial-temporal similarity:

$$A_{ij} = \frac{\exp(f_i^\top g_j)}{\sum_k \exp(f_i^\top g_k)} \tag{5.6}$$

The predicted target color is computed as a weighted combination of reference colors: $\hat{c}_i = \sum_j A_{ij} c_j$.

Key Insight

Emergence of Tracking Without Human Labels:
Because the only way to accurately colorize a moving object across video frames is to establish pixel-level correspondence, **visual object tracking emerges naturally as an unsupervised side-effect**. As shown in Figure 5.9, the learned attention matrix A_{ij} successfully propagates semantic segmentation masks and human pose keypoints across time without any annotated training examples.

—

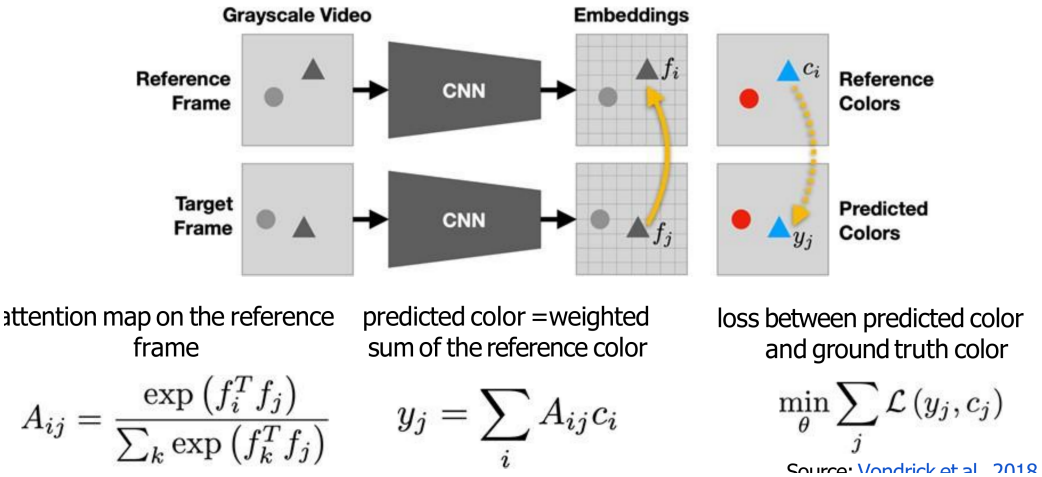


Figure 5.8: Video colorization via space-time correspondence attention (from course slides [24, 56]). The network calculates dense similarity between target grayscale patches and reference colored patches.

Figure 5.9: Emergent unsupervised tracking from video colorization (from course slides [24, 56]). Without human supervision, the attention weights learned to solve colorization automatically track semantic masks and human pose keypoints across frames.

5.4 Masked Autoencoders (MAE)

5.4.1 From Convolutional Inpainting to Vision Transformers

While early context encoders operated on convolutional networks with small masked regions (20%–30%), He et al. [32] introduced **Masked Autoencoders (MAE)**, adapting BERT-style masked modeling [15] to the Vision Transformer (ViT) architecture.

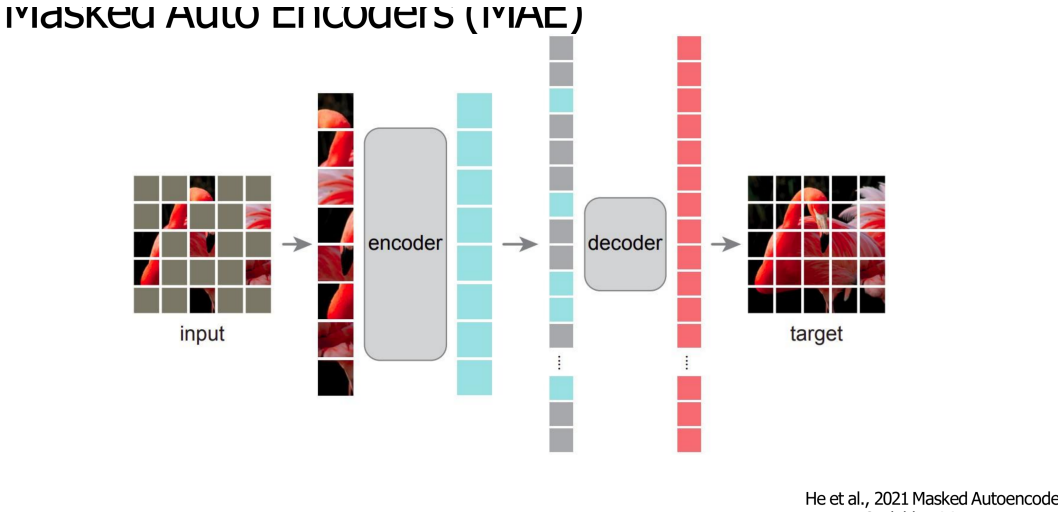


Figure 5.10: Masked Autoencoders (MAE) architecture (from course slides [24, 32]). Left: ViT Encoder processes exclusively the visible patches (25%). Right: A lightweight ViT Decoder merges encoded representations with shared learnable `[mask]` tokens to reconstruct pixel values.

MAE builds upon a critical insight distinguishing vision from natural language:

- In language, words are highly semantic, information-dense symbols; masking 15% of tokens creates an intellectually demanding completion task.
- In vision, images are continuous signals with **heavy spatial redundancy**. A patch can easily be interpolated from adjacent pixels without high-level semantic reasoning.
- **The Solution:** An **extreme masking ratio of** 75% **to** 80%. At 75% masking, neighboring information is stripped away, forcing the model to perform holistic holistic semantic scene understanding to synthesize plausible completions.

5.4.2 The Asymmetric Encoder-Decoder Architecture

The MAE architecture is uniquely asymmetric, consisting of two distinct stages:

1. **ViT Encoder (Visible Patches Only):** The original image is partitioned into non-overlapping patches (e.g., 16×16). A uniform random sample of 75% of the patches is completely removed. Crucially, the deep ViT encoder operates **only on the remaining** 25% **visible patches**. Because self-attention complexity scales quadratically with sequence length N ($\mathcal{O}(N^2)$), dropping 75% of tokens accelerates encoder training by more than an order of magnitude.
2. **ViT Decoder (Full Token Sequence):** The decoder is a lightweight Transformer stack. Its input is the full sequence of tokens: the transformed representations of visible patches merged with shared, learnable

`[mask]` tokens at the missing patch positions. Fixed sinusoidal positional embeddings are added to all tokens before decoding.

5.4.3 Reconstruction Objective and Normalization

The decoder terminates in a linear projection layer that outputs pixel values for each masked patch. The network optimizes the Mean Squared Error (MSE) computed **exclusively on the masked patches**:

$$\mathscr{L}_{\text{MAE}} = \frac{1}{|M|} \sum_{i \in M} \|x_i - \hat{x}_i\|_2^2 \tag{5.7}$$

where M denotes the set of masked patch indices.

Note

Per-Patch Normalized Reconstruction:
He et al. demonstrated that normalizing each patch's raw pixel values by its local mean and standard deviation:

$$x_i' = \frac{x_i - \mu_i}{\sigma_i + \varepsilon} \tag{5.8}$$

substantially boosts representation quality. Normalization forces the model to encode geometric shape, boundaries, and textures rather than trivial global illumination and contrast shifts.

5.4.4 Linear Probing vs. Fine-Tuning in Generative SSL

A defining empirical hallmark of MAE (and masked modeling in general) is the divergence between linear probing and fine-tuning:

- **Linear Probing:** MAE representations achieve modest linear probing accuracy ($\approx 65\%$–70% on ImageNet). Because generative pixel reconstruction optimizes for low-level visual fidelity, semantic features are distributed across multiple layers rather than linearly separated at the final layer.
- **End-to-End Fine-Tuning:** When fine-tuned end-to-end, MAE achieves top-tier results (87.8% on ImageNet-1K with ViT-Huge), demonstrating that the weights serve as an exceptional general-purpose visual prior.

—

5.5 Contrastive Representation Learning

5.5.1 Instance Discrimination and Hyperspherical Geometry

While heuristic pretext tasks depend on hand-crafted spatial tricks, **Contrastive Representation Learning** establishes a principled, task-agnostic paradigm: **Instance Discrimination**.

Every individual image is treated as its own discrete class. Given an image x, data augmentations generate correlated views x^+ (*positive pair*). All other distinct images in the dataset $\{x_1^-, x_2^-, \dots\}$ serve as *negative samples*. The objective is to map representations to a unit hypersphere $\mathbb{S}^{d-1}$ such that:

$$\text{sim}(f(x), f(x^+)) \to 1 \qquad \text{(Attract positive views)} \tag{5.9}$$

$$\text{sim}(f(x), f(x^-)) \to -1 \qquad \text{(Repel negative instances)} \tag{5.10}$$

5.5.2 The InfoNCE Loss Formulation

Van den Oord et al. [43] formalized contrastive learning via the **InfoNCE loss**. Given a query anchor representation q, a positive key k^+, and a dictionary of K negative keys $\{k_1^-, \ldots, k_K^-\}$, the loss is defined as:

$$\mathscr{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(q,k^+)/\tau)}{\exp(\text{sim}(q,k^+)/\tau) + \sum_{j=1}^{K} \exp(\text{sim}(q,k_j^-)/\tau)} \tag{5.11}$$

where $\text{sim}(u,v) = \frac{u^\top v}{\|u\|_2 \|v\|_2}$ denotes cosine similarity, and $\tau > 0$ is a temperature hyperparameter.

Key Insight

Connection to Cross-Entropy and Mutual Information:
The InfoNCE loss is mathematically equivalent to the **multinomial cross-entropy loss of an $(K+1)$-way Softmax classifier** whose task is to correctly classify the positive sample among K distractors. Furthermore, van den Oord et al. proved that minimizing $\mathscr{L}_{\text{InfoNCE}}$ maximizes a rigorous **lower bound on the Mutual Information** $I(X;Y)$ between augmented views:

$$I(X;Y) \geq \log(K+1) - \mathscr{L}_{\text{InfoNCE}} \tag{5.12}$$

A larger dictionary of negative keys K tightens the mutual information bound, directly motivating architectures with massive negative sets.

5.5.3 SimCLR: A Simple Framework for Contrastive Learning

Chen et al. [9] proposed **SimCLR**, formalizing four essential components of modern contrastive learning, as depicted in Figure 5.11:

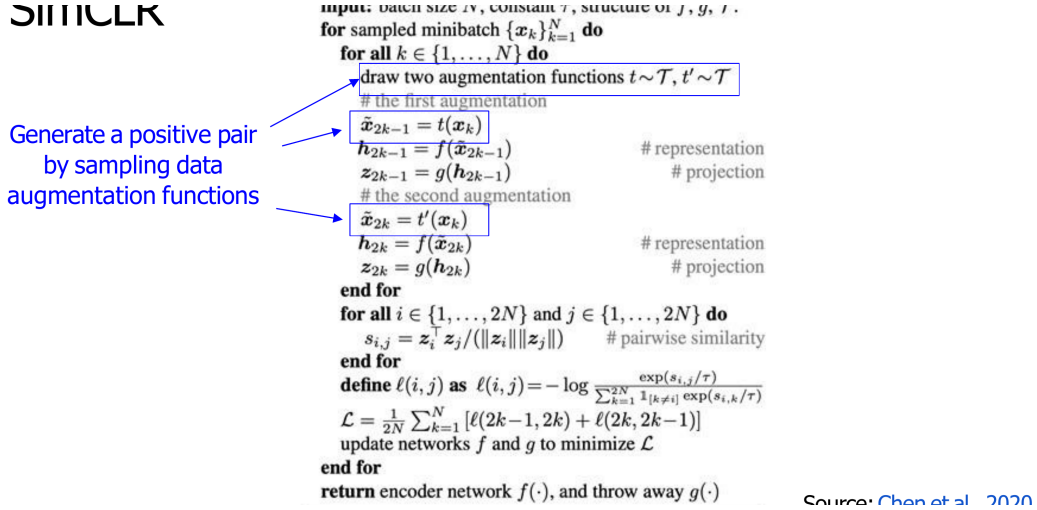


Figure 5.11: The SimCLR framework (from course slides [24, 9]). An image x is transformed into two views $\tilde{x}_i, \tilde{x}_j$. A shared encoder $f(\cdot)$ extracts representations h, which are mapped by a non-linear projection head $g(\cdot)$ to space z where the NT-Xent contrastive loss is evaluated.

1. Composition of Data Augmentations

Data augmentations define the invariance properties of the learned representation. SimCLR established that no single augmentation suffices; the critical synergy requires:

- **Random Crop and Resize:** Forces scale and spatial translation invariance.
- **Random Color Distortion (Color Jittering and Grayscale): Crucial safeguard!** Without color distortion, the model exploits 8-bit color histograms as a shortcut, matching views of the same image based on global tint rather than object geometry.
- **Gaussian Blur:** Removes high-frequency texture shortcuts.

2. The Non-Linear Projection Head

SimCLR maps encoder representations $h = f(x) \in \mathbb{R}^d$ through a small Multi-Layer Perceptron (MLP) **projection head**:

$$z = g(h) = W^{(2)}\sigma(W^{(1)}h) \tag{5.13}$$

The contrastive loss is applied on $z \in \mathbb{R}^{d'}$, but after pre-training, **the projection head** $g(\cdot)$ **is discarded**, and downstream tasks evaluate representations directly on h.

Key Insight

Why Discard the Projection Head?

The contrastive objective forces z to be strictly invariant to data augmentations (such as color shifts, scaling, and rotation). However, downstream tasks (e.g., fine-grained bird classification or bounding-box detection) often require precise color and spatial scale information! The intermediate representation h preserves richer visual features that the non-linear projection layer strips out to minimize the contrastive objective on z.

3. Normalized Temperature-Scaled Cross-Entropy (NT-Xent)

In a mini-batch of N images, data augmentation yields $2N$ views. For any positive pair (i, j) derived from the same source image, the remaining $2(N-1)$ augmented views in the mini-batch serve as negative samples:

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{I}_{[k\neq i]} \exp(\text{sim}(z_i, z_k)/\tau)} \tag{5.14}$$

The total mini-batch loss is computed symmetrically across all $2N$ samples.

4. Large Batch Dependency

Because negatives are sourced strictly within the current mini-batch ($K = 2N-2$), SimCLR requires massive batch sizes ($N = 2048$ to 4096, providing 4094 to 8190 negatives) to achieve optimal performance, necessitating large TPU/GPU cluster parallelization.

5.5.4 Momentum Contrast (MoCo and MoCo v2)

To overcome the large batch requirement of SimCLR, He et al. [33] introduced **Momentum Contrast (MoCo)**, framing contrastive learning as a **continuous dictionary lookup**, shown in Figure 5.12:

MoCo introduces two pivotal architectural mechanisms:

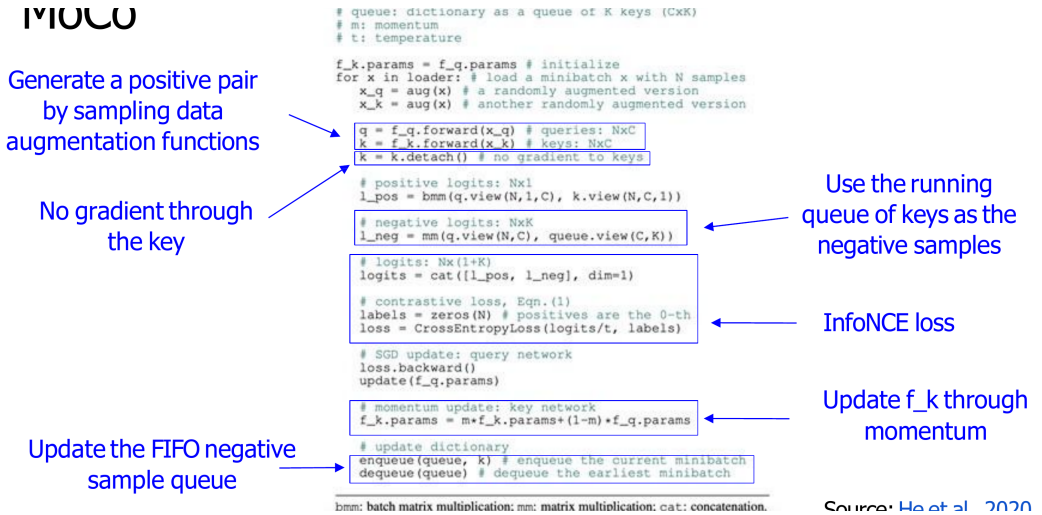


Figure 5.12: The MoCo framework (from course slides [24, 33]). A dynamic FIFO queue decouples negative dictionary size from mini-batch size. A momentum-updated key encoder ensures key representations remain temporally consistent without backpropagating through the entire queue.

1. **Dynamic Memory Queue (Decoupling Dictionary Size):** Instead of sourcing negatives from the current mini-batch, MoCo maintains a large first-in, first-out (FIFO) queue of negative keys ($K = 65{,}536$). At each training step, the current mini-batch keys are enqueued, and the oldest keys are evicted. This decouples dictionary size K from mini-batch size N, allowing massive negative dictionaries with a modest batch size ($N = 256$).
2. **Momentum Encoder Update:** Backpropagating gradients through 65,536 historical keys stored in memory is computationally intractable. However, updating the key encoder weights θ_k directly via SGD causes catastrophic feature drift: keys enqueued at step $t - 100$ were computed with an obsolete encoder and are inconsistent with current query representations.

MoCo solves feature consistency by updating the Key Encoder θ_k via an **Exponential Moving Average (EMA)** of the Query Encoder θ_q:

$$\theta_k \leftarrow m\,\theta_k + (1-m)\,\theta_q \tag{5.15}$$

where $m \approx 0.999$ is a momentum coefficient. The query encoder θ_q is updated by backpropagation; the key encoder θ_k receives **no direct gradients**. The slow, smooth momentum evolution ensures that keys across the 65,536-sample queue remain mutually consistent.

Chen et al. [11] subsequently released **MoCo v2**, integrating SimCLR's non-linear projection head and cosine learning rate decay into the MoCo framework. MoCo v2 surpassed SimCLR while running on standard, affordable GPU hardware without requiring multi-node TPU pods.

—

5.6 Non-Contrastive Siamese Architectures

5.6.1 The Representation Collapse Problem

In contrastive learning, negative pairs supply the repulsive force that prevents **representation collapse**—the degenerate failure mode where all images map to an identical, constant output vector $f(x) = c$.

However, maintaining thousands of negative samples introduces substantial memory and computational overhead. Can a Siamese network learn high-quality representations **without any negative samples**?

5.6.2 BYOL: Bootstrap Your Own Latent

Grill et al. [29] demonstrated that negative samples are not required if the network introduces **architectural asymmetry**. The resulting method, **Bootstrap Your Own Latent (BYOL)**, is illustrated in Figure 5.13:

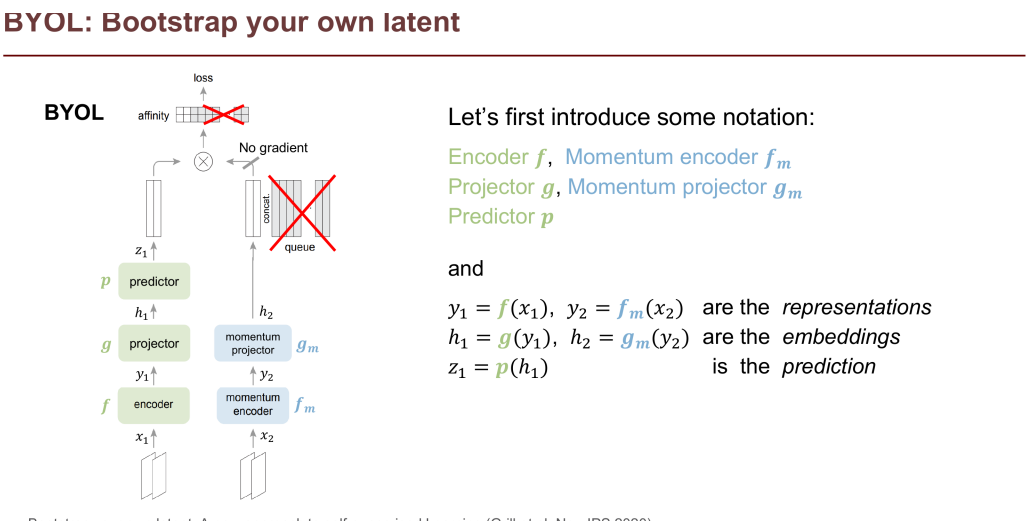


Figure 5.13: The BYOL architecture (from course slides [24, 29]). BYOL eliminates negative pairs entirely by coupling an online network (with an additional predictor head) to an EMA target network with a stop-gradient operator.

BYOL partitions computation between two asymmetric branches:

- **Online Network (θ):** Comprises an encoder f_θ, a projector g_θ, and an additional **predictor head** q_θ:

$$y_1 = f_\theta(x_1), \quad z_1 = g_\theta(y_1), \quad p_1 = q_\theta(z_1) \tag{5.16}$$

- **Target Network (ξ):** Comprises an encoder f_ξ and projector g_ξ, but **no predictor head**:

$$y_2 = f_\xi(x_2), \quad z_2 = g_\xi(y_2) \tag{5.17}$$

The target network parameters ξ are updated via EMA: $\xi \leftarrow \tau\xi + (1-\tau)\theta$, and receive **no gradients** (stopgrad).

The loss is the Mean Squared Error between normalized predictions and normalized target projections:

$$\mathscr{L}_{\text{BYOL}} = 2 - 2\,\frac{\langle p_1, z_2 \rangle}{\|p_1\|_2 \|z_2\|_2} \tag{5.18}$$

The loss is symmetrized by feeding x_2 to the online branch and x_1 to the target branch.

Key Insight

Why Does BYOL Not Collapse?

Collapse is prevented by the dynamic interplay between the predictor head and the EMA target network:

1. The target network acts as an evolving, non-stationary regression target; the online network is constantly chasing a moving representation rather than a static point.
2. The predictor q_θ breaks mathematical symmetry: the online network must predict the target representation from an augmented view, forcing the encoder to capture invariant semantic structure.

5.6.3 SimSiam: Exploring Simple Siamese Representation Learning

Chen and He [10] pushed non-contrastive simplification to its ultimate limit with **SimSiam (Simple Siamese)**, shown in Figure 5.14:

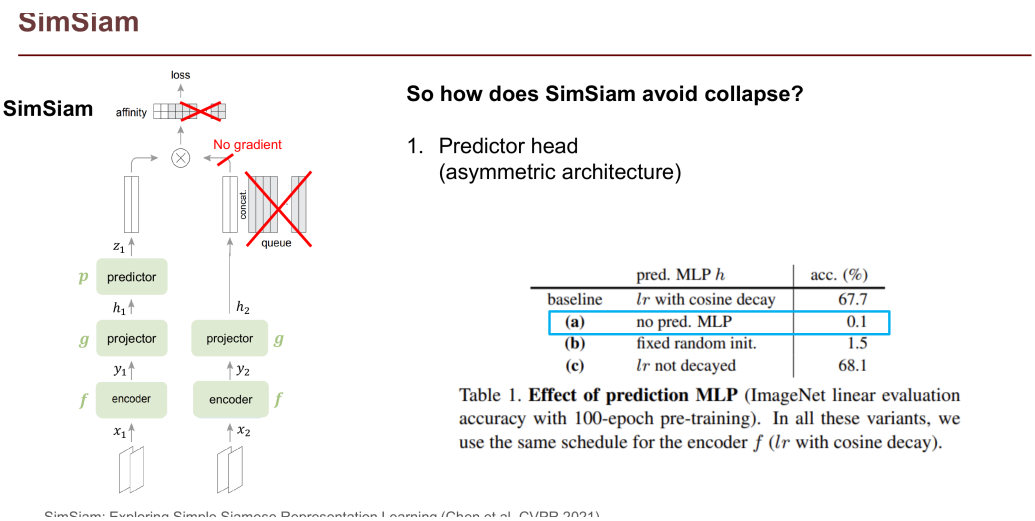


Figure 5.14: The SimSiam architecture (from course slides [24, 10]). SimSiam operates without negative pairs and without momentum encoders, relying solely on an asymmetric predictor head and a stop-gradient operator to prevent collapse.

SimSiam discards **both negative pairs and the momentum encoder**. Both branches share the exact same encoder and projector parameters θ. The only asymmetric element is a predictor head $h(\cdot)$ on one branch and a **stop-gradient** (sg) on the other:

$$\mathscr{D}(p_1, \text{sg}(z_2)) = -\frac{p_1^\top z_2}{\|p_1\|_2 \|z_2\|_2} \tag{5.19}$$

The symmetric loss function is:

$$\mathscr{L}_{\text{SimSiam}} = \frac{1}{2}\mathscr{D}(p_1, \text{stopgrad}(z_2)) + \frac{1}{2}\mathscr{D}(p_2, \text{stopgrad}(z_1)) \tag{5.20}$$

Theoretical Insight: Stop-Gradient as Implicit EM Optimization

Why does SimSiam not collapse? Chen and He hypothesized that the stop-gradient operator transforms the optimization into an implicit **Expectation-Maximization (EM-like) alternating optimization**:

$$\min_{\theta} \mathbb{E}_x \left[\min_{\eta} \mathscr{L}(\theta, \eta; x) \right] \tag{5.21}$$

where η represents an underlying cluster assignment or latent target representation. The inner subproblem evaluates the expectation with fixed θ, while the outer subproblem updates parameters θ. Removing the stop-gradient causes immediate representation collapse ($f(x) = \text{const}$).

5.6.4 DINO: Self-Distillation with No Labels

Caron et al. [8] formulated **DINO (Self-Distillation with No Labels)**, adapting self-supervised learning to Vision Transformers (ViT) via knowledge distillation principles, as shown in Figure 5.15:

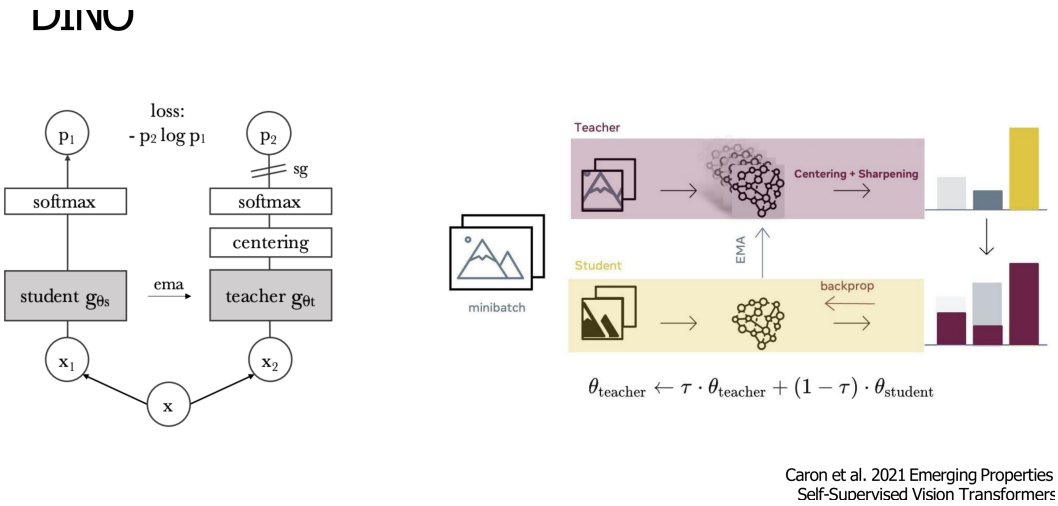


Figure 5.15: DINO self-distillation architecture (from course slides [24, 8]). A student ViT learns to match the output distribution of an EMA teacher ViT. Representation collapse is avoided through centering and temperature sharpening.

A student network g_{θ_s} and a teacher network g_{θ_t} (parameterized via EMA of the student) output probability distributions over K dimensions via Softmax:

$$P_{s,k}(x) = \frac{\exp(g_{\theta_s,k}(x)/\tau_s)}{\sum_{j=1}^{K} \exp(g_{\theta_s,j}(x)/\tau_s)} \tag{5.22}$$

$$P_{t,k}(x) = \frac{\exp((g_{\theta_t,k}(x) - c_k)/\tau_t)}{\sum_{j=1}^{K} \exp((g_{\theta_t,j}(x) - c_j)/\tau_t)} \tag{5.23}$$

The student minimizes cross-entropy with the teacher's predictions:

$$\mathscr{L}_{\text{DINO}} = -\sum_{k=1}^{K} P_{t,k}(x) \log P_{s,k}(x) \tag{5.24}$$

Avoiding Collapse: Centering and Sharpening

Without negative pairs, distillation models risk two distinct failure modes:

1. **Uniform Collapse:** All outputs degenerate into a uniform distribution $P_k(x) = 1/K$.
2. **One-Hot Collapse:** A single dimension dominates, outputting 1 for all inputs and 0 elsewhere.

DINO eliminates both failure modes through dual complementary mechanisms:

- **Centering (c):** A bias vector c tracks the running empirical mean of teacher outputs: $c \leftarrow mc + (1-m)\frac{1}{B}\sum_{i=1}^{B} g_{\theta_t}(x_i)$. Subtracting c prevents any single output dimension from consistently dominating, **preventing one-hot collapse**.
- **Sharpening ($\tau_t < \tau_s$):** Setting a low teacher temperature τ_t (e.g., 0.04 vs $\tau_s = 0.1$) sharpens probabilities into confident distributions, **preventing uniform collapse**.

Emergence of Unsupervised Semantic Segmentation

A remarkable breakthrough of DINO is the emergence of scene segmentation: without any human labels, the self-attention maps of the final $[CLS]$ token in a DINO-trained ViT naturally attend to the boundaries of semantic foreground objects, as shown in Figure 5.16.

DINO

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

Caron et al. 2021 Emerging Properties Self-Supervised Vision Transformers

Figure 5.16: Emergent unsupervised semantic segmentation in DINO (from course slides [24, 8]). The multi-head self-attention maps of a Vision Transformer trained with DINO automatically delineate semantic objects and boundaries without any segmentation labels.

Oquab et al. [44] extended this in **DINOv2**, combining image-level DINO distillation with patch-level masked modeling (iBOT) across hundreds of millions of curated images. DINOv2 generates universal, frozen visual features that achieve state-of-the-art results in monocular depth estimation, dense segmentation, and image retrieval without fine-tuning.

—

5.7 Chapter Summary and Conceptual Graph

5.7.1 Comparative Summary of SSL Paradigms

Table 5.1 summarizes the core algorithmic families analyzed in this chapter:

Paradigm	Key Methods	Negatives?	Anti-Collapse Mechanism	Primary Downstream Strength
Heuristic Pretext	Rotation, Jigsaw, Inpainting, Color	No	Transformation task constraints	Historical proof-of-concept; modest linear probing
Masked Modeling	MAE, Context Encoders	No	High masking ratio (75%) + Pixel reconstruction	Outstanding fine-tuning; state-of-the-art visual prior
Contrastive Learning	SimCLR, MoCo (v1, v2)	Yes	InfoNCE negative repulsion on hypersphere	Exceptional linear probing; invariant global features
Non-Contrastive Siamese	BYOL, SimSiam	No	Predictor head + Stop-gradient / EMA	High efficiency; robust to batch size constraints
Self-Distillation	DINO, DINOv2	No	Centering (mean drift) + Sharpening ($\tau_t < \tau_s$)	Unsupervised semantic discovery; dense correspondence

Table 5.1: Systematic comparison across modern Self-Supervised Learning paradigms.

5.7.2 Logical Roadmap of Representation Learning

Figure 5.17 maps the conceptual progression from supervised limitations to modern self-supervised foundation models:

Key Insight

Connecting the Self-Supervised Paradigms:

- **Supervised → Heuristic:** Replacing manual annotations with data-driven transformations eliminates human labeling but risks low-level heuristic shortcuts.
- **Heuristic → Masked Modeling (MAE):** Elevating inpainting to Vision Transformers with 75% masking forces holistic semantic scene understanding while slashing encoder computational cost.
- **Heuristic → Contrastive Learning:** Replacing domain heuristics with task-agnostic instance discrimination maps semantic views to a unit hypersphere via mutual information maximization.
- **SimCLR → MoCo:** Decoupling dictionary size from mini-batch size via a dynamic FIFO memory queue and momentum encoder enables massive negative sets on standard hardware.
- **Contrastive → Non-Contrastive (BYOL/SimSiam):** Proving negative samples are unnecessary; architectural asymmetry and stop-gradient prevent representation collapse.
- **Non-Contrastive → Self-Distillation (DINO):** Unifying ViT distillation with centering and sharpening yields representations with emergent, unsupervised semantic segmentation capabilities.

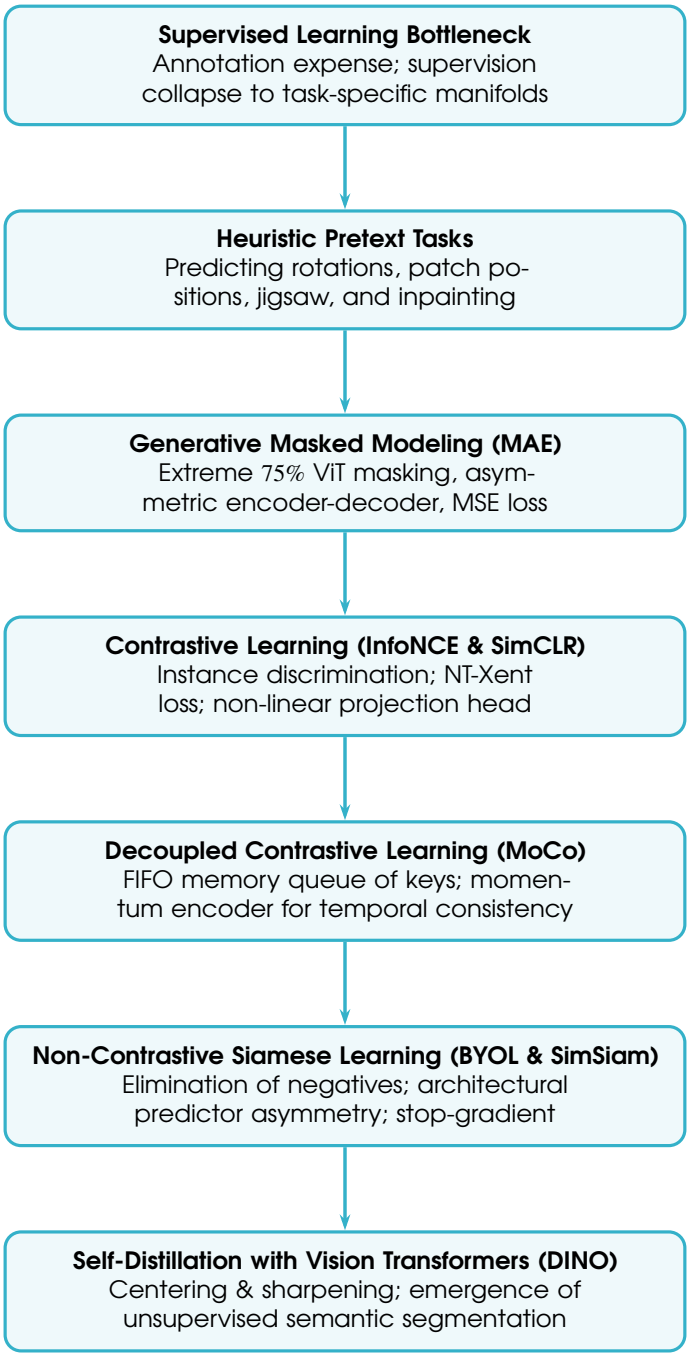


Figure 5.17: Conceptual roadmap of Self-Supervised Learning paradigms.

Bibliography

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. "Layer Normalization". In: *arXiv preprint arXiv:1607.06450* (2016).

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. "Neural Machine Translation by Jointly Learning to Align and Translate". In: *International Conference on Learning Representations (ICLR)*. 2015.

[3] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling". In: *arXiv preprint arXiv:1803.01271* (2018).

[4] Vasileios Belagiannis, Azade Farshad, and Fabio Galasso. "Adversarial Network Compression". In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2018, pp. 434–449.

[5] Antoni Buades, Bartomeu Coll, and Jean-Michel Morel. "A non-local algorithm for image denoising". In: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*. Vol. 2. 2005, pp. 60–65.

[6] Cristian Bucilă, Rich Caruana, and Alexandru Niculescu-Mizil. "Model compression". In: *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 2006, pp. 535–541.

[7] Nicolas Carion et al. "End-to-End Object Detection with Transformers". In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2020, pp. 213–229.

[8] Mathilde Caron et al. "Emerging Properties in Self-Supervised Vision Transformers". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021, pp. 9650–9660.

[9] Ting Chen et al. "A Simple Framework for Contrastive Learning of Visual Representations". In: *Proceedings of the International Conference on Machine Learning (ICML)*. 2020, pp. 1597–1607.

[10] Xinlei Chen and Kaiming He. "Exploring Simple Siamese Representation Learning". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021, pp. 15750–15758.

[11] Xinlei Chen et al. "Improved Baselines with Momentum Contrastive Learning". In: *arXiv preprint arXiv:2003.04297* (2020).

[12] Kyunghyun Cho et al. "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation". In: *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1724–1734.

[13] Kevin Clark et al. “ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators”. In: *International Conference on Learning Representations (ICLR)*. 2020.

[14] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. “Fast and accurate deep network learning by exponential linear units (ELUs)”. In: *arXiv preprint arXiv:1511.07289* (2015).

[15] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. 2019, pp. 4171–4186.

[16] Terrance DeVries and Graham W. Taylor. “Improved Regularization of Convolutional Neural Networks with Cutout”. In: *arXiv preprint arXiv:1708.04552* (2017).

[17] Carl Doersch, Abhinav Gupta, and Alexei A. Efros. “Unsupervised Visual Representation Learning by Context Prediction”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2015, pp. 1422–1430.

[18] Jeffrey Donahue et al. “Long-term Recurrent Convolutional Networks for Visual Recognition and Description”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 2625–2634.

[19] Alexey Dosovitskiy et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *arXiv preprint arXiv:2010.11929* (2020).

[20] Alexei A. Efros and Thomas K. Leung. “Texture synthesis by non-parametric sampling”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. Vol. 2. 1999, pp. 1033–1038.

[21] Haoqi Fan et al. “Multiscale Vision Transformers”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021, pp. 6824–6835.

[22] Fabio Galasso. *Recap on Convolutional Neural Networks*. Lecture Slides, Advanced Machine Learning and Computer Vision, Sapienza Università di Roma. 2025.

[23] Fabio Galasso. *Recap on Image Classification and Neural Networks*. Lecture Slides, Advanced Machine Learning and Computer Vision, Sapienza Università di Roma. 2025.

[24] Fabio Galasso. *Self-Supervised Learning*. Lecture Slides, Advanced Machine Learning and Computer Vision, Sapienza Università di Roma. 2025.

[25] Fabio Galasso. *Sequence Models*. Lecture Slides, Advanced Machine Learning and Computer Vision, Sapienza Università di Roma. 2025.

[26] Fabio Galasso. *Sequence Models II: Transformer Networks*. Lecture Slides, Advanced Machine Learning and Computer Vision, Sapienza Università di Roma. 2025.

[27] Fabio Galasso. *Training Neural Networks*. Lecture Slides, Advanced Machine Learning and Computer Vision, Sapienza Università di Roma. 2025.

[28] Spyros Gidaris, Praveer Singh, and Nikos Komodakis. “Unsupervised Representation Learning by Predicting Image Rotations”. In: *International Conference on Learning Representations (ICLR)*. 2018.

[29] Jean-Bastien Grill et al. “Bootstrap Your Own Latent - A New Approach to Self-Supervised Learning”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 21271–21284.

[30] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.

[31] Kaiming He et al. “Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2015, pp. 1026–1034.

[32] Kaiming He et al. “Masked Autoencoders Are Scalable Vision Learners”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022, pp. 16000–16009.

[33] Kaiming He et al. “Momentum Contrast for Unsupervised Visual Representation Learning”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 9729–9738.

[34] Dan Hendrycks and Kevin Gimpel. “Gaussian Error Linear Units (GELUs)”. In: *arXiv preprint arXiv:1606.08415* (2016).

[35] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. “Distilling the Knowledge in a Neural Network”. In: *arXiv preprint arXiv:1503.02531* (2015).

[36] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[37] Sergey Ioffe and Christian Szegedy. “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”. In: *International Conference on Machine Learning (ICML)*. 2015, pp. 448–456.

[38] Andrej Karpathy. “The Unreasonable Effectiveness of Recurrent Neural Networks”. In: *Andrej Karpathy blog* (2015). URL: `http://karpathy.github.io/2015/05/21/rnn-effectiveness/`.

[39] Andrej Karpathy and Li Fei-Fei. “Deep Visual-Semantic Alignments for Generating Image Descriptions”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 3128–3137.

[40] Ze Liu et al. “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021, pp. 10012–10022.

[41] Andrew L. Maas, Awni Y. Hannun, and Andrew Y. Ng. “Rectifier nonlinearities improve neural network acoustic models”. In: *International Conference on Machine Learning (ICML)*. Vol. 30. 1. 2013, p. 3.

[42] Mehdi Noroozi and Paolo Favaro. “Unsupervised Learning of Visual Representations by Solving Jigsaw Puzzles”. In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2016, pp. 69–84.

[43] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. “Representation Learning with Contrastive Predictive Coding”. In: *arXiv preprint arXiv:1807.03748* (2018).

[44] Maxime Oquab et al. “DINOv2: Learning Robust Visual Features without Supervision”. In: *arXiv preprint arXiv:2304.07193* (2023).

[45] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. “On the difficulty of training recurrent neural networks”. In: *International Conference on Machine Learning (ICML)*. 2013, pp. 1310–1318.

[46] Deepak Pathak et al. “Context Encoders: Feature Learning by Inpainting”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 2536–2544.

[47] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning representations by back-propagating errors”. In: *Nature* 323.6088 (1986), pp. 533–536.

[48] Arthur L. Samuel. “Some Studies in Machine Learning Using the Game of Checkers”. In: *IBM Journal of Research and Development* 3.3 (1959), pp. 210–229.

[49] Karen Simonyan and Andrew Zisserman. “Very Deep Convolutional Networks for Large-Scale Image Recognition”. In: *arXiv preprint arXiv:1409.1556* (2014).

[50] Nitish Srivastava et al. “Dropout: A simple way to prevent neural networks from overfitting”. In: *The Journal of Machine Learning Research* 15.1 (2014), pp. 1929–1958.

[51] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. “Sequence to Sequence Learning with Neural Networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 27. 2014, pp. 3104–3112.

[52] Iulia Turc et al. “Well-Read Students Learn Better: On the Importance of Pre-training Compact Models”. In: *arXiv preprint arXiv:1908.08962* (2019).

[53] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. “Instance Normalization: The Missing Ingredient for Fast Stylization”. In: *arXiv preprint arXiv:1607.08022* (2016).

[54] Ashish Vaswani et al. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017, pp. 5998–6008.

[55] Oriol Vinyals et al. “Show and Tell: A Neural Image Caption Generator”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 3156–3164.

[56] Carl Vondrick et al. “Tracking Emerges by Colorizing Videos”. In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2018, pp. 391–408.

[57] Paul J. Werbos. “Backpropagation through time: what it does and how to do it”. In: *Proceedings of the IEEE* 78.10 (1990), pp. 1550–1560.

[58] Yuxin Wu and Kaiming He. “Group Normalization”. In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2018, pp. 3–19.

[59] Kelvin Xu et al. “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention”. In: *International Conference on Machine Learning (ICML)*. 2015, pp. 2048–2057.

[60] Hongyi Zhang et al. “mixup: Beyond Empirical Risk Minimization”. In: *arXiv preprint arXiv:1710.09412* (2017).

[61] Richard Zhang, Phillip Isola, and Alexei A. Efros. “Colorful Image Colorization”. In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2016, pp. 649–666.

[62] Richard Zhang, Phillip Isola, and Alexei A. Efros. “Split-Brain Autoencoders: Unsupervised Learning by Cross-Channel Prediction”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 1058–1067.